2

PROCESOS E HILOS

Estamos a punto de emprender el estudio detallado de cómo los sistemas operativos son diseñados y construidos. El concepto más importante en cualquier sistema operativo es el de *proceso*, una abstracción de un programa en ejecución; todo lo demás depende de este concepto, por lo cual es importante que el diseñador del sistema operativo (y el estudiante) tenga una comprensión profunda acerca de lo que es un proceso lo más pronto posible.

Los procesos son una de las abstracciones más antiguas e importantes que proporcionan los sistemas operativos: proporcionan la capacidad de operar (pseudo) concurrentemente, incluso cuando hay sólo una CPU disponible. Convierten una CPU en varias CPU virtuales. Sin la abstracción de los procesos, la computación moderna no podría existir. En este capítulo examinaremos con gran detalle los procesos y sus primeros primos, los hilos (*threads*).

2.1 PROCESOS

Todas las computadoras modernas ofrecen varias cosas al mismo tiempo; quienes están acostumbrados a trabajar con ellas tal vez no estén completamente conscientes de este hecho, por lo que utilizaremos algunos ejemplos para aclarar este punto. Consideremos primero un servidor Web, a donde convergen las peticiones de páginas Web provenientes de todos lados. Cuando llega una petición, el servidor verifica si la página que se necesita está en la caché. De ser así, devuelve la página; en caso contrario, inicia una petición al disco para obtenerla y, desde la perspectiva de la CPU, estas peticiones tardan eternidades. Mientras se espera el cumplimiento de una petición, muchas

www.FreeLibros.me

más pueden llegar. Si hay varios discos presentes, algunas o todas las demás peticiones podrían dirigirse a otros discos mucho antes de que se cumpla la primera petición. Es evidente que se necesita cierta forma de modelar y controlar esta concurrencia. Los procesos (y en especial los hilos) pueden ayudar en este caso.

Ahora consideremos una PC de usuario. Cuando se arranca el sistema se inician muchos procesos en forma secreta, lo que a menudo el usuario desconoce. Por ejemplo, se podría iniciar un proceso para esperar el correo electrónico entrante; otro que permite al antivirus comprobar periódicamente la disponibilidad de nuevas definiciones de virus; algunos procesos de usuario explícitos para, por ejemplo, imprimir archivos y quemar un CD-ROM, y todo esto mientras el usuario navega por la Web. Toda esta actividad se tiene que administrar, y en este caso un sistema de multiprogramación con soporte para múltiples procesos es muy útil.

En cualquier sistema de multiprogramación, la CPU conmuta de un proceso a otro con rapidez, ejecutando cada uno durante décimas o centésimas de milisegundos: hablando en sentido estricto, en cualquier instante la CPU está ejecutando sólo un proceso, y en el transcurso de 1 segundo podría trabajar en varios de ellos, dando la apariencia de un paralelismo (o **pseudoparalelismo**, para distinguirlo del verdadero paralelismo de hardware de los sistemas **multiprocesadores** con dos o más CPUs que comparten la misma memoria física). Es difícil para las personas llevar la cuenta de varias actividades en paralelo; por lo tanto, los diseñadores de sistemas operativos han evolucionado con el paso de los años a un modelo conceptual (procesos secuenciales) que facilita el trabajo con el paralelismo. Este modelo, sus usos y algunas de sus consecuencias conforman el tema de este capítulo.

2.1.1 El modelo del proceso

En este modelo, todo el software ejecutable en la computadora, que algunas veces incluye al sistema operativo, se organiza en varios **procesos secuenciales** (**procesos**, para abreviar). Un proceso no es más que una instancia de un programa en ejecución, incluyendo los valores actuales del contador de programa, los registros y las variables. En concepto, cada proceso tiene su propia CPU virtual; en la realidad, la CPU real conmuta de un proceso a otro, pero para entender el sistema es mucho más fácil pensar en una colección de procesos que se ejecutan en (pseudo) paralelo, en vez de tratar de llevar la cuenta de cómo la CPU conmuta de programa en programa. Esta conmutación rápida de un proceso a otro se conoce como **multiprogramación**, como vimos en el capítulo 1.

La figura 2-1(a) muestra una computadora multiprogramando cuatro programas en memoria; la figura 2-1(b) lista cuatro procesos, cada uno con su propio flujo de control (es decir, su propio contador de programa lógico) y cada uno ejecutándose en forma independiente. Desde luego que sólo hay un contador de programa físico, por lo que cuando se ejecuta cada proceso, se carga su contador de programa lógico en el contador de programa real. Cuando termina (por el tiempo que tenga asignado), el contador de programa físico se guarda en el contador de programa lógico almacenado, del proceso en memoria. En la figura 2-1(c) podemos ver que durante un intervalo suficientemente largo todos los procesos han progresado, pero en cualquier momento dado sólo hay un proceso en ejecución.

www.FreeLibros.me

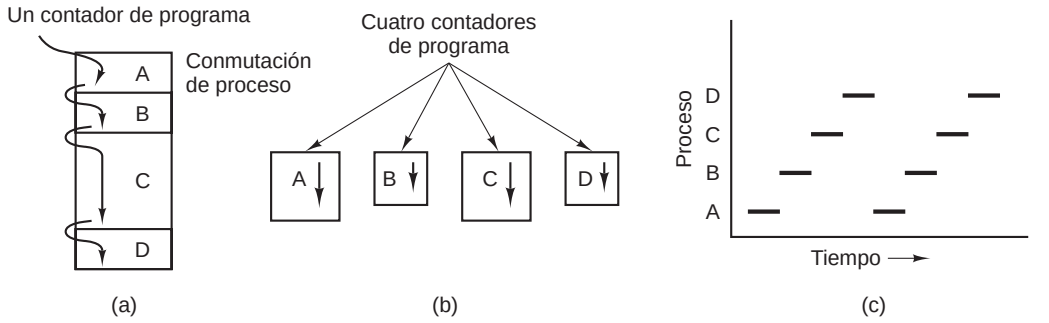


Figura 2-1. (a) Multiprogramación de cuatro programas. (b) Modelo conceptual de cuatro procesos secuenciales independientes. (c) Sólo hay un programa activo a la vez.

En este capítulo vamos a suponer que sólo hay una CPU, aunque esa condición cada vez sucede menos en la realidad debido a que los nuevos chips son multinúcleo, con dos, cuatro o más CPUs. En el capítulo 8 analizaremos los chips multinúcleo y los multiprocesadores en general; mientras tanto es más simple considerar sólo una CPU a la vez. Así, cuando decimos que una CPU puede ejecutar sólo un proceso a la vez, si hay dos núcleos (o CPUs) cada uno de ellos puede ejecutar sólo un proceso a la vez.

Dado que la CPU conmuta rápidamente entre un proceso y otro, la velocidad a la que un proceso ejecuta sus cálculos no es uniforme y tal vez ni siquiera sea reproducible si se ejecutan los mismos procesos de nuevo. Por ende, al programar los procesos debe asumirse esta variación de velocidad. Por ejemplo, considere un proceso de E/S que inicia una unidad de cinta magnética para restaurar los archivos respaldados, ejecuta un ciclo de inactividad 10,000 veces para permitir que obtenga la velocidad adecuada y después emite un comando para leer el primer registro. Si la CPU decide conmutar a otro proceso durante el ciclo de inactividad, el proceso de la cinta tal vez no se ejecute de nuevo sino hasta que el primer registro se encuentre más allá de la cabeza de lectura. Cuando un proceso tiene requerimientos críticos de tiempo real como éste, es decir, cuando *deben* ocurrir eventos específicos dentro de un número especificado de milisegundos, es necesario tomar medidas especiales para asegurar que ocurran. Sin embargo, por lo general la mayoría de los procesos no se ven afectados por la multiprogramación subyacente de la CPU o las velocidades relativas de distintos procesos.

La diferencia entre un proceso y un programa es sutil pero crucial. Aquí podría ayudarnos una analogía: un científico computacional con mente culinaria hornea un pastel de cumpleaños para su hija; tiene la receta para un pastel de cumpleaños y una cocina bien equipada con todos los ingredientes: harina, huevos, azúcar, extracto de vainilla, etcétera. En esta analogía, la receta es el programa (es decir, un algoritmo expresado en cierta notación adecuada), el científico computacional es el procesador (CPU) y los ingredientes del pastel son los datos de entrada. El proceso es la actividad que consiste en que nuestro cocinero vaya leyendo la receta, obteniendo los ingredientes y horneando el pastel.

www.FreeLibros.me

Ahora imagine que el hijo del científico entra corriendo y gritando que una abeja acaba de picarlo. El científico computacional registra el punto de la receta en el que estaba (el estado del proceso en curso se guarda), saca un libro de primeros auxilios y empieza a seguir las instrucciones que contiene. Aquí el procesador conmuta de un proceso (hornear el pastel) a uno de mayor prioridad (administrar cuidados médicos), cada uno de los cuales tiene un programa distinto (la receta y el libro de primeros auxilios). Cuando ya se ha ocupado de la picadura de abeja, el científico computacional regresa a su pastel y continúa en el punto en el que se había quedado.

La idea clave es que un proceso es una actividad de cierto tipo: tiene un programa, una entrada, una salida y un estado. Varios procesos pueden compartir un solo procesador mediante el uso de un algoritmo de planificación para determinar cuándo se debe detener el trabajo en un proceso para dar servicio a otro.

Vale la pena recalcar que si un programa se está ejecutando por duplicado cuenta como dos procesos. Por ejemplo, a menudo es posible iniciar un procesador de palabras dos veces o imprimir dos archivos al mismo tiempo si hay dos impresoras disponibles. El hecho de que dos procesos en ejecución tengan el mismo programa no importa; son procesos distintos. El sistema operativo puede compartir el código entre ellos de manera que sólo haya una copia en la memoria, pero ése es un detalle técnico que no cambia la situación conceptual de dos procesos en ejecución.

2.1.2 Creación de un proceso

Los sistemas operativos necesitan cierta manera de crear procesos. En sistemas muy simples o sistemas diseñados para ejecutar sólo una aplicación (por ejemplo, el controlador en un horno de microondas), es posible tener presentes todos los procesos que se vayan a requerir cuando el sistema inicie. No obstante, en los sistemas de propósito general se necesita cierta forma de crear y terminar procesos según sea necesario durante la operación. Ahora analizaremos varias de estas cuestiones.

Hay cuatro eventos principales que provocan la creación de procesos:

1. El arranque del sistema.
2. La ejecución, desde un proceso, de una llamada al sistema para creación de procesos.
3. Una petición de usuario para crear un proceso.
4. El inicio de un trabajo por lotes.

Generalmente, cuando se arranca un sistema operativo se crean varios procesos. Algunos de ellos son procesos en primer plano; es decir, procesos que interactúan con los usuarios (humanos) y realizan trabajo para ellos. Otros son procesos en segundo plano, que no están asociados con usuarios específicos sino con una función específica. Por ejemplo, se puede diseñar un proceso en segundo plano para aceptar el correo electrónico entrante, que permanece inactivo la mayor parte del día pero que se activa cuando llega un mensaje. Se puede diseñar otro proceso en segundo plano para

www.FreeLibros.me

aceptar peticiones entrantes para las páginas Web hospedadas en ese equipo, que despierte cuando llegue una petición para darle servicio. Los procesos que permanecen en segundo plano para manejar ciertas actividades como correo electrónico, páginas Web, noticias, impresiones, etcétera, se conocen como **demonios** (*daemons*). Los sistemas grandes tienen comúnmente docenas de ellos. En UNIX podemos utilizar el programa *ps* para listar los procesos en ejecución. En Windows podemos usar el administrador de tareas.

Además de los procesos que se crean al arranque, posteriormente se pueden crear otros. A menudo, un proceso en ejecución emitirá llamadas al sistema para crear uno o más procesos nuevos, para que le ayuden a realizar su trabajo. En especial, es útil crear procesos cuando el trabajo a realizar se puede formular fácilmente en términos de varios procesos interactivos relacionados entre sí, pero independientes en los demás aspectos. Por ejemplo, si se va a obtener una gran cantidad de datos a través de una red para su posterior procesamiento, puede ser conveniente crear un proceso para obtener los datos y colocarlos en un búfer compartido, mientras un segundo proceso remueve los elementos de datos y los procesa. En un multiprocesador, al permitir que cada proceso se ejecute en una CPU distinta también se puede hacer que el trabajo se realice con mayor rapidez.

En los sistemas interactivos, los usuarios pueden iniciar un programa escribiendo un comando o haciendo (doble) clic en un icono. Cualquiera de las dos acciones inicia un proceso y ejecuta el programa seleccionado. En los sistemas UNIX basados en comandos que ejecutan X, el nuevo proceso se hace cargo de la ventana en la que se inició. En Microsoft Windows, cuando se inicia un proceso no tiene una ventana, pero puede crear una (o más) y la mayoría lo hace. En ambos sistemas, los usuarios pueden tener varias ventanas abiertas a la vez, cada una ejecutando algún proceso. Mediante el ratón, el usuario puede seleccionar una ventana e interactuar con el proceso, por ejemplo para proveer datos cuando sea necesario.

La última situación en la que se crean los procesos se aplica sólo a los sistemas de procesamiento por lotes que se encuentran en las mainframes grandes. Aquí los usuarios pueden enviar trabajos de procesamiento por lotes al sistema (posiblemente en forma remota). Cuando el sistema operativo decide que tiene los recursos para ejecutar otro trabajo, crea un proceso y ejecuta el siguiente trabajo de la cola de entrada.

Técnicamente, en todos estos casos, para crear un proceso hay que hacer que un proceso existente ejecute una llamada al sistema de creación de proceso. Ese proceso puede ser un proceso de usuario en ejecución, un proceso del sistema invocado mediante el teclado o ratón, o un proceso del administrador de procesamiento por lotes. Lo que hace en todo caso es ejecutar una llamada al sistema para crear el proceso. Esta llamada al sistema indica al sistema operativo que cree un proceso y le indica, directa o indirectamente, cuál programa debe ejecutarlo.

En UNIX sólo hay una llamada al sistema para crear un proceso: `fork`. Esta llamada crea un clon exacto del proceso que hizo la llamada. Después de `fork`, los dos procesos (padre e hijo) tienen la misma imagen de memoria, las mismas cadenas de entorno y los mismos archivos abiertos. Eso es todo. Por lo general, el proceso hijo ejecuta después a `execve` o una llamada al sistema similar para cambiar su imagen de memoria y ejecutar un nuevo programa. Por ejemplo, cuando un usuario escribe un comando tal como *sort* en el shell, éste crea un proceso hijo, que a su vez ejecuta a *sort*. La razón de este proceso de dos pasos es para permitir al hijo manipular sus descriptores de archivo después de `fork`, pero antes de `execve`, para poder lograr la redirección de la entrada estándar, la salida estándar y el error estándar.

www.FreeLibros.me

Por el contrario, en Windows una sola llamada a una función de Win32 (`CreateProcess`) maneja la creación de procesos y carga el programa correcto en el nuevo proceso. Esta llamada tiene 10 parámetros, que incluyen el programa a ejecutar, los parámetros de la línea de comandos para introducir datos a ese programa, varios atributos de seguridad, bits que controlan si los archivos abiertos se heredan, información de prioridad, una especificación de la ventana que se va a crear para el proceso (si se va a crear una) y un apuntador a una estructura en donde se devuelve al proceso que hizo la llamada la información acerca del proceso recién creado. Además de `CreateProcess`, Win32 tiene cerca de 100 funciones más para administrar y sincronizar procesos y temas relacionados.

Tanto en UNIX como en Windows, una vez que se crea un proceso, el padre y el hijo tienen sus propios espacios de direcciones distintos. Si cualquiera de los procesos modifica una palabra en su espacio de direcciones, esta modificación no es visible para el otro proceso. En UNIX, el espacio de direcciones inicial del hijo es una *copia* del padre, pero en definitiva hay dos espacios de direcciones distintos involucrados; no se comparte memoria en la que se pueda escribir (algunas implementaciones de UNIX comparten el texto del programa entre los dos, debido a que no se puede modificar). Sin embargo, es posible para un proceso recién creado compartir algunos de los otros recursos de su creador, como los archivos abiertos. En Windows, los espacios de direcciones del hijo y del padre son distintos desde el principio.

2.1.3 Terminación de procesos

Una vez que se crea un proceso, empieza a ejecutarse y realiza el trabajo al que está destinado. Sin embargo, nada dura para siempre, ni siquiera los procesos. Tarde o temprano el nuevo proceso terminará, por lo general debido a una de las siguientes condiciones:

1. Salida normal (voluntaria).
2. Salida por error (voluntaria).
3. Error fatal (involuntaria).
4. Eliminado por otro proceso (involuntaria).

La mayoría de los procesos terminan debido a que han concluido su trabajo. Cuando un compilador ha compilado el programa que recibe, ejecuta una llamada al sistema para indicar al sistema operativo que ha terminado. Esta llamada es `exit` en UNIX y `ExitProcess` en Windows. Los programas orientados a pantalla también admiten la terminación voluntaria. Los procesadores de palabras, navegadores de Internet y programas similares siempre tienen un icono o elemento de menú en el que el usuario puede hacer clic para indicar al proceso que elimine todos los archivos temporales que tenga abiertos y después termine.

La segunda razón de terminación es que el proceso descubra un error. Por ejemplo, si un usuario escribe el comando

```
cc foo.c
```

www.FreeLibros.me

para compilar el programa *foo.c* y no existe dicho archivo, el compilador simplemente termina. Los procesos interactivos orientados a pantalla por lo general no terminan cuando reciben parámetros incorrectos. En vez de ello, aparece un cuadro de diálogo y se le pide al usuario que intente de nuevo.

La tercera razón de terminación es un error fatal producido por el proceso, a menudo debido a un error en el programa. Algunos ejemplos incluyen el ejecutar una instrucción ilegal, hacer referencia a una parte de memoria no existente o la división entre cero. En algunos sistemas (como UNIX), un proceso puede indicar al sistema operativo que desea manejar ciertos errores por sí mismo, en cuyo caso el proceso recibe una señal (se interrumpe) en vez de terminar.

La cuarta razón por la que un proceso podría terminar es que ejecute una llamada al sistema que indique al sistema operativo que elimine otros procesos. En UNIX esta llamada es `kill`. La función correspondiente en Win32 es `TerminateProcess`. En ambos casos, el proceso eliminador debe tener la autorización necesaria para realizar la eliminación. En algunos sistemas, cuando un proceso termina (ya sea en forma voluntaria o forzosa) todos los procesos que creó se eliminan de inmediato también. Sin embargo, ni Windows ni UNIX trabajan de esta forma.

2.1.4 Jerarquías de procesos

En algunos sistemas, cuando un proceso crea otro, el proceso padre y el proceso hijo continúan asociados en ciertas formas. El proceso hijo puede crear por sí mismo más procesos, formando una jerarquía de procesos. Observe que, a diferencia de las plantas y los animales que utilizan la reproducción sexual, un proceso sólo tiene un padre (pero cero, uno, dos o más hijos).

En UNIX, un proceso y todos sus hijos, junto con sus posteriores descendientes, forman un grupo de procesos. Cuando un usuario envía una señal del teclado, ésta se envía a todos los miembros del grupo de procesos actualmente asociado con el teclado (por lo general, todos los procesos activos que se crearon en la ventana actual). De manera individual, cada proceso puede atrapar la señal, ignorarla o tomar la acción predeterminada que es ser eliminado por la señal.

Como otro ejemplo dónde la jerarquía de procesos juega su papel, veamos la forma en que UNIX se inicializa a sí mismo cuando se enciende la computadora. Hay un proceso especial (llamado *init*) en la imagen de inicio. Cuando empieza a ejecutarse, lee un archivo que le indica cuántas terminales hay. Después utiliza `fork` para crear un proceso por cada terminal. Estos procesos esperan a que alguien inicie la sesión. Si un inicio de sesión tiene éxito, el proceso de inicio de sesión ejecuta un shell para aceptar comandos. Éstos pueden iniciar más procesos y así sucesivamente. Por ende, todos los procesos en el sistema completo pertenecen a un solo árbol, con *init* en la raíz.

En contraste, Windows no tiene un concepto de una jerarquía de procesos. Todos los procesos son iguales. La única sugerencia de una jerarquía de procesos es que, cuando se crea un proceso, el padre recibe un indicador especial un *token* (llamado **manejador**) que puede utilizar para controlar al hijo. Sin embargo, tiene la libertad de pasar este indicador a otros procesos, con lo cual invalida la jerarquía. Los procesos en UNIX no pueden desheredar a sus hijos.

www.FreeLibros.me

2.1.5 Estados de un proceso

Aunque cada proceso es una entidad independiente, con su propio contador de programa y estado interno, a menudo los procesos necesitan interactuar con otros. Un proceso puede generar cierta salida que otro proceso utiliza como entrada. En el comando de shell

```
cat capitulo1 capitulo2 capitulo3 | grep arbol
```

el primer proceso, que ejecuta *cat*, concatena tres archivos y los envía como salida. El segundo proceso, que ejecuta *grep*, selecciona todas las líneas que contengan la palabra "arbol". Dependiendo de la velocidad relativa de los dos procesos (que dependen tanto de la complejidad relativa de los programas, como de cuánto tiempo ha tenido cada uno la CPU), puede ocurrir que *grep* esté listo para ejecutarse, pero que no haya una entrada esperándolo. Entonces debe bloquear hasta que haya una entrada disponible.

Cuando un proceso se bloquea, lo hace debido a que por lógica no puede continuar, comúnmente porque está esperando una entrada que todavía no está disponible. También es posible que un proceso, que esté listo en concepto y pueda ejecutarse, se detenga debido a que el sistema operativo ha decidido asignar la CPU a otro proceso por cierto tiempo. Estas dos condiciones son completamente distintas. En el primer caso, la suspensión está inherente en el problema (no se puede procesar la línea de comandos del usuario sino hasta que éste la haya escrito mediante el teclado). En el segundo caso, es un tecnicismo del sistema (no hay suficientes CPUs como para otorgar a cada proceso su propio procesador privado). En la figura 2-2 podemos ver un diagrama de estados que muestra los tres estados en los que se puede encontrar un proceso:

1. En ejecución (en realidad está usando la CPU en ese instante).
2. Listo (ejecutable; se detuvo temporalmente para dejar que se ejecute otro proceso).
3. Bloqueado (no puede ejecutarse sino hasta que ocurra cierto evento externo).

En sentido lógico, los primeros dos estados son similares. En ambos casos el proceso está deseoso de ejecutarse; sólo en el segundo no hay temporalmente una CPU para él. El tercer estado es distinto de los primeros dos en cuanto a que el proceso no se puede ejecutar, incluso aunque la CPU no tenga nada que hacer.

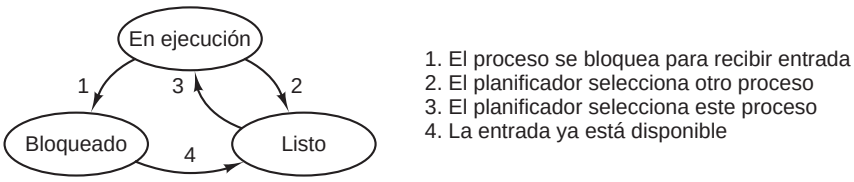


Figura 2-2. Un proceso puede encontrarse en estado "en ejecución", "bloqueado" o "listo". Las transiciones entre estos estados son como se muestran.

Hay cuatro transiciones posibles entre estos tres estados, como se indica. La transición 1 ocurre cuando el sistema operativo descubre que un proceso no puede continuar justo en ese momento. En

www.FreeLibros.me

algunos sistemas el proceso puede ejecutar una llamada al sistema, como pause, para entrar al estado bloqueado. En otros sistemas, incluyendo a UNIX, cuando un proceso lee datos de una canalización o de un archivo especial (como una terminal) y no hay entrada disponible, el proceso se bloquea en forma automática.

Las transiciones 2 y 3 son producidas por el planificador de procesos, una parte del sistema operativo, sin que el proceso sepa siquiera acerca de ellas. La transición 2 ocurre cuando el planificador decide que el proceso en ejecución se ha ejecutado el tiempo suficiente y es momento de dejar que otro proceso tenga una parte del tiempo de la CPU. La transición 3 ocurre cuando todos los demás procesos han tenido su parte del tiempo de la CPU y es momento de que el primer proceso obtenga la CPU para ejecutarse de nuevo. El tema de la planificación de procesos (decidir qué proceso debe ejecutarse en qué momento y por cuánto tiempo) es importante; más adelante en este capítulo lo analizaremos. Se han ideado muchos algoritmos para tratar de balancear las contrastantes demandas de eficiencia para el sistema como un todo y de equidad para los procesos individuales; más adelante en este capítulo estudiaremos algunas.

La transición 4 ocurre cuando se produce el evento externo por el que un proceso estaba esperando (como la llegada de ciertos datos de entrada). Si no hay otro proceso en ejecución en ese instante, se activa la transición 3 y el proceso empieza a ejecutarse. En caso contrario, tal vez tenga que esperar en el estado *listo* por unos instantes, hasta que la CPU esté disponible y sea su turno de utilizarla.

Si utilizamos el modelo de los procesos, es mucho más fácil pensar en lo que está ocurriendo dentro del sistema. Algunos de los procesos ejecutan programas que llevan a cabo los comandos que escribe un usuario; otros son parte del sistema y se encargan de tareas como cumplir con las peticiones de los servicios de archivos o administrar los detalles de ejecutar una unidad de disco o de cinta magnética. Cuando ocurre una interrupción de disco, el sistema toma una decisión para dejar de ejecutar el proceso actual y ejecutar el proceso de disco que está bloqueado esperando esta interrupción. Así, en vez de pensar en las interrupciones, podemos pensar en los procesos de usuario, procesos de disco, procesos de terminal, etc., que se bloquean cuando están esperando a que algo ocurra. Cuando se ha leído el disco o se ha escrito el carácter, el proceso que espera se desbloquea y es elegible para continuar ejecutándose.

Este punto de vista da pie al modelo que se muestra en la figura 2-3. Aquí el nivel más bajo del sistema operativo es el planificador, con un variedad de procesos encima de él. Todo el manejo de las interrupciones y los detalles relacionados con iniciar y detener los procesos se ocultan en lo que aquí se denomina planificador, que en realidad no es mucho código. El resto del sistema operativo está muy bien estructurado en forma de procesos. Sin embargo, pocos sistemas reales están tan bien estructurados como éste.

2.1.6 Implementación de los procesos

Para implementar el modelo de procesos, el sistema operativo mantiene una tabla (un arreglo de estructuras) llamada **tabla de procesos**, con sólo una entrada por cada proceso (algunos autores llaman a estas entradas **bloques de control de procesos**). Esta entrada contiene información importante acerca del estado del proceso, incluyendo su contador de programa, apuntador de pila, asignación de memoria, estado de sus archivos abiertos, información de contabilidad y planificación, y todo lo de-

www.FreeLibros.me

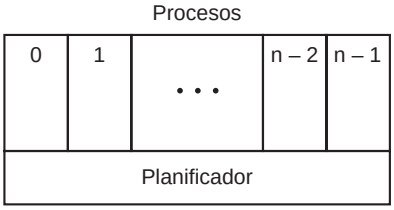


Figura 2-3. La capa más baja de un sistema operativo estructurado por procesos se encarga de las interrupciones y la planificación. Encima de esa capa están los procesos secuenciales.

más que debe guardarse acerca del proceso cuando éste cambia del estado *en ejecución* a *listo* o *bloqueado*, de manera que se pueda reiniciar posteriormente como si nunca se hubiera detenido.

La figura 2-4 muestra algunos de los campos clave en un sistema típico. Los campos en la primera columna se relacionan con la administración de procesos; los otros dos se relacionan con la administración de memoria y archivos, respectivamente. Hay que recalcar que los campos contenidos en la tabla de procesos varían de un sistema a otro, pero esta figura nos da una idea general de los tipos de información necesaria.

Administración de procesos	**Administración de memoria**	**Administración de archivos**
Registros	Apuntador a la información del segmento de texto	Directorio raíz
Contador del programa	Apuntador a la información del segmento de datos	Directorio de trabajo
Palabra de estado del programa	Apuntador a la información del segmento de pila	Descripciones de archivos
Apuntador de la pila		ID de usuario
Estado del proceso		ID de grupo
Prioridad		
Parámetros de planificación		
ID del proceso		
Proceso padre		
Grupo de procesos		
Señales		
Tiempo de inicio del proceso		
Tiempo utilizado de la CPU		
Tiempo de la CPU utilizado por el hijo		
Hora de la siguiente alarma		

Figura 2-4. Algunos de los campos de una entrada típica en la tabla de procesos.

Ahora que hemos analizado la tabla de procesos, es posible explicar un poco más acerca de cómo la ilusión de varios procesos secuenciales se mantiene en una (o en varias) CPU. Con cada clase de E/S hay una ubicación asociada (por lo general, en una ubicación cerca de la parte final de la memoria), a la cual se le llama **vector de interrupción**. Esta ubicación contiene la dirección del procedimiento del servicio de interrupciones. Suponga que el proceso de usuario 3 está en ejecución cuando ocurre una interrupción de disco. El contador de programa, la palabra de estado

www.FreeLibros.me

del programa y algunas veces uno o más registros del proceso del usuario 3 se meten en la pila (actual) mediante el hardware de interrupción. Después, la computadora salta a la dirección especificada en el vector de interrupción. Esto es todo lo que hace el hardware. De aquí en adelante, depende del software y en especial del procedimiento del servicio de interrupciones.

Todas las interrupciones empiezan por guardar los registros, a menudo en la entrada de la tabla de procesos para el proceso actual. Después, se quita la información que la interrupción metió en la pila y el apuntador de pila se establece para que apunte a una pila temporal utilizada por el manejador de procesos. Las acciones como guardar los registros y establecer el apuntador de pila no se pueden expresar ni siquiera en lenguajes de alto nivel tales como C, por lo que se realizan mediante una pequeña rutina en lenguaje ensamblador, que por lo general es la misma para todas las interrupciones, ya que el trabajo de guardar los registros es idéntico, sin importar cuál sea la causa de la interrupción.

Cuando termina esta rutina, llama a un procedimiento en C para realizar el resto del trabajo para este tipo de interrupción específico (suponemos que el sistema operativo está escrito en C, la elección usual para todos los sistemas operativos reales). Cuando ha terminado su trabajo y tal vez ocasionando que algún otro proceso esté entonces listo, el planificador es llamado para ver qué proceso se debe ejecutar a continuación. Después de eso, el control se pasa de vuelta al código en lenguaje ensamblador para cargar los registros y el mapa de memoria para el proceso que entonces es el actual y se empieza a ejecutar. En la figura 2-5 se sintetizan el manejo de interrupciones y la planificación de proceso. Vale la pena recalcar que los detalles varían dependiendo del sistema.

1. El hardware mete el contador del programa a la pila, etc.
2. El hardware carga el nuevo contador de programa del vector de interrupciones.
3. Procedimiento en lenguaje ensamblador guarda los registros.
4. Procedimiento en lenguaje ensamblador establece la nueva pila.
5. El servicio de interrupciones de C se ejecuta (por lo general lee y guarda la entrada en el búfer).
6. El planificador decide qué proceso se va a ejecutar a continuación.
7. Procedimiento en C regresa al código de ensamblador.
8. Procedimiento en lenguaje ensamblador inicia el nuevo proceso actual.

Figura 2-5. Esqueleto de lo que hace el nivel más bajo del sistema operativo cuando ocurre una interrupción.

Cuando el proceso termina, el sistema operativo muestra un carácter indicador y espera un nuevo comando. Cuando recibe el comando, carga un nuevo programa en memoria, sobrescribiendo el anterior.

2.1.7 Modelación de la multiprogramación

Cuando se utiliza la multiprogramación, el uso de la CPU se puede mejorar. Dicho en forma cruda: si el proceso promedio realiza cálculos sólo 20 por ciento del tiempo que está en la memoria, con cinco procesos en memoria a la vez la CPU deberá estar ocupada todo el tiempo. Sin embargo, este modelo es demasiado optimista, ya que supone que los cinco procesos nunca estarán esperando la E/S al mismo tiempo.

www.FreeLibros.me

Un mejor modelo es analizar el uso de la CPU desde un punto de vista probabilístico. Suponga que un proceso gasta una fracción *p* de su tiempo esperando a que se complete una operación de E/S. Con *n* procesos en memoria a la vez, la probabilidad de que todos los *n* procesos estén esperando la E/S (en cuyo caso, la CPU estará inactiva) es p^n. Entonces, el uso de la CPU se obtiene mediante la fórmula

$$\text{Uso de la CPU} = 1 - p^n$$

La figura 2-6 muestra el uso de la CPU como una función de *n*, a lo cual se le conoce como el **grado de multiprogramación**.

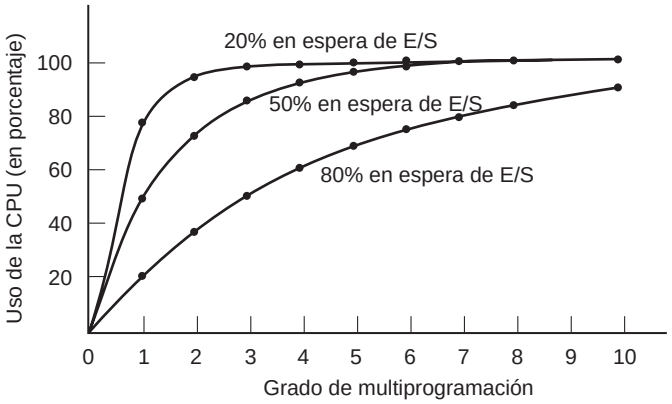


Figura 2-6. Uso de la CPU como una función del número de procesos en memoria.

La figura deja claro que, si los procesos gastan 80 por ciento de su tiempo esperando las operaciones de E/S, por lo menos debe haber 10 procesos en memoria a la vez para que el desperdicio de la CPU esté por debajo de 10%. Cuando nos damos cuenta de que un proceso interactivo, que espera a que un usuario escriba algo en una terminal, está en un estado de espera de E/S, debe estar claro que los tiempos de espera de E/S de 80 por ciento o más no son poco comunes. Pero incluso en los servidores, los procesos que realizan muchas operaciones de E/S en disco tendrán a menudo este porcentaje o más.

Para obtener una precisión completa, debemos recalcar que el modelo probabilístico que acabamos de describir es sólo una aproximación. Supone en forma implícita que los *n* procesos son independientes, lo cual significa que es bastante aceptable para un sistema con cinco procesos en memoria, tener tres en ejecución y dos en espera. Pero con una sola CPU no podemos tener tres procesos en ejecución a la vez, por lo que un proceso que cambie al estado listo mientras la CPU esté ocupada tendrá que esperar. Por ende, los procesos no son independientes. Podemos construir un modelo más preciso mediante la teoría de colas, pero el punto que queremos establecer (la multiprogramación permite que los procesos utilicen la CPU cuando de lo contrario estaría inactiva) es, desde luego, todavía válido, aun si las curvas verdaderas de la figura 2-6 son ligeramente distintas de las que se muestran en la figura.

www.FreeLibros.me

Aun cuando el modelo de la figura 2-6 es simple, de todas formas se puede utilizar para realizar predicciones específicas (aunque aproximadas) acerca del rendimiento de la CPU. Por ejemplo, suponga que una computadora tiene 512 MB de memoria, de la cual el sistema operativo ocupa 128 MB y cada programa de usuario ocupa otros 128 MB. Estos tamaños permiten que haya tres programas de usuario en memoria a la vez. Con un promedio de 80 por ciento de tiempo de espera de E/S, tenemos una utilización de la CPU (ignorando la sobrecarga del sistema operativo) de $1 - 0.8^3$ o de aproximadamente 49 por ciento. Si agregamos 512 MB más de memoria, el sistema puede pasar de la multiprogramación de tres vías a una multiprogramación de siete vías, con lo cual el uso de la CPU se eleva hasta 79 por ciento. En otras palabras, los 512 MB adicionales elevarán el rendimiento por 30 por ciento.

Si agregamos otros 512 MB, el uso de la CPU sólo se incrementaría de 79 a 91 por ciento, con lo cual se elevaría el rendimiento sólo en 12% adicional. Utilizando este modelo, el propietario de la computadora podría decidir que la primera adición es una buena inversión, pero la segunda no.

2.2 HILOS

En los sistemas operativos tradicionales, cada proceso tiene un espacio de direcciones y un solo hilo de control. De hecho, ésa es casi la definición de un proceso. Sin embargo, con frecuencia hay situaciones en las que es conveniente tener varios hilos de control en el mismo espacio de direcciones que se ejecuta en cuasi-paralelo, como si fueran procesos (casi) separados (excepto por el espacio de direcciones compartido). En las siguientes secciones hablaremos sobre estas situaciones y sus implicaciones.

2.2.1 Uso de hilos

¿Por qué alguien querría tener un tipo de proceso dentro de otro proceso? Resulta ser que hay varias razones de tener estos miniprocesos, conocidos como **hilos**. Ahora analizaremos algunos de ellos. La principal razón de tener hilos es que en muchas aplicaciones se desarrollan varias actividades a la vez. Algunas de ésas se pueden bloquear de vez en cuando. Al descomponer una aplicación en varios hilos secuenciales que se ejecutan en cuasi-paralelo, el modelo de programación se simplifica.

Ya hemos visto este argumento antes: es precisamente la justificación de tener procesos. En vez de pensar en interrupciones, temporizadores y conmutaciones de contexto, podemos pensar en procesos paralelos. Sólo que ahora con los hilos agregamos un nuevo elemento: la habilidad de las entidades en paralelo de compartir un espacio de direcciones y todos sus datos entre ellas. Esta habilidad es esencial para ciertas aplicaciones, razón por la cual no funcionará el tener varios procesos (con sus espacios de direcciones separados).

Un segundo argumento para tener hilos es que, como son más ligeros que los procesos, son mas fáciles de crear (es decir, rápidos) y destruir. En muchos sistemas, la creación de un hilo es de 10 a 100 veces más rápida que la de un proceso. Cuando el número de hilos necesarios cambia de manera dinámica y rápida, es útil tener esta propiedad.

www.FreeLibros.me

Una tercera razón de tener hilos es también un argumento relacionado con el rendimiento. Los hilos no producen un aumento en el rendimiento cuando todos ellos están ligados a la CPU, pero cuando hay una cantidad considerable de cálculos y operaciones de E/S, al tener hilos estas actividades se pueden traslapar, con lo cual se agiliza la velocidad de la aplicación.

Por último, los hilos son útiles en los sistemas con varias CPUs, en donde es posible el verdadero paralelismo. En el capítulo 8 volveremos a ver esta cuestión.

Es más fácil ver por qué los hilos son útiles si utilizamos ejemplos concretos. Como primer ejemplo considere un procesador de palabras. Por lo general, los procesadores de palabras muestran el documento que se va crear en la pantalla exactamente como aparecerá en la página impresa. En especial, todos los saltos de línea y de página están en sus posiciones correctas y finales, de manera que el usuario pueda inspeccionarlas y cambiar el documento si es necesario (por ejemplo, para eliminar *viudas* y *huérfanas*, líneas superiores e inferiores incompletas que se consideran estéticamente desagradables).

Suponga que el usuario está escribiendo un libro. Desde el punto de vista del autor, es más fácil mantener todo el libro en un solo archivo para facilitar la búsqueda de temas, realizar sustituciones globales, etc. También, cada capítulo podría estar en un archivo separado; sin embargo, tener cada sección y subsección como un archivo separado puede ser una verdadera molestia si hay que realizar cambios globales en todo el libro, ya que entonces tendrían que editarse cientos de archivos en forma individual. Por ejemplo, si se aprueba el estándar xxxx propuesto justo antes de que el libro vaya a la imprenta, todas las ocurrencias del "Estándar xxxx en borrador" tendrían que cambiarse por "Estándar xxxx" a última hora. Si todo el libro está en un archivo, por lo general un solo comando puede realizar todas las sustituciones. Por el contrario, si el libro está esparcido en más de 300 archivos, cada uno se debe editar por separado.

Ahora considere lo que ocurre cuando el usuario repentinamente elimina un enunciado de la página 1 de un documento de 800 páginas. Después de revisar que la página modificada esté correcta, el usuario desea realizar otro cambio en la página 600 y escribe un comando que indica al procesador de palabras que vaya a esa página (posiblemente mediante la búsqueda de una frase que sólo esté en esa página). Entonces, el procesador de palabras tiene que volver a dar formato a todo el libro hasta la página 600 en ese momento, debido a que no sabe cuál será la primera línea de la página 600 sino hasta que haya procesado las demás páginas. Puede haber un retraso considerable antes de que pueda mostrar la página 600 y el usuario estaría descontento.

Aquí pueden ayudar los hilos. Suponga que el procesador de palabras se escribe como un programa con dos hilos. Un hilo interactúa con el usuario y el otro se encarga de volver a dar formato en segundo plano. Tan pronto como se elimina el enunciado de la página 1, el hilo interactivo indica al hilo encargado de volver a dar formato que aplique de nuevo formato a todo el libro. Mientras tanto, el hilo interactivo sigue esperando las acciones del teclado y del ratón y responde a comandos simples como desplazarse por la página 1, mientras el otro hilo está realizando cálculos intensivos en segundo plano. Con algo de suerte, el proceso de volver a dar formato se completará antes de que el usuario pida ver la página 600, para que pueda mostrarse al instante.

Ya que estamos en ello, ¿por qué no agregar un tercer hilo? Muchos procesadores de palabras tienen la característica de guardar de manera automática todo el archivo en el disco cada cierto número de minutos, para proteger al usuario contra la pérdida de todo un día de trabajo en caso de un fallo en el programa, el sistema o la energía. El tercer hilo se puede encargar de los

www.FreeLibros.me

respaldos en disco sin interferir con los otros dos. En la figura 2-7 se muestra esta situación con tres hilos.

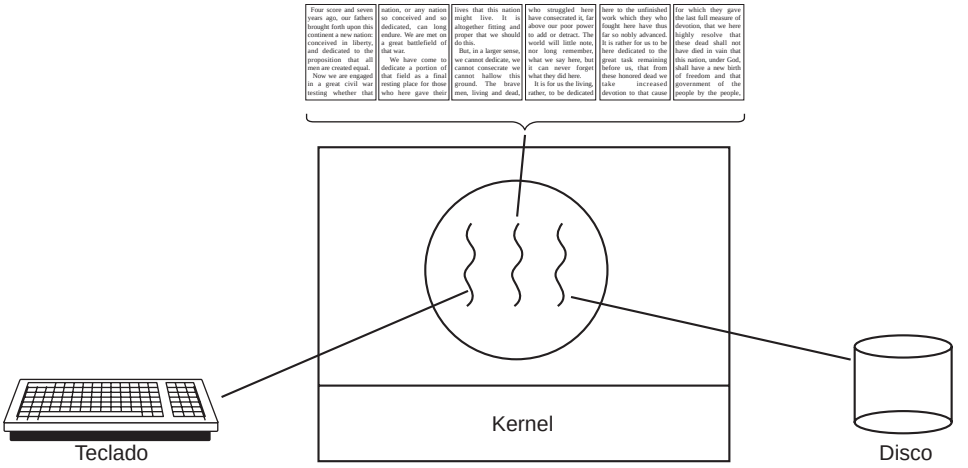


Figura 2-7. Un procesador de palabras con tres hilos.

Si el programa tuviera sólo un hilo, entonces cada vez que iniciara un respaldo en el disco se ignorarían los comandos del teclado y el ratón hasta que se terminara el respaldo. El usuario sin duda consideraría esto como un rendimiento pobre. De manera alternativa, los eventos de teclado y ratón podrían interrumpir el respaldo en disco, permitiendo un buen rendimiento pero produciendo un modelo de programación complejo, controlado por interrupciones. Con tres hilos, el modelo de programación es mucho más simple. El primer hilo interactúa sólo con el usuario, el segundo proceso vuelve a dar formato al documento cuando se le indica y el tercero escribe el contenido de la RAM al disco en forma periódica.

Debemos aclarar que aquí no funcionaría tener tres procesos separados, ya que los tres hilos necesitan operar en el documento. Al tener tres hilos en vez de tres procesos, comparten una memoria común y por ende todos tienen acceso al documento que se está editando.

Hay una situación parecida con muchos otros programas interactivos. Por ejemplo, una hoja de cálculo electrónica es un programa que permite a un usuario mantener una matriz, de la cual algunos elementos son datos proporcionados por el usuario. Otros elementos se calculan con base en los datos de entrada, usando fórmulas potencialmente complejas. Cuando un usuario modifica un elemento, tal vez haya que recalcular muchos otros elementos. Al tener un hilo en segundo plano que realice esos cálculos, el hilo interactivo puede permitir al usuario realizar modificaciones adicionales mientras el cálculo se está llevando a cabo. De manera similar, un tercer hilo se puede encargar de los respaldos periódicos al disco por sí solo.

Ahora considere otro ejemplo más de la utilidad de los hilos: un servidor para un sitio en World Wide Web. Las solicitudes de páginas llegan y la página solicitada se envía de vuelta al cliente. En la mayoría de los sitios Web, algunas páginas se visitan con más frecuencia que otras. Por ejemplo,

www.FreeLibros.me

la página de inicio de Sony recibe muchas más visitas que una página a varios niveles de profundidad en el árbol que contenga las especificaciones técnicas de cualquier cámara de video. Los servidores Web utilizan este hecho para mejorar el rendimiento, al mantener una colección de páginas de uso frecuente en la memoria principal para eliminar la necesidad de ir al disco a obtenerlas. A dicha colección se le conoce como **caché** y se utiliza en muchos otros contextos también. Por ejemplo, en el capítulo 1 vimos las cachés de la CPU.

En la figura 2-8(a) se muestra una forma de organizar el servidor Web. Aquí un hilo, el **despachador**, lee las peticiones entrantes de trabajo de la red. Después de examinar la solicitud, selecciona un **hilo trabajador** inactivo (es decir, bloqueado) y le envía la solicitud, tal vez escribiendo un apuntador al mensaje en una palabra especial asociada con cada hilo. Después, el despachador despierta al trabajador inactivo y lo pasa del estado bloqueado al estado listo.

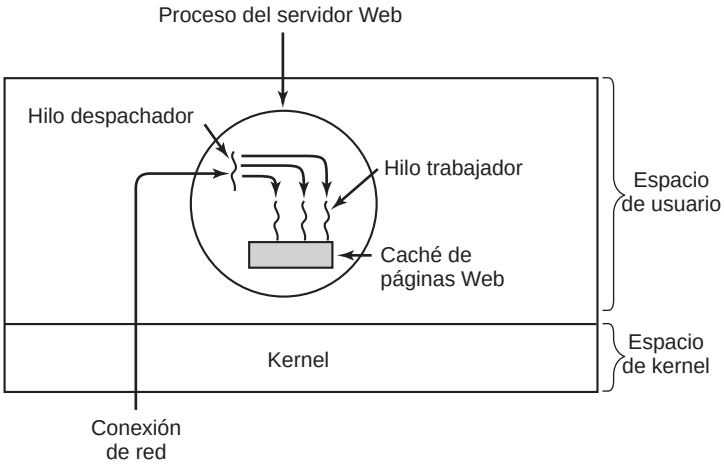


Figura 2-8. Un servidor Web con múltiples hilos.

Cuando el trabajador se despierta, comprueba si la petición se puede satisfacer desde la caché de páginas Web, a la que todos los hilos tienen acceso. De no ser así, inicia una operación `read` para obtener la página del disco y se bloquea hasta que se complete la operación de disco. Cuando el hilo se bloquea en la operación de disco, se selecciona otro hilo para ejecutarlo, posiblemente el despachador, para adquirir más trabajo o tal vez otro trabajador que queda listo para ejecutarse.

Este modelo permite escribir en el servidor en forma de una colección de hilos secuenciales. El programa del despachador consiste en un ciclo infinito para obtener una petición de trabajo y enviarla a un trabajador. El código de cada trabajador consiste en un ciclo infinito que se encarga de aceptar una petición del despachador y comprobar la caché de páginas Web para ver si la página está presente. De ser así, se devuelve al cliente y el trabajador se bloquea para esperar una nueva petición.

www.FreeLibros.me

En la figura 2-9 se muestra un bosquejo del código. Aquí, como en el resto de este libro, se supone que *TRUE* es la constante 1. Además, *buf* y *pagina* son estructuras apropiadas para contener una petición de trabajo y una página Web, respectivamente.

```
while (TRUE) {
    obtener_siguiente_peticion(&buf);
    pasar_trabajo(&buf);
}
```

(a)

```
while (TRUE) {
    esperar_trabajo(&buf)
    buscar_pagina_en_cache(&buf,&pagina);
    if (pagina_no_esta_en_cache(&pagina))
        leer_pagina_de_disco(&buf, &pagina);
    devolver_pagina(&pagina);
}
```

(b)

Figura 2-9. Un bosquejo del código para la figura 2-8. (a) Hilo despachador. (b) Hilo trabajador.

Considere la forma en que podría escribirse el servidor Web sin hilos. Una posibilidad es hacer que opere como un solo hilo. El ciclo principal del servidor Web recibe una petición, la examina y la lleva a cabo hasta completarla antes de obtener la siguiente. Mientras espera al disco, el servidor está inactivo y no procesa otras peticiones entrantes. Si el servidor Web se está ejecutando en una máquina dedicada, como es común el caso, la CPU simplemente está inactiva mientras el servidor Web espera al disco. El resultado neto es que se pueden procesar menos peticiones/segundo. Por ende, los hilos obtienen un aumento considerable en el rendimiento, pero cada hilo se programa de manera secuencial, en forma usual.

Hasta ahora hemos visto dos posibles diseños: un servidor Web con multihilado y un servidor Web con un solo hilo. Suponga que no hay hilos disponibles, pero que los diseñadores del sistema consideran que la pérdida de rendimiento debido a un solo hilo es inaceptable. Si hay una versión sin bloqueo de la llamada al sistema `read` disponible, es posible un tercer diseño. Cuando entra una petición, el único hilo existente la examina. Si se puede satisfacer desde la caché está bien, pero si no es posible, se inicia una operación de disco sin bloqueo.

El servidor registra el estado de la petición actual en una tabla y después pasa a obtener el siguiente evento. Éste puede ser una petición para un nuevo trabajo o una respuesta del disco acerca de una operación anterior. Si es un nuevo trabajo, se inicia; si es una respuesta del disco, la información relevante se obtiene de la tabla y se procesa la respuesta. Con E/S de disco sin bloqueo, una respuesta probablemente tendrá que tomar la forma de una señal o interrupción.

En este diseño, el modelo de "proceso secuencial" que tuvimos en los primeros dos casos se pierde. El estado del cálculo debe guardarse y restaurarse de manera explícita en la tabla, cada vez que el servidor cambia de trabajar en una petición a otra. En efecto, estamos simulando los hilos y sus pilas de la manera difícil. Un diseño como éste, en el que cada cálculo tiene un estado de guardado y existe cierto conjunto de eventos que pueden ocurrir para cambiar el estado, se conoce como **máquina de estados finitos**. Este concepto se utiliza ampliamente en las ciencias computacionales.

www.FreeLibros.me

Ahora el lector debe tener claro qué ofrecen los hilos: posibilitan el concepto de procesos secuenciales que realizan llamadas al sistema con bloqueo (por ejemplo, para la E/S de disco) y de todas formas logran un paralelismo. Las llamadas al sistema con bloqueo facilitan el proceso de programación y el paralelismo mejora el rendimiento. El servidor de un solo hilo retiene la simpleza de las llamadas al sistema con bloqueo, pero pierde rendimiento. El tercer método logra un alto rendimiento a través del paralelismo, pero utiliza llamadas e interrupciones sin bloqueo y por ende, es difícil de programar. En la figura 2-10 se sintetizan estos modelos.

Modelo	Características
Hilos	Paralelismo, llamadas al sistema con bloqueo
Proceso con un solo hilo	Sin paralelismo, llamadas al sistema con bloqueo
Máquina de estados finitos	Paralelismo, llamadas al sistema sin bloqueo, interrupciones

Figura 2-10. Tres formas de construir un servidor.

Un tercer ejemplo en donde los hilos son de utilidad es en las aplicaciones que deben procesar cantidades muy grandes de datos. El método normal es leer un bloque de datos, procesarlo y después escribirlo nuevamente como salida. El problema aquí es que, si sólo hay disponibles llamadas al sistema con bloqueo, el proceso se bloquea mientas los datos están entrando y saliendo. Hacer que la CPU quede inactiva cuando hay muchos cálculos que realizar es sin duda muy ineficiente y debemos evitarlo siempre que sea posible.

Los hilos ofrecen una solución. El proceso podría estructurarse con un hilo de entrada, un hilo de procesamiento y un hilo de salida. El hilo de entrada lee datos y los coloca en un búfer de entrada. El hilo de procesamiento toma datos del búfer de entrada, los procesa y coloca los resultados en un búfer de salida. El búfer de salida escribe estos resultados de vuelta en el disco. De esta forma, todas las operaciones de entrada, salida y procesamiento pueden estar ocurriendo al mismo tiempo. Desde luego que este modelo sólo funciona si una llamada al sistema bloquea sólo al hilo que hizo la llamada, no a todo el proceso.

2.2.2 El modelo clásico de hilo

Ahora que hemos visto por qué podrían ser útiles los hilos y cómo se pueden utilizar, vamos a investigar la idea un poco más de cerca. El modelo de procesos se basa en dos conceptos independientes: agrupamiento de recursos y ejecución. Algunas veces es útil separarlos; aquí es donde entran los hilos. Primero analizaremos el modelo clásico de hilos; después el modelo de hilos de Linux, que desaparece la línea entre los procesos y los hilos.

Una manera de ver a un proceso es como si fuera una forma de agrupar recursos relacionados. Un proceso tiene un espacio de direcciones que contiene texto y datos del programa, así como otros recursos. Estos pueden incluir archivos abiertos, procesos hijos, alarmas pendientes, manejadores de señales, información contable y mucho más. Al reunirlos en forma de un proceso, pueden administrarse con más facilidad.

www.FreeLibros.me

El otro concepto que tiene un proceso es un hilo de ejecución, al que por lo general sólo se le llama **hilo**. El hilo tiene un contador de programa que lleva el registro de cuál instrucción se va a ejecutar a continuación. Tiene registros que contienen sus variables de trabajo actuales. Tiene una pila, que contiene el historial de ejecución, con un conjunto de valores para cada procedimiento al que se haya llamado, pero del cual no se haya devuelto todavía. Aunque un hilo se debe ejecutar en cierto proceso, el hilo y su proceso son conceptos distintos y pueden tratarse por separado. Los procesos se utilizan para agrupar los recursos; son las entidades planificadas para su ejecución en la CPU.

Lo que agregan los hilos al modelo de procesos es permitir que se lleven a cabo varias ejecuciones en el mismo entorno del proceso, que son en gran parte independientes unas de las otras. Tener varios procesos ejecutándose en paralelo en un proceso es algo similar a tener varios procesos ejecutándose en paralelo en una computadora. En el primer caso, los hilos comparten un espacio de direcciones y otros recursos; en el segundo, los procesos comparten la memoria física, los discos, las impresoras y otros recursos. Como los hilos tienen algunas de las propiedades de los procesos, algunas veces se les llama **procesos ligeros**. El término **multihilamiento** también se utiliza para describir la situación de permitir varios hilos en el mismo proceso. Como vimos en el capítulo 1, algunas CPUs tienen soporte directo en el hardware para el **multihilamiento** y permiten que las conmutaciones de hilos ocurran en una escala de tiempo en nanosegundos.

En la figura 2-11(a) podemos ver tres procesos tradicionales. Cada proceso tiene su propio espacio de direcciones y un solo hilo de control. Por el contrario, en la figura 2-11(b) vemos un solo proceso con tres hilos de control. Aunque en ambos casos tenemos tres hilos, en la figura 2-11(a) cada uno de ellos opera en un espacio de direcciones distinto, mientras que en la figura 2-11(b) los tres comparten el mismo espacio de direcciones.

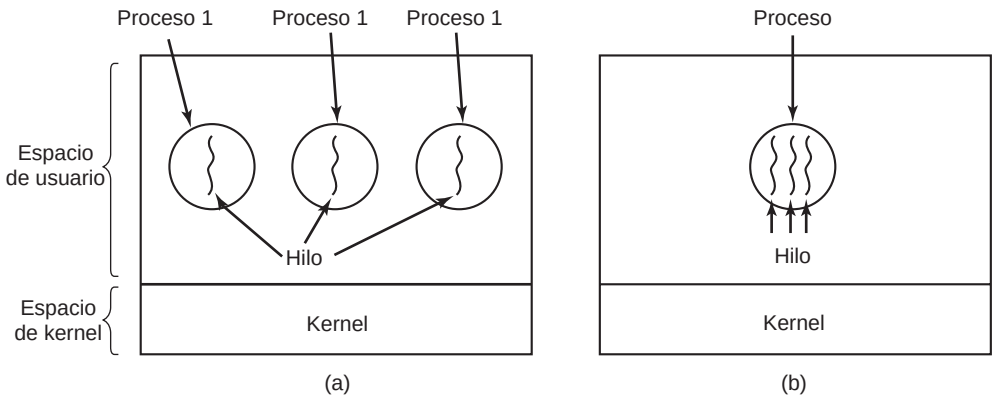


Figura 2-11. (a) Tres procesos, cada uno con un hilo. (b) Un proceso con tres hilos.

Cuando se ejecuta un proceso con **multihilamiento** en un sistema con una CPU, los hilos toman turnos para ejecutarse. En la figura 2-1 vimos cómo funciona la multiprogramación de procesos. Al conmutar de un proceso a otro entre varios procesos, el sistema da la apariencia de que hay

www.FreeLibros.me

procesos secuenciales separados ejecutándose en paralelo. El **multihilamiento** funciona de la misma manera. La CPU conmuta rápidamente entre un hilo y otro, dando la ilusión de que los hilos se ejecutan en paralelo, aunque en una CPU más lenta que la verdadera. Con tres hilos limitados a cálculos en un proceso, los hilos parecerían ejecutarse en paralelo, cada uno en una CPU con un tercio de la velocidad de la CPU verdadera.

Los distintos hilos en un proceso no son tan independientes como los procesos. Todos los hilos tienen el mismo espacio de direcciones, lo cual significa que también comparten las mismas variables globales. Como cada hilo puede acceder a cada dirección de memoria dentro del espacio de direcciones del proceso, un hilo puede leer, escribir o incluso borrar la pila de otro hilo. No hay protección entre los hilos debido a que (1) es imposible y (2) no debe ser necesario. A diferencia de tener procesos diferentes, que pueden ser de distintos usuarios y hostiles entre sí, un proceso siempre es propiedad de un solo usuario, quien se supone que ha creado varios hilos para que puedan cooperar, no pelear. Además de compartir un espacio de direcciones, todos los hilos pueden compartir el mismo conjunto de archivos abiertos, procesos hijos, alarmas y señales, etc., como se muestra en la figura 2-12. Por ende, la organización de la figura 2-11(a) se utilizaría cuando los tres procesos no estén esencialmente relacionados, mientras que la figura 2-11(b) sería apropiada cuando los tres hilos en realidad formen parte del mismo trabajo y estén cooperando en forma activa y estrecha entre sí.

Elementos por proceso	**Elementos por hilo**
Espacio de direcciones	Contador de programa
Variables globales	Registros
Archivos abiertos	Pila
Procesos hijos	Estado
Alarmas pendientes	
Señales y manejadores de señales	
Información contable	

Figura 2-12. La primera columna lista algunos elementos compartidos por todos los hilos en un proceso; la segunda, algunos elementos que son privados para cada hilo.

Los elementos en la primera columna son propiedades de un proceso, no de un hilo. Por ejemplo, si un hilo abre un archivo, ese archivo está visible para los demás hilos en el proceso y pueden leer y escribir datos en él. Esto es lógico, ya que el proceso es la unidad de administración de recursos, no el hilo. Si cada hilo tuviera su propio espacio de direcciones, archivos abiertos, alarmas pendientes, etcétera, sería un proceso separado. Lo que estamos tratando de lograr con el concepto de los hilos es la habilidad de que varios hilos de ejecución compartan un conjunto de recursos, de manera que puedan trabajar en conjunto para realizar cierta tarea.

Al igual que un proceso tradicional (es decir, un proceso con sólo un hilo), un hilo puede estar en uno de varios estados: en ejecución, bloqueado, listo o terminado. Un hilo en ejecución tiene la CPU en un momento dado y está activo. Un hilo bloqueado está esperando a que cierto evento lo

www.FreeLibros.me

desbloquee. Por ejemplo, cuando un hilo realiza una llamada al sistema para leer datos del teclado, se bloquea hasta que se escribe la entrada. Un hilo puede bloquearse en espera de que ocurra algún evento externo o que algún otro hilo lo desbloquee. Un hilo listo se programa para ejecutarse y lo hará tan pronto como sea su turno. Las transiciones entre los estados de los hilos son las mismas que las transiciones entre los estados de los procesos y se ilustran en la figura 2-2.

Es importante tener en cuenta que cada hilo tiene su propia pila, como la figura 2-13 lo ilustra. La pila de cada hilo contiene un conjunto de valores para cada procedimiento llamado, pero del que todavía no se ha regresado. Este conjunto de valores contiene las variables locales del procedimiento y la dirección de retorno que se debe utilizar cuando haya terminado la llamada al procedimiento. Por ejemplo, si el procedimiento *X* llama al procedimiento *Y* y *Y* llama al procedimiento *Z*, entonces mientras se ejecuta *Z*, los conjuntos de valores para *X*, *Y* y *Z* estarán todos en la pila. Por lo general, cada hilo llama a distintos procedimientos y por ende, tiene un historial de ejecución diferente. Esta es la razón por la cual cada hilo necesita su propia pila.

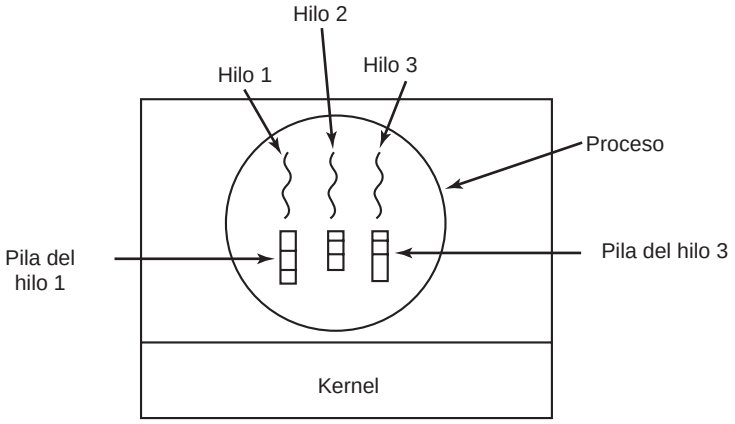


Figura 2-13. Cada hilo tiene su propia pila.

Cuando hay multihilamiento, por lo general los procesos empiezan con un solo hilo presente. Este hilo tiene la habilidad de crear hilos mediante la llamada a un procedimiento de biblioteca, como *thread_create*. Comúnmente, un parámetro para *thread_create* especifica el nombre de un procedimiento para que se ejecute el nuevo hilo. No es necesario (ni posible) especificar nada acerca del espacio de direcciones del nuevo hilo, ya que se ejecuta de manera automática en el espacio de direcciones del hilo creador. Algunas veces los hilos son jerárquicos, con una relación padre-hijo, pero a menudo no existe dicha relación y todos los hilos son iguales. Con o sin una relación jerárquica, el hilo creador generalmente recibe un identificador de hilo que da nombre al nuevo hilo.

Cuando un hilo termina su trabajo, puede salir mediante la llamada a un procedimiento de biblioteca, como *thread_exit*. Después desaparece y ya no puede planificarse para volver a ejecutarse. En algunos sistemas con hilos, un hilo puede esperar a que un hilo (específico) termine mediante

www.FreeLibros.me

la llamada a un procedimiento, por ejemplo *thread_join*. Este procedimiento bloquea al hilo llamador hasta que un hilo (específico) haya terminado. En este aspecto, la creación y terminación de hilos es algo muy parecido a la creación y terminación de procesos, conteniendo casi las mismas opciones.

Otra llamada de hilos común es *thread_yield*, que permite a un hilo entregar voluntariamente la CPU para dejar que otro hilo se ejecute. Dicha llamada es importante, ya que no hay una interrupción de reloj para implementar en realidad la multiprogramación, como en los procesos. Por ende, es importante que los hilos sean amables y entreguen de manera voluntaria la CPU de vez en cuando, para dar a otros hilos la oportunidad de ejecutarse. Otras llamadas permiten a un hilo esperar a que otro termine cierto trabajo, y que otro anuncie que ya terminó cierto trabajo, y así sucesivamente.

Aunque los hilos son útiles a menudo, también introducen cierto número de complicaciones en el modelo de programación. Para empezar, considere los efectos de la llamada al sistema `fork` de UNIX. Si el proceso padre tiene varios hilos, ¿deberá el hijo tenerlos también? Si no es así, el proceso podría no funcionar en forma apropiada, ya que todos ellos podrían ser esenciales.

No obstante, si el proceso hijo obtiene tantos hilos como el padre, ¿qué ocurre si un hilo en el padre se bloqueó en una llamada `read`, por ejemplo, del teclado? ¿Hay ahora dos hilos bloqueados en el teclado, uno en el padre y otro en el hijo? Cuando se escriba una línea, ¿obtendrán ambos hilos una copia de ella? ¿Será sólo para el padre? ¿O sólo para el hijo? El mismo problema existe con las conexiones abiertas de red.

Otra clase de problemas se relaciona con el hecho de que los hilos comparten muchas estructuras de datos. ¿Qué ocurre si un hilo cierra un archivo mientras otro aún está leyendo datos de él? Suponga que un hilo detecta que hay muy poca memoria y empieza a asignar más. A mitad del proceso, ocurre una conmutación de hilos y el nuevo hilo también detecta que hay muy poca memoria y también empieza a asignar más memoria. Es muy probable que se asigne memoria dos veces. Estos problemas se pueden resolver con cierto esfuerzo, pero es necesario pensar y diseñar con cuidado para que los programas con multihilamiento funcionen de la manera correcta.

2.2.3 Hilos en POSIX

Para que sea posible escribir programas con hilos portátiles, el IEEE ha definido un estándar para los hilos conocido como 1003.1c. El paquete de hilos que define se conoce como **Pthreads**. La mayoría de los sistemas UNIX aceptan este paquete. El estándar define más de 60 llamadas a funciones, que son demasiadas como para verlas en este libro. En vez de ello, describiremos sólo algunas de las más importantes, para que el lector tenga una idea de cómo funcionan. Las llamadas que describiremos se listan en la figura 2-14.

Todos los hilos Pthreads tienen ciertas propiedades. Cada uno tiene un identificador, un conjunto de registros (incluyendo el contador de programa) y un conjunto de atributos, que se almacenan en una estructura. Los atributos incluyen el tamaño de la pila, parámetros de planificación y otros elementos necesarios para utilizar el hilo.

www.FreeLibros.me

Llamada de hilo	Descripción
Pthread_create	Crea un nuevo hilo
Pthread_exit	Termina el hilo llamador
Pthread_join	Espera a que un hilo específico termine
Pthread_yield	Libera la CPU para dejar que otro hilo se ejecute
Pthread_attr_init	Crea e inicializa la estructura de atributos de un hilo
Pthread_attr_destroy	Elimina la estructura de atributos de un hilo

Figura 2-14. Algunas de las llamadas a funciones de Pthreads.

Para crear un hilo se utiliza la llamada a *pthread_create*. El identificador de hilo o el hilo recién creado se devuelven como el valor de la función. La llamada es intencionalmente muy parecida a la llamada al sistema `fork`, donde el identificador del hilo juega el papel del PID, en buena medida para identificar a los hilos referenciados en otras llamadas.

Cuando un hilo ha terminado el trabajo que se le asignó, puede terminar llamando a *pthread_exit*. Esta llamada detiene el hilo y libera su pila.

A menudo, un hilo necesita esperar a que otro termine su trabajo y salga para poder continuar. El hilo que está esperando llama a *pthread_join* para esperar a que otro hilo específico termine. El identificador de hilo del hilo al que se va a esperar se proporciona como parámetro.

Algunas veces sucede que un hilo no está lógicamente bloqueado, pero siente que se ha ejecutado el tiempo suficiente y desea dar a otro hilo la oportunidad de ejecutarse. Para lograr este objetivo hace una llamada a *pthread_yield*. No hay una llamada así para los procesos, debido a que se supone que los procesos son en extremo competitivos y cada uno desea tener a la CPU todo el tiempo que pueda. Sin embargo, como los hilos de un proceso están trabajando en conjunto y su código fue escrito indudablemente por el mismo programador, algunas veces éste desea darles a otros hilos una oportunidad.

Las siguientes dos llamadas a hilos tratan con los atributos. *Pthread_attr_init* crea la estructura de atributos asociada con un hilo y la inicializa con los valores predeterminados. Estos valores (como la prioridad) se pueden modificar mediante la manipulación de campos en la estructura de atributos.

Por último, *pthread_attr_destroy* elimina la estructura de atributos de un hilo, liberando su memoria. No afecta a los hilos que la utilizan; éstos siguen existiendo.

Para tener una mejor idea de la forma en que funciona Pthreads, considere el ejemplo simple de la figura 2-15. Aquí el programa principal itera *NUMERO_DE_HILOS* veces, creando un nuevo hilo en cada iteración, después de anunciar su intención. Si falla la creación del hilo, imprime un mensaje de error y después termina. Después de crear todos los hilos, el programa principal termina.

Cuando se crea un hilo, éste imprime un mensaje de una línea anunciándose a sí mismo y después termina. El orden en el que se intercalan los diversos mensajes no es determinado, y puede variar entre una ejecución del programa y otra.

www.FreeLibros.me

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMERO_DE_HILOS             10

void *imprimir_hola_mundo(void *tid)
{
    /* Esta funcion imprime el identificador del hilo y después termina. */
    printf("Hola mundo. Saludos del hilo %d0, tid);
    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    /* El programa principal crea 10 hilos y después termina. */
    pthread_t hilos[NUMERO_DE_HILOS];
    int estado, i;

    for(i=0; i < NUMERO_DE_HILOS; i++) {
        printf("Aqui main. Creando hilo %d0, i);
        estado = pthread_create(%hilos[i], NULL, imprimir_hola_mundo, (void *)i);

        if (estado != 0) {
            printf("Ups. pthread_create devolvió el codigo de error %d0, estado);
            exit(-1);
        }
    }
    exit(NULL);
}
```

Figura 2-15. Un programa de ejemplo que utiliza hilos.

Las llamadas de Pthreads antes descritas no son en definitiva las únicas; hay muchas más. Más adelante analizaremos algunas de las otras, después de hablar sobre los procesos y la sincronización de hilos.

2.2.4 Implementación de hilos en el espacio de usuario

Hay dos formas principales de implementar un paquete de hilos: en espacio de usuario y en el kernel. La elección es un poco controversial y también es posible una implementación híbrida. Ahora describiremos estos métodos, junto con sus ventajas y desventajas.

El primer método es colocar el paquete de hilos completamente en espacio de usuario. El kernel no sabe nada acerca de ellos. En lo que al kernel concierne, está administrando procesos ordinarios con un solo hilo. La primera ventaja, la más obvia, es que un paquete de hilos de nivel usuario puede implementarse en un sistema operativo que no acepte hilos. Todos los sistemas operativos

www.FreeLibros.me

solían entrar en esta categoría e incluso hoy en día algunos todavía lo están. Con este método, los hilos se implementan mediante una biblioteca.

Todas estas implementaciones tienen la misma estructura general, que se ilustra en la figura 2-16(a). Los hilos se ejecutan encima de un sistema en tiempo de ejecución, el cual es una colección de procedimientos que administran hilos. Ya hemos visto cuatro de éstos: *pthread_create*, *pthread_exit*, *pthread_join* y *pthread_yield*, pero por lo general hay más.

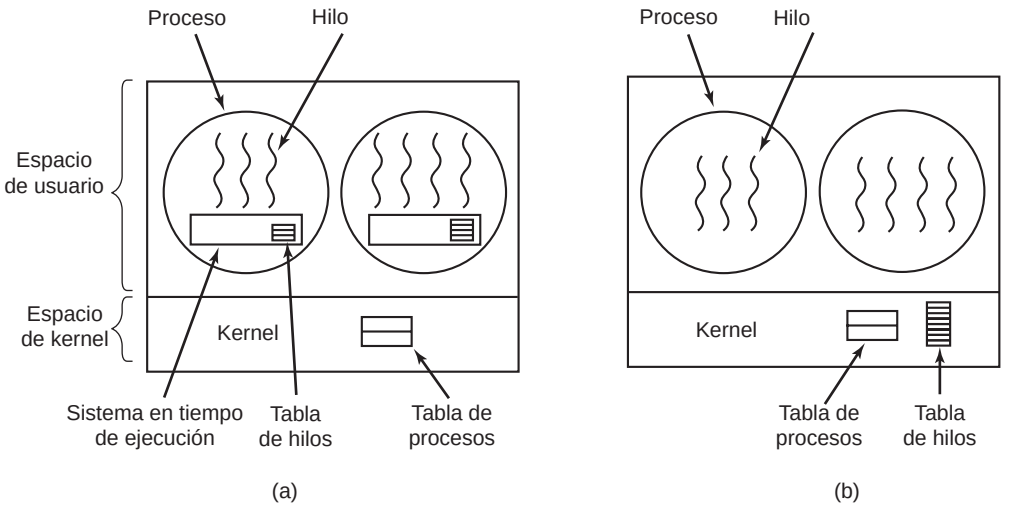


Figura 2-16. (a) Un paquete de hilos de nivel usuario. (b) Un paquete de hilos administrado por el kernel.

Cuando los hilos se administran en espacio de usuario, cada proceso necesita su propia **tabla de hilos** privada para llevar la cuenta de los hilos en ese proceso. Esta tabla es similar a la tabla de procesos del kernel, excepto porque sólo lleva la cuenta de las propiedades por cada hilo, como el contador de programa, apuntador de pila, registros, estado, etc. La tabla de hilos es administrada por el sistema en tiempo de ejecución. Cuando un hilo pasa al estado listo o bloqueado, la información necesaria para reiniciarlo se almacena en la tabla de hilos, en la misma forma exacta que el kernel almacena la información acerca de los procesos en la tabla de procesos.

Cuando un hilo hace algo que puede ponerlo en estado bloqueado en forma local —por ejemplo, esperar a que otro hilo dentro de su proceso complete cierto trabajo— llama a un procedimiento del sistema en tiempo de ejecución. Este procedimiento comprueba si el hilo debe ponerse en estado bloqueado. De ser así, almacena los registros del hilo (es decir, sus propios registros) en la tabla de hilos, busca en la tabla un hilo listo para ejecutarse y vuelve a cargar los registros de la máquina con los valores guardados del nuevo hilo. Tan pronto como se conmutan el apuntador de pila y el contador de programa, el nuevo hilo vuelve otra vez a la vida de manera automática. Si la máquina tiene una instrucción para guardar todos los registros y otra para cargarlos de una sola vez, toda la conmutación de hilos se puede realizar con sólo unas cuantas instrucciones. Realizar una conmutación de hilos como éste es por lo menos una orden de magnitud (o tal vez más) más

www.FreeLibros.me

veloz que hacer el trap al kernel y es un sólido argumento a favor de los paquetes de hilos de nivel usuario.

Sin embargo, hay una diferencia clave con los procesos. Cuando un hilo termina de ejecutarse por el momento, por ejemplo, cuando llama a *thread_yield*, el código de *thread_yield* puede guardar la información del hilo en la tabla de hilos. Lo que es más, así puede entonces llamar al planificador de hilos para elegir otro hilo y ejecutarlo. El procedimiento que guarda el estado del hilo y el planificador son sólo procedimientos locales, por lo que es mucho más eficiente invocarlos que realizar una llamada al kernel. Entre otras cuestiones, no se necesita un trap ni una conmutación de contexto, la memoria caché no necesita vaciarse, etc. Esto hace que la planificación de hilos sea muy rápida.

Los hilos de nivel usuario también tienen otras ventajas. Permiten que cada proceso tenga su propio algoritmo de planificación personalizado. Por ejemplo, para algunas aplicaciones, las que tienen un hilo recolector de basura, es una ventaja no tener que preocuparse porque un hilo se detenga en un momento inconveniente. También se escalan mejor, ya que los hilos del kernel requieren sin duda algo de espacio en la tabla y en la pila del kernel, lo cual puede ser un problema si hay una gran cantidad de hilos.

A pesar de su mejor rendimiento, los paquetes de hilos de nivel usuario tienen algunos problemas importantes. El primero de todos es la manera en que se implementan las llamadas al sistema de bloqueo. Suponga que un hilo lee del teclado antes de que se haya oprimido una sola tecla. Es inaceptable permitir que el hilo realice la llamada al sistema, ya que esto detendrá a todos los hilos. Uno de los principales objetivos de tener hilos en primer lugar era permitir que cada uno utilizara llamadas de bloqueo, pero para evitar que un hilo bloqueado afectara a los demás. Con las llamadas al sistema de bloqueo, es difícil ver cómo se puede lograr este objetivo sin problemas.

Todas las llamadas al sistema se podrían cambiar para que quedaran sin bloqueo (por ejemplo, un `read` en el teclado sólo devolvería 0 bytes si no hubiera caracteres en el búfer), pero es inconveniente requerir cambios en el sistema operativo. Además, uno de los argumentos para los hilos de nivel usuario era precisamente que se podían ejecutar con los sistemas operativos *existentes*. Además, si se cambia la semántica de `read` se requerirán cambios en muchos programas de usuario.

Es posible otra alternativa si se puede saber de antemano si una llamada va a bloquear. En algunas versiones de UNIX existe una llamada al sistema (`select`), la cual permite al procedimiento que hace la llamada saber si una posible llamada a `read` realizará un bloqueo. Cuando esta llamada está presente, el procedimiento de biblioteca *read* se puede reemplazar con uno nuevo que primero realice una llamada a `select` y después sólo realice la llamada a `read` si es seguro (es decir, si no va a realizar un bloqueo). Si la llamada a `read` va a bloquear, no se hace; en vez de ello, se ejecuta otro hilo. La próxima vez que el sistema en tiempo de ejecución obtenga el control, puede comprobar de nuevo para ver si la llamada a `read` es ahora segura. Este método requiere que se vuelvan a escribir partes de la biblioteca de llamadas al sistema, es ineficiente y nada elegante, pero hay muy poca opción. El código colocado alrededor de la llamada al sistema que se encarga de la comprobación se conoce como **envoltura**.

Algo similar al problema de las llamadas al sistema de bloqueo es el problema de los fallos de página. En el capítulo 3 estudiaremos este tema. Por ahora, basta con decir que las computadoras

www.FreeLibros.me

se pueden configurar de forma que no todo el programa se encuentre en memoria a la vez. Si el programa llama o salta a una instrucción que no esté en memoria, ocurre un fallo de página y el sistema operativo obtiene la instrucción faltante (y las instrucciones aledañas) del disco. A esto se le conoce como fallo de página. El proceso se bloquea mientras la instrucción necesaria se localiza y se lee. Si un hilo produce un fallo de página, el kernel (que ni siquiera sabe de la existencia de los hilos) bloquea naturalmente todo el proceso hasta que se complete la operación de E/S, incluso si otros hilos pudieran ser ejecutados.

Otro problema con los paquetes de hilos de nivel usuario es que, si un hilo empieza a ejecutarse, ningún otro hilo en ese proceso se ejecutará a menos que el primero renuncie de manera voluntaria a la CPU. Dentro de un solo proceso no hay interrupciones de reloj, lo cual hace que sea imposible planificar procesos en el formato *round robin* (tomando turnos). A menos que un hilo entre al sistema en tiempo de ejecución por su propia voluntad, el planificador nunca tendrá una oportunidad.

Una posible solución al problema de los hilos que se ejecutan en forma indefinida es hacer que el sistema en tiempo de ejecución solicite una señal de reloj (interrupción) una vez por segundo para dar el control, pero esto también es crudo y complicado para un programa. No siempre son posibles las interrupciones periódicas de reloj a una frecuencia más alta e incluso si lo son, la sobrecarga total podría ser considerable. Lo que es peor: un hilo podría requerir también una interrupción de reloj, interfiriendo con el uso que el sistema en tiempo de ejecución da al reloj.

Otro argumento (que en realidad es el más devastador) contra los hilos de nivel usuario es que, por lo general, los programadores desean hilos precisamente en aplicaciones donde éstos se bloquean con frecuencia, como, por ejemplo, un servidor Web con multihilado. Estos hilos están realizando llamadas al sistema en forma constante. Una vez que ocurre un trap al kernel, de manera que lleve a cabo la llamada al sistema, no le cuesta mucho al kernel conmutar hilos si el anterior está bloqueado y, al hacer esto el kernel, se elimina la necesidad de realizar llamadas al sistema `select` en forma constante que comprueben si las llamadas al sistema `read` son seguras. Para las aplicaciones que en esencia están completamente limitadas a la CPU y raras veces se bloquean, ¿cuál es el objetivo de tener hilos? Nadie propondría con seriedad calcular los primeros *n* números primos o jugar ajedrez utilizando hilos, debido a que no se obtiene ninguna ventaja al hacerlo de esta forma.

2.2.5 Implementación de hilos en el kernel

Ahora vamos a considerar el caso en que el kernel sabe acerca de los hilos y los administra. No se necesita un sistema en tiempo de ejecución para ninguna de las dos acciones, como se muestra en la figura 2-16(b). Además, no hay tabla de hilos en cada proceso. En vez de ello, el kernel tiene una tabla de hilos que lleva la cuenta de todos los hilos en el sistema. Cuando un hilo desea crear un nuevo hilo o destruir uno existente, realiza una llamada al kernel, la cual se encarga de la creación o destrucción mediante una actualización en la tabla de hilos del kernel.

La tabla de hilos del kernel contiene los registros, el estado y demás información de cada hilo. Esta información es la misma que con los hilos de nivel usuario, pero ahora se mantiene en el kernel, en vez de hacerlo en espacio de usuario (dentro del sistema en tiempo de ejecución). Esta información es un subconjunto de la información que mantienen tradicionalmente los kernels acerca de

www.FreeLibros.me

sus procesos con un solo hilo; es decir, el estado del proceso. Además, el kernel también mantiene la tabla de procesos tradicional para llevar la cuenta de los procesos.

Todas las llamadas que podrían bloquear un hilo se implementan como llamadas al sistema, a un costo considerablemente mayor que una llamada a un procedimiento del sistema en tiempo de ejecución. Cuando un hilo se bloquea, el kernel, según lo que decida, puede ejecutar otro hilo del mismo proceso (si hay uno listo) o un hilo de un proceso distinto. Con los hilos de nivel usuario, el sistema en tiempo de ejecución ejecuta hilos de su propio proceso hasta que el kernel le quita la CPU (o cuando ya no hay hilos para ejecutar).

Debido al costo considerablemente mayor de crear y destruir hilos en el kernel, algunos sistemas optan por un método ambientalmente correcto, reciclando sus hilos. Cuando se destruye un hilo, se marca como no ejecutable pero las estructuras de datos de su kernel no se ven afectadas de ninguna otra forma. Más adelante, cuando debe crearse un hilo, se reactiva uno anterior, lo que ahorra cierta sobrecarga. El reciclaje de hilos también es posible para los hilos de nivel usuario, pero como la sobrecarga de la administración de los hilos es mucho menor, hay menos incentivos para hacer esto.

Los hilos de kernel no requieren de nuevas llamadas al sistema sin bloqueo. Además, si un hilo en un proceso produce un fallo de página, el kernel puede comprobar con facilidad si el proceso tiene otros hilos que puedan ejecutarse y de ser así, ejecuta uno de ellos mientras espera a que se traiga la página requerida desde el disco. Su principal desventaja es que el costo de una llamada al sistema es considerable, por lo que si las operaciones de hilos (de creación o terminación, por ejemplo) son comunes, se incurrirá en una mayor sobrecarga.

Los hilos de kernel resuelven sólo algunos problemas, no todos. Por ejemplo, ¿qué ocurre cuando un proceso con multihilamiento utiliza la llamada a `fork` para crear otro proceso? ¿El nuevo proceso tiene los mismos hilos que el anterior o tiene sólo uno? En muchos casos, la mejor elección depende de lo que el proceso planee realizar a continuación. Si va a llamar a `exec` para iniciar un nuevo programa, probablemente la elección correcta sea un hilo, pero si continúa en ejecución, tal vez lo mejor sea reproducir todos.

Otra cuestión es la relacionada con las señales. Recuerde que las señales se envían a los procesos, no a los hilos, por lo menos en el modelo clásico. Cuando entra una señal, ¿qué hilo debe hacerse cargo de ella? Es posible que los hilos puedan registrar su interés en ciertas señales, de manera que cuando llegue una señal se envíe al hilo que la está esperando. Pero ¿que ocurre si dos o más hilos se registran para la misma señal? Éstos son sólo dos de los problemas que introducen los hilos, pero hay más.

2.2.6 Implementaciones híbridas

Se han investigado varias formas de tratar de combinar las ventajas de los hilos de nivel usuario con los hilos de nivel kernel. Una de esas formas es utilizar hilos de nivel kernel y después multiplexar los hilos de nivel usuario con alguno o con todos los hilos de nivel kernel, como se muestra en la figura 2-17. Cuando se utiliza este método, el programador puede determinar cuántos hilos de kernel va a utilizar y cuántos hilos de nivel usuario va a multiplexar en cada uno. Este modelo proporciona lo último en flexibilidad.

www.FreeLibros.me

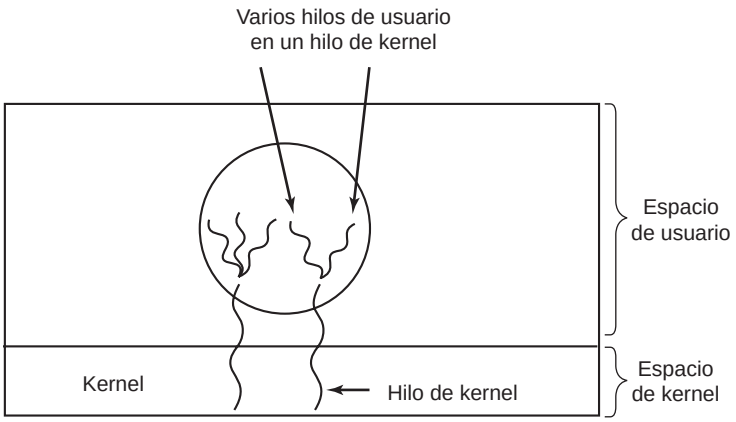


Figura 2-17. Multiplexaje de hilos del nivel usuario sobre hilos del nivel kernel.

Con este método, el kernel está consciente *sólo* de los hilos de nivel kernel y los planifica. Algunos de esos hilos pueden tener varios hilos de nivel usuario multiplexados encima de ellos; los hilos de nivel de usuario se crean, destruyen y planifican de igual forma que los hilos de nivel usuario en un proceso que se ejecuta en un sistema operativo sin capacidad de multihilamiento. En este modelo, cada hilo de nivel kernel tiene algún conjunto de hilos de nivel usuario que toman turnos para utilizarlo.

2.2.7 Activaciones del planificador

Aunque los hilos de kernel son mejores que los hilos de nivel usuario en ciertas formas clave, también son sin duda más lentos. Como consecuencia, los investigadores han buscado formas de mejorar la situación sin renunciar a sus propiedades positivas. A continuación describimos uno de esos métodos ideado por Anderson y colaboradores (1992), conocido como **activaciones del planificador**. El trabajo relacionado se describe en las obras de Edler y colaboradores (1988), y de Scott y colaboradores (1990).

Los objetivos del trabajo de una activación del planificador son imitar la funcionalidad de los hilos de kernel, pero con el mejor rendimiento y la mayor flexibilidad que por lo general se asocian con los paquetes de hilos implementados en espacio de usuario. En especial, los hilos de usuario no deben tener que realizar llamadas especiales al sistema sin bloqueo, ni comprobar de antemano que sea seguro realizar ciertas llamadas al sistema. Sin embargo, cuando un hilo se bloquea en una llamada al sistema o un fallo de página, debe ser posible ejecutar otros hilos dentro del mismo proceso, si hay alguno listo.

La eficiencia se obtiene evitando transiciones innecesarias entre los espacios de usuario y de kernel. Por ejemplo, si un hilo se bloquea en espera de que otro hilo realice alguna acción, no hay razón para involucrar al kernel, con lo cual se ahorra la sobrecarga de la transición de kernel a usuario. El sistema en tiempo de ejecución en espacio de usuario puede bloquear el hilo sincronizador y programar uno nuevo por sí solo.

www.FreeLibros.me

Cuando se utilizan las activaciones del planificador, el kernel asigna cierto número de procesadores virtuales a cada proceso y deja que el sistema en tiempo de ejecución (en espacio de usuario) asigne hilos a los procesadores. Este mecanismo también se puede utilizar en un multiprocesador, donde los procesadores virtuales podrían ser CPUs reales. Al principio, el número de procesadores virtuales que se asigna a un proceso es uno, pero el proceso puede pedir más y también devolver procesadores si ya no los necesita. El kernel también puede obtener de vuelta los procesadores virtuales que ya estén asignados, para poder asignarlos a procesos que tengan más necesidad.

La idea básica que hace que este esquema funcione es que, cuando el kernel sabe que un hilo se ha bloqueado (por ejemplo, al ejecutar una llamada al sistema de bloqueo o al ocasionar un fallo de página), se lo notifica al sistema en tiempo de ejecución del proceso, pasándole como parámetros a la pila el número del hilo en cuestión y una descripción del evento que ocurrió. Para realizar la notificación, el kernel activa el sistema en tiempo de ejecución en una dirección inicial conocida, no muy similar a una señal en UNIX. A este mecanismo se le conoce como **llamada ascendente** (*upcall*).

Una vez que se activa de esta forma, el sistema en tiempo de ejecución puede replanificar sus hilos, para lo cual comúnmente marca al hilo actual como bloqueado, tomando otro hilo de la lista de hilos listos, establece sus registros y lo reinicia. Más adelante, cuando el kernel detecta que el hilo original se puede ejecutar de nuevo (por ejemplo, la canal del que trataba de leer ya contiene datos, o la página que falló se trajo ya del disco), realiza otra llamada ascendente al sistema en tiempo de ejecución para informarle sobre este evento. El sistema en tiempo de ejecución, a su propia discreción, puede reiniciar el hilo bloqueado de inmediato o colocarlo en la lista de hilos listos para ejecutarlo más adelante.

Cuando ocurre una interrupción de hardware mientras un hilo de usuario se ejecuta, la CPU que se interrumpió conmuta al modo kernel. Si, cuando el manejador de interrupciones termina, la interrupción es producida por un evento que no sea de interés para el proceso interrumpido (como la terminación de una operación de E/S de otro proceso) coloca el hilo interrumpido de vuelta en el estado en el que estaba antes de la interrupción. No obstante, si el proceso está interesado en la interrupción, como la llegada de una página que requiere uno de los hilos del procesador, el hilo interrumpido no se reinicia. En vez de ello, el hilo interrumpido se suspende y el sistema en tiempo de ejecución se inicia en esa CPU virtual, con el estado del hilo interrumpido en la pila. Después, es responsabilidad del sistema en tiempo de ejecución decidir cuál hilo debe planificar en esa CPU: el que se interrumpió, el que acaba de pasar al estado listo o alguna otra tercera opción.

Una objeción a las activaciones del planificador es la dependencia fundamental en las llamadas ascendentes, un concepto que viola la estructura inherente en cualquier sistema por capas. Por lo general, la capa n ofrece ciertos servicios que la capa $n + 1$ puede llamar, pero la capa n no puede llamar a los procedimientos en la capa $n + 1$. Las llamadas ascendentes no siguen este principio fundamental.

2.2.8 Hilos emergentes

Los hilos se utilizan con frecuencia en los sistemas distribuidos. Un importante ejemplo es la forma en que se manejan los mensajes entrantes (por ejemplo, las peticiones de servicio). El método tradicional es hacer que un proceso o hilo, que está bloqueado en una llamada al sistema `receive`,

www.FreeLibros.me

espere un mensaje entrante. Cuando llega un mensaje, lo acepta, lo desempaqueta, examina su contenido y lo procesa.

Sin embargo, también es posible utilizar un método completamente distinto, en el cual la llegada de un mensaje hace que el sistema cree un nuevo hilo para manejar el mensaje. A dicho hilo se le conoce como **hilo emergente** (*pop-up thread*) y se ilustra en la figura 2-18. Una ventaja clave de los hilos emergentes es que, como son nuevos, no tienen historial (registros, pila, etcétera) que sea necesario restaurar. Cada uno empieza desde cero y es idéntico a los demás. Esto hace que sea posible crear dicho hilo con rapidez. El nuevo hilo recibe el mensaje entrante que va a procesar. El resultado de utilizar hilos emergentes es que la latencia entre la llegada del mensaje y el inicio del procesamiento puede ser muy baja.

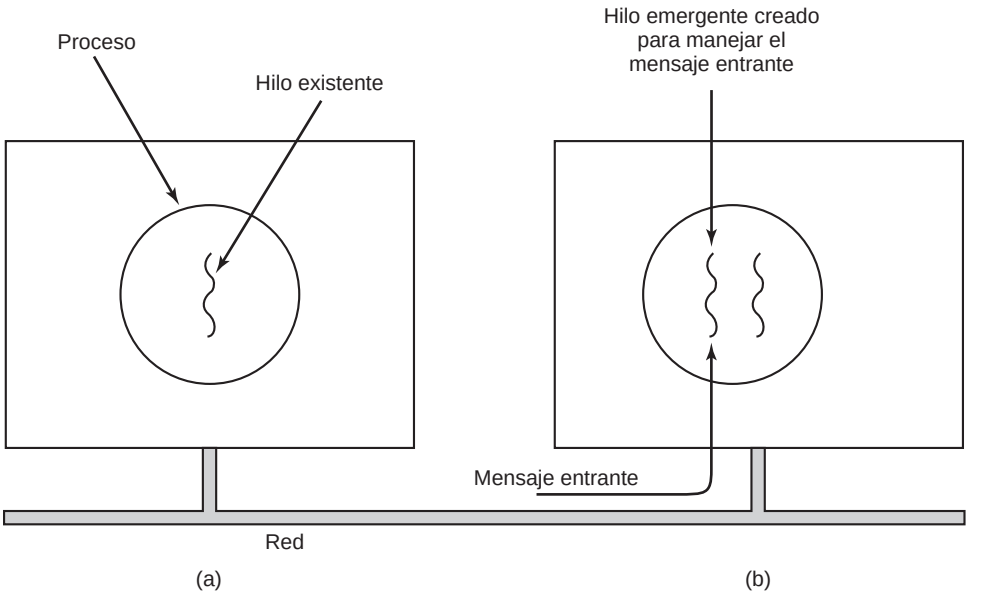


Figura 2.18. Creación de un nuevo hilo cuando llega un mensaje. (a) Antes de que llegue el mensaje. (b) Después de que llega el mensaje.

Es necesaria cierta planeación anticipada cuando se utilizan hilos emergentes. Por ejemplo, ¿en qué proceso se ejecuta el hilo? Si el sistema acepta hilos que se ejecutan en el contexto del kernel, el hilo se puede ejecutar ahí (lo que explica por qué no hemos mostrado el kernel en la figura 2-18). Hacer que el hilo emergente se ejecute en espacio de kernel es por lo general más rápido y sencillo que colocarlo en espacio de usuario. Además, un hilo emergente en espacio de kernel puede acceder con facilidad a todas las tablas del kernel y a los dispositivos de E/S, que pueden ser necesarios para el procesamiento de interrupciones. Por otro lado, un hilo de kernel con errores puede hacer más daño que un hilo de usuario con errores. Por ejemplo, si se ejecuta durante demasiado tiempo y no hay manera de quitarlo, los datos entrantes se pueden perder.

www.FreeLibros.me

2.2.9 Conversión de código de hilado simple a multihilado

Muchos programas existentes se escribieron para procesos con un solo hilo. Es mucho más difícil convertir estos programas para que utilicen multihilamiento de lo que podría parecer en un principio. A continuación analizaremos unos cuantos de los obstáculos.

Para empezar, el código de un hilo normalmente consiste de varios procedimientos, al igual que un proceso. Éstos pueden tener variables locales, variables globales y parámetros. Las variables y parámetros locales no ocasionan problemas, pero las variables que son globales a un hilo, pero no globales para todo el programa, son un problema. Hay variables que son globales en el sentido en el que muchos procedimientos dentro del hilo los utilizan (como podrían utilizar cualquier variable global), pero otros hilos deberían, por lógica, dejarlas en paz.

Como ejemplo, considere la variable *errno* que mantiene UNIX. Cuando un proceso (o hilo) realiza una llamada al sistema que falla, el código de error se coloca en *errno*. En la figura 2-19, el hilo 1 ejecuta la llamada al sistema access para averiguar si tiene permiso para acceder a cierto archivo. El sistema operativo devuelve la respuesta en la variable global *errno*. Una vez que el control regresa al hilo 1, pero antes de que tenga la oportunidad de leer *errno*, el planificador decide que el hilo 1 ha tenido suficiente tiempo de la CPU por el momento y decide conmutar al hilo 2. El hilo 2 ejecuta una llamada a open que falla, lo cual hace que se sobrescriba el valor de *errno* y el código de acceso del hilo 1 se pierde para siempre. Cuando el hilo 1 se continúe más adelante, leerá el valor incorrecto y se comportará de manera incorrecta.

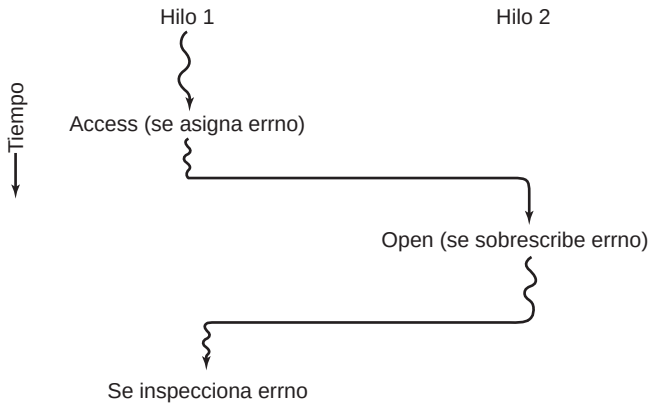


Figura 2-19. Conflictos entre los hilos por el uso de una variable global.

Hay varias soluciones posibles para este problema. Una es prohibir las variables globales por completo. Aunque este ideal parezca conveniente, entra en conflicto con una gran parte del software existente. Otra solución es asignar a cada hilo sus propias variables globales privadas, como se muestra en la figura 2-20. De esta forma, cada hilo tiene su propia copia privada de *errno* y de otras variables globales, por lo que se evitan los conflictos. En efecto, esta decisión crea un nivel de alcance en el que las variables están visibles para todos los procedimientos de un hilo, además de los

www.FreeLibros.me

niveles de alcance existentes en los que las variables están visibles sólo para un procedimiento y en los que las variables están visibles en todas partes del programa.

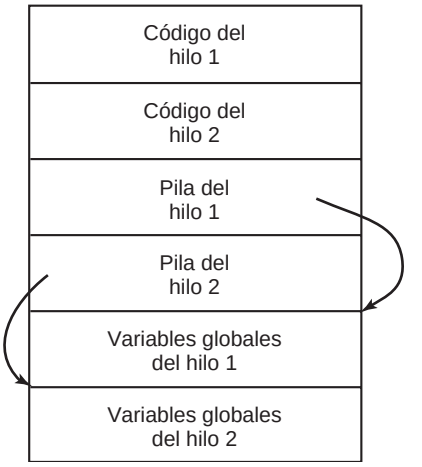


Figura 2-20. Los hilos pueden tener variables globales privadas.

Sin embargo, acceder a las variables globales privadas es algo problemático, ya que la mayoría de los lenguajes de programación tienen una manera de expresar las variables locales y las variables globales, pero no formas intermedias. Es posible asignar un trozo de memoria para las variables globales y pasarlo a cada procedimiento en el hilo como un parámetro adicional. Aunque no es una solución elegante, funciona.

De manera alternativa, pueden introducirse nuevos procedimientos de biblioteca para crear, establecer y leer estas variables globales a nivel de hilo. La primera llamada podría ser como se muestra a continuación:

```
create_global("bufptr");
```

Esta llamada asigna espacio de almacenamiento para un apuntador llamado *bufptr* en el heap (pila para memoria dinámica) o en un área especial de almacenamiento reservada para el hilo invocador. Sin importar dónde se asigne el espacio de almacenamiento, sólo el hilo invocador tiene acceso a la variable global; si otro hilo crea una variable global con el mismo nombre, obtiene una ubicación distinta de almacenamiento que no entre en conflicto con la existente.

Se necesitan dos llamadas para acceder a las variables globales: una para escribirlas y la otra para leerlas. Para escribir, algo como:

```
set_global("bufptr", &buf);
```

es suficiente. Esta llamada almacena el valor de un apuntador en la ubicación de almacenamiento que se creó previamente mediante la llamada a *create_global*. Para leer una variable global, la llamada podría ser algo como lo siguiente:

```
bufptr = read_global("bufptr");
```

Esta llamada devuelve la dirección almacenada en la variable global, de manera que se pueda acceder a sus datos.

www.FreeLibros.me

El siguiente problema al convertir un programa con un solo hilo en un programa con múltiples hilos es que muchos procedimientos de biblioteca no son re-entrantes; es decir, no se diseñaron para hacer una segunda llamada a cualquier procedimiento dado mientras que una llamada anterior no haya terminado. Por ejemplo, podemos programar el envío de un mensaje a través de la red ensamblando el mensaje en un búfer fijo dentro de la biblioteca, para después hacer un trap al kernel para enviarlo. ¿Qué ocurre si un hilo ha ensamblado su mensaje en el búfer y después una interrupción de reloj obliga a que se haga la conmutación a un segundo hilo que de inmediato sobrescribe el búfer con su propio mensaje?

De manera similar, los procedimientos de asignación de memoria (como *malloc* en UNIX) mantienen tablas cruciales acerca del uso de memoria; por ejemplo, una lista ligada de trozos disponibles de memoria. Mientras el procedimiento *malloc* está ocupado actualizando estas listas, pueden estar temporalmente en un estado inconsistente, con apuntadores que apuntan a ningún lado. Si ocurre una conmutación de hilos mientras las tablas son inconsistentes y llega una nueva llamada de un hilo distinto, tal vez se utilice un apuntador inválido y se produzca un fallo en el programa. Para corregir estos problemas de manera efectiva, tal vez sea necesario reescribir la biblioteca completa, lo que no es insignificante.

Una solución distinta es proporcionar a cada procedimiento una envoltura que fije un bit para marcar la librería como si estuviera en uso. Si otro hilo intenta usar un procedimiento de biblioteca mientras no se haya completado una llamada anterior, se bloquea. Aunque se puede hacer que este método funcione, elimina en gran parte el paralelismo potencial.

Ahora, considere las señales. Algunas señales son lógicamente específicas para cada hilo, mientras que otras no. Por ejemplo, si un hilo llama a alarm, tiene sentido que la señal resultante vaya al hilo que hizo la llamada. Sin embargo, cuando los hilos se implementan totalmente en espacio de usuario, el kernel no sabe siquiera acerca de los hilos, por lo que apenas si puede dirigir la señal al hilo correcto. Hay una complicación adicional si un proceso puede tener sólo una alarma pendiente a la vez y varios hilos pueden llamar a alarm de manera independiente.

Otras señales, como la interrupción del teclado, no son específicas para cada hilo. ¿Quién debe atraparlas? ¿Un hilo designado? ¿Todos los hilos? ¿Un hilo emergente recién creado? Lo que es más, ¿qué ocurre si un hilo cambia los manejadores de señales sin indicar a los demás hilos? ¿Y qué ocurre si un hilo desea atrapar una señal específica (por ejemplo, cuando el usuario oprime CTRL-C) y otro hilo desea esta señal para terminar el proceso? Esta situación puede surgir si uno o más hilos ejecutan procedimientos de biblioteca estándar y otros son escritos por el usuario. Es evidente que estos deseos son incompatibles. En general, las señales son bastante difíciles de manejar en un entorno con un solo hilo. Cambiar a un entorno con multihilamiento no facilita su manejo.

Un último problema que introducen los hilos es la administración de la pila. En muchos sistemas, cuando la pila de un proceso se desborda, el kernel sólo proporciona más pila a ese proceso de manera automática. Cuando un proceso tiene múltiples hilos, también debe tener varias pilas. Si el kernel no está al tanto de todas ellas, no puede hacer que su tamaño aumente de manera automática cuando ocurra un fallo de la pila. De hecho, ni siquiera puede detectar que un fallo de memoria está relacionado con el aumento de tamaño de la pila de algún otro hilo.

Sin duda estos problemas no son imposibles de resolver, pero muestran que no se resolverán con sólo introducir hilos en un sistema existente sin un rediseño considerable del mismo sistema.

www.FreeLibros.me

Tal vez haya que redefinir la semántica de las llamadas al sistema y reescribir las bibliotecas, cuando menos. Y todo esto se debe realizar de tal forma que se mantenga una compatibilidad hacia atrás con los programas existentes para el caso limitante de un proceso con sólo un hilo. Para obtener información adicional acerca de los hilos, consulte (Hauser y colaboradores, 1993; y Marsh y colaboradores, 1991).

2.3 COMUNICACIÓN ENTRE PROCESOS

Con frecuencia, los procesos necesitan comunicarse con otros procesos. Por ejemplo, en una canalización del shell, la salida del primer proceso se debe pasar al segundo proceso y así sucesivamente. Por ende, existe una necesidad de comunicación entre procesos, de preferencia en una forma bien estructurada sin utilizar interrupciones. En las siguientes secciones analizaremos algunas de las cuestiones relacionadas con esta **comunicación entre procesos** o **IPC**.

En resumen, hay tres cuestiones aquí. La primera se alude a lo anterior: cómo un proceso puede pasar información a otro. La segunda está relacionada con hacer que dos o más procesos no se interpongan entre sí; por ejemplo, dos procesos en un sistema de reservaciones de una aerolínea, cada uno de los cuales trata de obtener el último asiento en un avión para un cliente distinto. La tercera trata acerca de obtener la secuencia apropiada cuando hay dependencias presentes: si el proceso *A* produce datos y el proceso *B* los imprime, *B* tiene que esperar hasta que *A* haya producido algunos datos antes de empezar a imprimir. En la siguiente sección analizaremos las tres cuestiones.

También es importante mencionar que dos de estas cuestiones se aplican de igual forma a los hilos. La primera (el paso de información) es fácil para los hilos, ya que comparten un espacio de direcciones común (los hilos en distintos espacios de direcciones que necesitan comunicarse entran en la categoría de los procesos en comunicación). Sin embargo, las otras dos (evitar que un hilo entre en conflicto con los demás y obtener la secuencia apropiada) se aplican de igual forma a los hilos. Existen los mismos problemas y se aplican las mismas soluciones. A continuación veremos el problema en el contexto de los procesos, pero tenga en cuenta que también se aplican los mismos problemas y soluciones a los hilos.

2.3.1 Condiciones de carrera

En algunos sistemas operativos, los procesos que trabajan en conjunto pueden compartir cierto espacio de almacenamiento en el que pueden leer y escribir datos. El almacenamiento compartido puede estar en la memoria principal (posiblemente en una estructura de datos del kernel) o puede ser un archivo compartido; la ubicación de la memoria compartida no cambia la naturaleza de la comunicación o los problemas que surgen. Para ver cómo funciona la comunicación entre procesos en la práctica, consideremos un ejemplo simple pero común: un spooler de impresión. Cuando un proceso desea imprimir un archivo, introduce el nombre del archivo en un **directorio de spooler** especial. Otro proceso, el **demonio de impresión**, comprueba en forma periódica si hay archivos que deban imprimirse y si los hay, los imprime y luego elimina sus nombres del directorio.

www.FreeLibros.me

Imagine que nuestro directorio de spooler tiene una cantidad muy grande de ranuras, numeradas como 0, 1, 2, …, cada una de ellas capaz de contener el nombre de un archivo. Imagine también que hay dos variables compartidas: *sal*, que apunta al siguiente archivo a imprimir, y *ent*, que apunta a la siguiente ranura libre en el directorio. Estas dos variables podrían mantenerse muy bien en un archivo de dos palabras disponible para todos los procesos. En cierto momento, las ranuras de la 0 a la 3 están vacías (ya se han impreso los archivos) y las ranuras de la 4 a la 6 están llenas (con los nombres de los archivos en la cola de impresión). De manera más o menos simultánea, los procesos *A* y *B* deciden que desean poner en cola un archivo para imprimirlo. Esta situación se muestra en la figura 2-21.

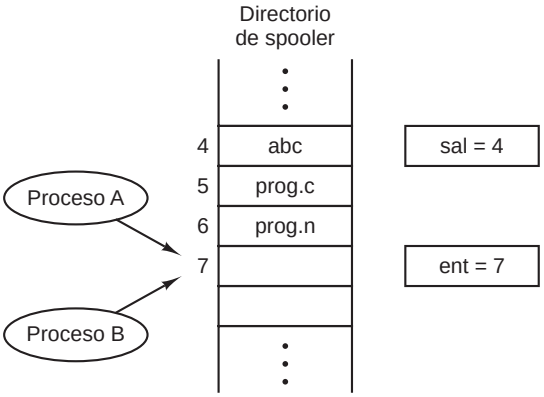


Figura 2-21. Dos procesos desean acceder a la memoria compartida al mismo tiempo.

En las jurisdicciones en las que se aplica la ley de Murphy† podría ocurrir lo siguiente. El proceso *A* lee *ent* y guarda el valor 7 en una variable local, llamada *siguiente_ranura_libre*. Justo entonces ocurre una interrupción de reloj y la CPU decide que el proceso *A* se ha ejecutado durante un tiempo suficiente, por lo que conmuta al proceso *B*. El proceso *B* también lee *ent* y también obtiene un 7. De igual forma lo almacena en *su* variable local *siguiente_ranura_libre*. En este instante, ambos procesos piensan que la siguiente ranura libre es la 7.

Ahora el proceso *B* continúa su ejecución. Almacena el nombre de su archivo en la ranura 7 y actualiza *ent* para que sea 8. Después realiza otras tareas.

En cierto momento el proceso *A* se ejecuta de nuevo, partiendo del lugar en el que se quedó. Busca en *siguiente_ranura_libre*, encuentra un 7 y escribe el nombre de su archivo en la ranura 7, borrando el nombre que el proceso *B* acaba de poner ahí. Luego calcula *siguiente_ranura_libre* + 1, que es 8 y fija *ent* para que sea 8. El directorio de spooler es ahora internamente consistente, por

† Si algo puede salir mal, lo hará.

www.FreeLibros.me

lo que el demonio de impresión no detectará nada incorrecto, pero el proceso *B* nunca recibirá ninguna salida.

El usuario *B* esperará en el cuarto de impresora por años, deseando con vehemencia obtener la salida que nunca llegará. Situaciones como ésta, en donde dos o más procesos están leyendo o escribiendo algunos datos compartidos y el resultado final depende de quién se ejecuta y exactamente cuándo lo hace, se conocen como **condiciones de carrera**. Depurar programas que contienen condiciones de carrera no es nada divertido. Los resultados de la mayoría de las ejecuciones de prueba están bien, pero en algún momento poco frecuente ocurrirá algo extraño e inexplicable.

2.3.2 Regiones críticas

¿Cómo evitamos las condiciones de carrera? La clave para evitar problemas aquí y en muchas otras situaciones en las que se involucran la memoria compartida, los archivos compartidos y todo lo demás compartido es buscar alguna manera de prohibir que más de un proceso lea y escriba los datos compartidos al mismo tiempo. Dicho en otras palabras, lo que necesitamos es **exclusión mutua**, cierta forma de asegurar que si un proceso está utilizando una variable o archivo compartido, los demás procesos se excluirán de hacer lo mismo. La dificultad antes mencionada ocurrió debido a que el proceso *B* empezó a utilizar una de las variables compartidas antes de que el proceso *A* terminara con ella. La elección de operaciones primitivas apropiadas para lograr la exclusión mutua es una cuestión de diseño importante en cualquier sistema operativo y un tema que analizaremos con mayor detalle en las siguientes secciones.

El problema de evitar las condiciones de carrera también se puede formular de una manera abstracta. Parte del tiempo, un proceso está ocupado realizando cálculos internos y otras cosas que no producen condiciones de carrera. Sin embargo, algunas veces un proceso tiene que acceder a la memoria compartida o a archivos compartidos, o hacer otras cosas críticas que pueden producir carreras. Esa parte del programa en la que se accede a la memoria compartida se conoce como **región crítica** o **sección crítica**. Si pudiéramos ordenar las cosas de manera que dos procesos nunca estuvieran en sus regiones críticas al mismo tiempo, podríamos evitar las carreras.

Aunque este requerimiento evita las condiciones de carrera, no es suficiente para que los procesos en paralelo cooperen de la manera correcta y eficiente al utilizar datos compartidos. Necesitamos cumplir con cuatro condiciones para tener una buena solución:

1. No puede haber dos procesos de manera simultánea dentro de sus regiones críticas.

2. No pueden hacerse suposiciones acerca de las velocidades o el número de CPUs.

3. Ningún proceso que se ejecute fuera de su región crítica puede bloquear otros procesos.

4. Ningún proceso tiene que esperar para siempre para entrar a su región crítica.

En sentido abstracto, el comportamiento que deseamos se muestra en la figura 2-22. Aquí el proceso *A* entra a su región crítica en el tiempo T_1. Un poco después, en el tiempo T_2 el proceso *B* intenta entrar a su región crítica, pero falla debido a que otro proceso ya se encuentra en su

www.FreeLibros.me

región crítica y sólo se permite uno a la vez. En consecuencia, *B* se suspende temporalmente hasta el tiempo T_3 cuando *A* sale de su región crítica, con lo cual se permite a *B* entrar de inmediato. En algún momento dado *B* sale (en T_4) y regresamos a la situación original, sin procesos en sus regiones críticas.

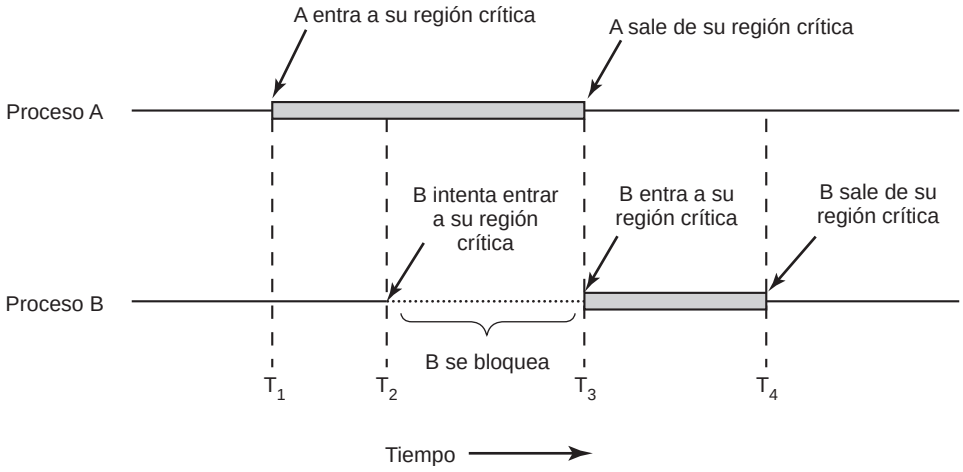


Figura 2-22. Exclusión mutua mediante el uso de regiones críticas.

2.3.3 Exclusión mutua con espera ocupada

En esta sección examinaremos varias proposiciones para lograr la exclusión mutua, de manera que mientras un proceso esté ocupado actualizando la memoria compartida en su región crítica, ningún otro proceso puede entrar a *su* región crítica y ocasionar problemas.

Deshabilitando interrupciones

En un sistema con un solo procesador, la solución más simple es hacer que cada proceso deshabilite todas las interrupciones justo después de entrar a su región crítica y las rehabilite justo después de salir. Con las interrupciones deshabilitadas, no pueden ocurrir interrupciones de reloj. Después de todo, la CPU sólo se conmuta de un proceso a otro como resultado de una interrupción del reloj o de otro tipo, y con las interrupciones desactivadas la CPU no se conmutará a otro proceso. Por ende, una vez que un proceso ha deshabilitado las interrupciones, puede examinar y actualizar la memoria compartida sin temor de que algún otro proceso intervenga.

Por lo general este método es poco atractivo, ya que no es conveniente dar a los procesos de usuario el poder para desactivar las interrupciones. Suponga que uno de ellos lo hiciera y nunca las volviera a activar. Ése podría ser el fin del sistema; aún más: si el sistema es un multiprocesador (con dos o posiblemente más CPUs), al deshabilitar las interrupciones sólo se ve afectada la CPU que ejecutó la instrucción disable. Las demás continuarán ejecutándose y pueden acceder a la memoria compartida.

www.FreeLibros.me

Por otro lado, con frecuencia es conveniente para el mismo kernel deshabilitar las interrupciones por unas cuantas instrucciones mientras actualiza variables o listas. Por ejemplo, si ocurriera una interrupción mientras la lista de procesos se encuentra en un estado inconsistente, podrían producirse condiciones de carrera. La conclusión es que a menudo deshabilitar interrupciones es una técnica útil dentro del mismo sistema operativo, pero no es apropiada como mecanismo de exclusión mutua general para los procesos de usuario.

La posibilidad de lograr la exclusión mutua al deshabilitar las interrupciones (incluso dentro del kernel) está disminuyendo día con día debido al creciente número de chips multinúcleo que se encuentran hasta en las PCs de bajo rendimiento. Ya es común que haya dos núcleos, las máquinas actuales de alto rendimiento tienen cuatro y dentro de poco habrá ocho o 16. En un multinúcleo (es decir, sistema con multiprocesadores) al deshabilitar las interrupciones de una CPU no se evita que las demás CPUs interfieran con las operaciones que la primera CPU está realizando. En consecuencia, se requieren esquemas más sofisticados.

Variables de candado

Como segundo intento, busquemos una solución de software. Considere tener una sola variable compartida (de candado), que al principio es 0. Cuando un proceso desea entrar a su región crítica primero evalúa el candado. Si este candado es 0, el proceso lo fija en 1 y entra a la región crítica. Si el candado ya es 1 sólo espera hasta que el candado se haga 0. Por ende, un 0 significa que ningún proceso está en su región crítica y un 1 significa que algún proceso está en su región crítica.

Por desgracia, esta idea contiene exactamente el mismo error fatal que vimos en el directorio de spooler. Suponga que un proceso lee el candado y ve que es 0. Antes de que pueda fijar el candado a 1, otro proceso se planifica para ejecutarse y fija el candado a 1. Cuando el primer proceso se ejecuta de nuevo, también fija el candado a 1 y por lo tanto dos procesos se encontrarán en sus regiones críticas al mismo tiempo.

Ahora el lector podría pensar que podemos resolver este problema si leemos primero el valor de candado y después lo verificamos de nuevo justo antes de almacenar el nuevo valor en él, pero en realidad eso no ayuda. La condición de carrera se produce ahora si el segundo proceso modifica el candado justo después que el primer proceso haya terminado su segunda verificación.

Alternancia estricta

En la figura 2-23 se muestra la tercera solución para el problema de la exclusión mutua. Este fragmento de programa, al igual que casi todos los demás en este libro, está escrito en C. Aquí elegimos el lenguaje C debido a que los sistemas operativos reales casi siempre se escriben en C (o en algunas ocasiones, en C++), pero casi nunca en lenguajes como Java, Modula 3 o Pascal. C es potente, eficiente y predecible, características cruciales para escribir sistemas operativos. Por ejemplo, Java no es predecible debido a que se podría agotar el espacio de almacenamiento en un momento crítico y tendría que invocar al recolector de basura para reclamar la memoria en un momento verdaderamente inoportuno. Esto no puede ocurrir en C, debido a que no hay recolección

www.FreeLibros.me

de basura. Una comparación cuantitativa de C, C++, Java y cuatro lenguajes más se proporciona en (Prechelt, 2000).

```
while (TRUE) {
    while (turno != 0)      /* ciclo */ ;
    region_critica();
    turno = 1;
    region_nocritica();
}
```

(a)

```
while (TRUE) {
    while (turno != 1)      /* ciclo */ ;
    region_critica();
    turno = 0;
    region_nocritica();
}
```

(b)

Figura 2-23. Una solución propuesta para el problema de la región crítica. (a) Proceso 0. (b) Proceso 1. En ambos casos, asegúrese de tener en cuenta los signos de punto y coma que terminan las instrucciones `while`.

En la figura 2-23, la variable entera *turno* (que al principio es 0) lleva la cuenta acerca de a qué proceso le toca entrar a su región crítica y examinar o actualizar la memoria compartida. Al principio, el proceso 0 inspecciona *turno*, descubre que es 0 y entra a su región crítica. El proceso 1 también descubre que es 0 y por lo tanto se queda en un ciclo estrecho, evaluando *turno* en forma continua para ver cuándo se convierte en 1. A la acción de evaluar en forma continua una variable hasta que aparezca cierto valor se le conoce como **espera ocupada**. Por lo general se debe evitar, ya que desperdicia tiempo de la CPU. La espera ocupada sólo se utiliza cuando hay una expectativa razonable de que la espera será corta. A un candado que utiliza la espera ocupada se le conoce como **candado de giro**.

Cuando el proceso 0 sale de la región crítica establece *turno* a 1, para permitir que el proceso 1 entre a su región crítica. Suponga que el proceso 1 sale rápidamente de su región crítica, de manera que ambos procesos se encuentran en sus regiones no críticas, con *turno* establecido en 0. Ahora el proceso 0 ejecuta todo su ciclo rápidamente, saliendo de su región crítica y estableciendo *turno* a 1. En este punto, *turno* es 1 y ambos procesos se están ejecutando en sus regiones no críticas.

De repente, el proceso 0 termina en su región no crítica y regresa a la parte superior de su ciclo. Por desgracia no puede entrar a su región crítica ahora, debido a que *turno* es 1 y el proceso 1 está ocupado con su región no crítica. El proceso 0 espera en su ciclo `while` hasta que el proceso 1 establezca *turno* a 0. Dicho en forma distinta, tomar turnos no es una buena idea cuando uno de los procesos es mucho más lento que el otro.

Esta situación viola la condición 3 antes establecida: el proceso 0 está siendo bloqueado por un proceso que no está en su región crítica. Volviendo al directorio de spooler antes mencionado, si ahora asociamos la región crítica con la lectura y escritura del directorio de spooler, el proceso 0 no podría imprimir otro archivo debido a que el proceso 1 está haciendo otra cosa.

De hecho, esta solución requiere que los dos procesos se alternen de manera estricta al entrar en sus regiones críticas (por ejemplo, al poner archivos en la cola de impresión). Ninguno podría poner dos archivos en la cola al mismo tiempo. Aunque este algoritmo evita todas las condiciones de carrera, en realidad no es un candidato serio como solución, ya que viola la condición 3.

www.FreeLibros.me

Solución de Peterson

Al combinar la idea de tomar turnos con la idea de las variables de candado y las variables de advertencia, un matemático holandés llamado T. Dekker fue el primero en idear una solución de software para el problema de la exclusión mutua que no requiere de una alternancia estricta. Para ver un análisis del algoritmo de Dekker, consulte (Dijkstra, 1965).

En 1981, G.L. Peterson descubrió una manera mucho más simple de lograr la exclusión mutua, con lo cual la solución de Dekker se hizo obsoleta. El algoritmo de Peterson se muestra en la figura 2-24. Este algoritmo consiste de dos procedimientos escritos en ANSI C, lo cual significa que se deben suministrar prototipos para todas las funciones definidas y utilizadas. Sin embargo, para ahorrar espacio no mostraremos los prototipos en este ejemplo ni en los siguientes.

```
#define FALSE 0
#define TRUE 1
#define N   2                                   /* número de procesos */

int turno;                                      /* ¿de quién es el turno? */
int interesado[N];                              /* al principio todos los valores son 0 (FALSE) */

void entrar_region(int proceso);                /* el proceso es 0 o 1 */
{
    int otro;                                   /* número del otro proceso */

    otro = 1 – proceso;                         /* el opuesto del proceso */
    interesado[proceso] = TRUE;                 /* muestra que está interesado */
    turno = proceso;                            /* establece la bandera */
    while (turno == proceso && interesado[otro] == TRUE)  /* instrucción nula */;
}

void salir_region(int proceso)                  /* proceso: quién está saliendo */
{
    interesado[proceso] = FALSE;                /* indica que salió de la región crítica */
}
```

Figura 2-24. Solución de Peterson para lograr la exclusión mutua.

Antes de utilizar las variables compartidas (es decir, antes de entrar a su región crítica), cada proceso llama a *entrar_region* con su propio número de proceso (0 o 1) como parámetro. Esta llamada hará que espere, si es necesario, hasta que sea seguro entrar. Una vez que haya terminado con las variables compartidas, el proceso llama a *salir_region* para indicar que ha terminado y permitir que los demás procesos entren, si así lo desea.

Veamos cómo funciona esta solución. Al principio ningún proceso se encuentra en su región crítica. Ahora el proceso 0 llama a *entrar_region*. Indica su interés estableciendo su elemento del arreglo y fija *turno* a 0. Como el proceso 1 no está interesado, *entrar_region* regresa de inmediato. Si ahora el proceso 1 hace una llamada a *entrar_region*, se quedará ahí hasta que *interesado*[0] sea

www.FreeLibros.me

FALSE, un evento que sólo ocurre cuando el proceso 0 llama a *salir_region* para salir de la región crítica.

Ahora considere el caso en el que ambos procesos llaman a *entrar_region* casi en forma simultánea. Ambos almacenarán su número de proceso en *turno*. Cualquier almacenamiento que se haya realizado al último es el que cuenta; el primero se sobrescribe y se pierde. Suponga que el proceso 1 almacena al último, por lo que *turno* es 1. Cuando ambos procesos llegan a la instrucción `while`, el proceso 0 la ejecuta 0 veces y entra a su región crítica. El proceso 1 itera y no entra a su región crítica sino hasta que el proceso 0 sale de su región crítica.

La instrucción TSL

Ahora veamos una proposición que requiere un poco de ayuda del hardware. Algunas computadoras, en especial las diseñadas con varios procesadores en mente, tienen una instrucción tal como

```
TSL REGISTRO, CANDADO
```

(Evaluar y fijar el candado) que funciona de la siguiente manera. Lee el contenido de la palabra de memoria *candado* y lo guarda en el registro `RX`, y después almacena un valor distinto de cero en la dirección de memoria *candado*. Se garantiza que las operaciones de leer la palabra y almacenar un valor en ella serán indivisibles; ningún otro procesador puede acceder a la palabra de memoria sino hasta que termine la instrucción. La CPU que ejecuta la instrucción `TSL` bloquea el bus de memoria para impedir que otras CPUs accedan a la memoria hasta que termine.

Es importante observar que bloquear el bus de memoria es una acción muy distinta de la de deshabilitar las interrupciones. Al deshabilitar las interrupciones y después realizar una operación de lectura en una palabra de memoria, seguida de una operación de escritura, no se evita que un segundo procesador en el bus acceda a la palabra entre la lectura y la escritura. De hecho, si se deshabilitan las interrupciones en el procesador 1 no se produce efecto alguno en el procesador 2. La única forma de mantener el procesador 2 fuera de la memoria hasta que el procesador 1 termine es bloquear el bus, para lo cual se requiere una herramienta de hardware especial (básicamente, una línea de bus que afirme que el bus está bloqueado y no disponible para los demás procesadores aparte del que lo bloqueó).

Para usar la instrucción `TSL` necesitamos una variable compartida (*candado*) que coordine el acceso a la memoria compartida. Cuando *candado* es 0, cualquier proceso lo puede fijar en 1 mediante el uso de la instrucción `TSL` y después una lectura o escritura en la memoria compartida. Cuando termina, el proceso establece *candado* de vuelta a 0 mediante una instrucción `move` ordinaria.

¿Cómo se puede utilizar esta instrucción para evitar que dos procesos entren al mismo tiempo en sus regiones críticas? La solución se proporciona en la figura 2-25, donde se muestra una subrutina de cuatro instrucciones en un lenguaje ensamblador ficticio (pero común). La primera instrucción copia el antiguo valor de *candado* en el registro y después fija el *candado* a 1. Después, el valor anterior se compara con 0. Si es distinto de cero, el candado ya estaba cerrado, por lo que el programa sólo regresa al principio y lo vuelve a evaluar. Tarde o temprano se volverá 0 (cuando el proceso que esté actualmente en su región crítica se salga de ella) y la subrutina regresará, con el bloqueo establecido. Es muy simple quitar el bloqueo. El programa sólo almacena un 0 en *candado*. No se necesitan instrucciones especiales de sincronización.

www.FreeLibros.me

```
entrar_region:
    TSL REGISTRO,CANDADO        |copia candado al registro y fija candado a 1
    CMP REGISTRO,#0             |¿era candado cero?
    JNE entrar_region           |si era distinto de cero, el candado está cerrado, y se repite
    RET                         |regresa al llamador; entra a región crítica

salir_region:
    MOVE CANDADO,#0             |almacena 0 en candado
    RET                         |regresa al llamador
```

Figura 2-25. Cómo entrar y salir de una región crítica mediante la instrucción TSL.

Ahora tenemos una solución simple y directa para el problema de regiones críticas. Antes de entrar a su región crítica, un proceso llama a *entrar_region*, que lleva a cabo una espera ocupada hasta que el candado está abierto; después adquiere el candado y regresa. Después de la región crítica, el proceso llama a *salir_region*, que almacena un 0 en *candado*. Al igual que con todas las soluciones basadas en regiones críticas, los procesos deben llamar a *entrar_region* y *salir_region* en los momentos correctos para que el método funcione. Si un proceso hace trampa, la exclusión mutua fallará.

Una instrucción alternativa para `TSL` es `XCHG`, que intercambia el contenido de dos ubicaciones en forma atómica; por ejemplo, un registro y una palabra de memoria. El código se muestra en la figura 2-26 y como se puede ver, es en esencia el mismo que la solución con `TSL`. Todas las CPUs Intel x86 utilizan la instrucción `XCHG` para la sincronización de bajo nivel.

```
entrar_region:
    MOVE REGISTRO,#1            |coloca 1 en el registro
    XCHG REGISTRO,CANDADO       |intercambia el contenido del registro y la variable candado
    CMP REGISTRO,#0             |¿era candado cero?
    JNE entrar_region           |si era distinto de cero, el candado está cerrado, y se repite
    RET                         |regresa al que hizo la llamada; entra a región crítica

salir_region:
  MOVE CANDADO,#0               |almacena 0 en candado
  RET                           |regresa al que hizo la llamada
```

Figura 2-26. Cómo entrar y salir de una región crítica mediante la instrucción XCHG.

2.3.4 Dormir y despertar

Tanto la solución de Peterson como las soluciones mediante `TSL` o `XCHG` son correctas, pero todas tienen el defecto de requerir la espera ocupada. En esencia, estas soluciones comprueban si se

www.FreeLibros.me

permite la entrada cuando un proceso desea entrar a su región crítica. Si no se permite, el proceso sólo espera en un ciclo estrecho hasta que se permita la entrada.

Este método no sólo desperdicia tiempo de la CPU, sino que también puede tener efectos inesperados. Considere una computadora con dos procesos: *H* con prioridad alta y *L* con prioridad baja. Las reglas de planificación son tales que *H* se ejecuta cada vez que se encuentra en el estado listo. En cierto momento, con *L* en su región crítica, *H* cambia al estado listo para ejecutarse (por ejemplo, cuando se completa una operación de E/S). Entonces *H* empieza la espera ocupada, pero como *L* nunca se planifica mientras *H* está en ejecución, *L* nunca tiene la oportunidad de salir de su región crítica, por lo que *H* itera en forma indefinida. A esta situación se le conoce algunas veces como el **problema de inversión de prioridades**.

Veamos ahora ciertas primitivas de comunicación entre procesos que bloquean en vez de desperdiciar tiempo de la CPU cuando no pueden entrar a sus regiones críticas. Una de las más simples es el par `sleep` (dormir) y `wakeup` (despertar). `Sleep` es una llamada al sistema que hace que el proceso que llama se bloquee o desactive, es decir, que se suspenda hasta que otro proceso lo despierte. La llamada `wakeup` tiene un parámetro, el proceso que se va a despertar o activar. De manera alternativa, tanto `sleep` como `wakeup` tienen un parámetro, una dirección de memoria que se utiliza para asociar las llamadas a `sleep` con las llamadas a `wakeup`.

El problema del productor-consumidor

Como ejemplo de la forma en que se pueden utilizar estas primitivas, consideremos el problema del **productor-consumidor** (conocido también como el problema del **búfer limitado**). Dos procesos comparten un búfer común, de tamaño fijo. Uno de ellos (el productor) coloca información en el búfer y el otro (el consumidor) la saca (también es posible generalizar el problema de manera que haya *m* productores y *n* consumidores, pero sólo consideraremos el caso en el que hay un productor y un consumidor, ya que esta suposición simplifica las soluciones).

El problema surge cuando el productor desea colocar un nuevo elemento en el búfer, pero éste ya se encuentra lleno. La solución es que el productor se vaya a dormir (se desactiva) y que se despierte (se active) cuando el consumidor haya quitado uno o más elementos. De manera similar, si el consumidor desea quitar un elemento del búfer y ve que éste se encuentra vacío, se duerme hasta que el productor coloca algo en el búfer y lo despierta.

Este método suena lo bastante simple, pero produce los mismos tipos de condiciones de carrera que vimos antes con el directorio de spooler. Para llevar la cuenta del número de elementos en el búfer, necesitamos una variable (*cuenta*). Si el número máximo de elementos que puede contener el búfer es *N*, el código del productor comprueba primero si *cuenta* es *N*. Si lo es, el productor se duerme; si no lo es, el productor agrega un elemento e incrementará *cuenta*.

El código del consumidor es similar: primero evalúa *cuenta* para ver si es 0. Si lo es, se duerme; si es distinta de cero, quita un elemento y disminuye el contador cuenta. Cada uno de los procesos también comprueba si el otro se debe despertar y de ser así, lo despierta. El código para el productor y el consumidor se muestra en la figura 2-27.

Para expresar llamadas al sistema como `sleep` y `wakeup` en C, las mostraremos como llamadas a rutinas de la biblioteca. No forman parte de la biblioteca estándar de C, pero es de suponer que estarán disponibles en cualquier sistema que tenga realmente estas llamadas al sistema.

www.FreeLibros.me

```
#define N 100                                   /* número de ranuras en el búfer */
int cuenta = 0;                                 /* número de elementos en el búfer */

void productor(void)
{
     int elemento;

     while (TRUE) {                             /* se repite en forma indefinida */
          elemento = producir_elemento();       /* genera el siguiente elemento */
          if (cuenta == N) sleep();             /* si el búfer está lleno, pasa a inactivo */
          insertar_elemento(elemento);          /* coloca elemento en búfer */
          cuenta = cuenta + 1;                  /* incrementa cuenta de elementos en búfer */
          if (cuenta == 1) wakeup(consumidor);  /* ¿estaba vacío el búfer? */
     }
}


void consumidor(void)
{
     int elemento;

     while (TRUE) {                             /* se repite en forma indefinida */
          if (cuenta == 0) sleep();             /* si búfer está vacío, pasa a inactivo */
          elemento = quitar_elemento();         /* saca el elemento del búfer */
          cuenta = cuenta – 1;                  /* disminuye cuenta de elementos en búfer */
          if (cuenta==N–1) wakeup(productor);   /* ¿estaba lleno el búfer? */
          consumir_elemento(elemento);          /* imprime el elemento */
     }
}
```

Figura 2-27. El problema del productor-consumidor con una condición de carrera fatal.

Los procedimientos *insertar_elemento* y *quitar_elemento*, que no se muestran aquí, se encargan de colocar elementos en el búfer y sacarlos del mismo.

Ahora regresemos a la condición de carrera. Puede ocurrir debido a que el acceso a *cuenta* no está restringido. Es posible que ocurra la siguiente situación: el búfer está vacío y el consumidor acaba de leer *cuenta* para ver si es 0. En ese instante, el planificador decide detener al consumidor en forma temporal y empieza a ejecutar el productor. El productor inserta un elemento en el búfer, incrementa *cuenta* y observa que ahora es 1. Razonando que *cuenta* era antes 0, y que por ende el consumidor debe estar dormido, el productor llama a *wakeup* para despertar al consumidor.

Por desgracia, el consumidor todavía no está lógicamente dormido, por lo que la señal para despertarlo se pierde. Cuando es turno de que se ejecute el consumidor, evalúa el valor de *cuenta* que leyó antes, encuentra que es 0 y pasa a dormirse. Tarde o temprano el productor llenará el búfer y también pasará a dormirse. Ambos quedarán dormidos para siempre.

www.FreeLibros.me

La esencia del problema aquí es que una señal que se envía para despertar a un proceso que no está dormido (todavía) se pierde. Si no se perdiera, todo funcionaría. Una solución rápida es modificar las reglas para agregar al panorama un **bit de espera de despertar**. Cuando se envía una señal de despertar a un proceso que sigue todavía despierto, se fija este bit. Más adelante, cuando el proceso intenta pasar a dormir, si el bit de espera de despertar está encendido, se apagará pero el proceso permanecerá despierto. Este bit es una alcancía para almacenar señales de despertar.

Aunque el bit de espera de despertar logra su cometido en este ejemplo simple, es fácil construir ejemplos con tres o más procesos en donde un bit de espera de despertar es insuficiente. Podríamos hacer otra modificación y agregar un segundo bit de espera de despertar, tal vez hasta 8 o 32 de ellos, pero en principio el problema sigue ahí.

2.3.5 Semáforos

Esta era la situación en 1965, cuando E. W. Dijkstra (1965) sugirió el uso de una variable entera para contar el número de señales de despertar, guardadas para un uso futuro. En su propuesta introdujo un nuevo tipo de variable, al cual él le llamó **semáforo**. Un semáforo podría tener el valor 0, indicando que no se guardaron señales de despertar o algún valor positivo si estuvieran pendientes una o más señales de despertar.

Dijkstra propuso que se tuvieran dos operaciones, `down` y `up` (generalizaciones de `sleep` y `wakeup`, respectivamente). La operación `down` en un semáforo comprueba si el valor es mayor que 0. De ser así, disminuye el valor (es decir, utiliza una señal de despertar almacenada) y sólo continúa. Si el valor es 0, el proceso se pone a dormir sin completar la operación `down` por el momento. Las acciones de comprobar el valor, modificarlo y posiblemente pasar a dormir, se realizan en conjunto como una sola **acción atómica** indivisible. Se garantiza que, una vez que empieza una operación de semáforo, ningún otro proceso podrá acceder al semáforo sino hasta que la operación se haya completado o bloqueado. Esta atomicidad es absolutamente esencial para resolver problemas de sincronización y evitar condiciones de carrera. Las acciones atómicas, en las que un grupo de operaciones relacionadas se realizan sin interrupción o en definitiva no se realizan, son en extremo importantes en muchas otras áreas de las ciencias computacionales también.

La operación `up` incrementa el valor del semáforo direccionado. Si uno o más procesos estaban inactivos en ese semáforo, sin poder completar una operación `down` anterior, el sistema selecciona uno de ellos (al azar) y permite que complete su operación `down`. Así, después de una operación `up` en un semáforo que contenga procesos dormidos, el semáforo seguirá en 0 pero habrá un proceso menos dormido en él. La operación de incrementar el semáforo y despertar a un proceso también es indivisible. Ningún proceso se bloquea al realizar una operación `up`, de igual forma que ningún proceso se bloquea realizando una operación `wakeup` en el modelo anterior.

Como punto adicional, en su artículo original, Dijsktra utilizó los nombres `P` y `V` en vez de `down` y `up`, respectivamente. Como éstos no tienen un significado nemónico para las personas que no hablan holandés y sólo tienen un significado marginal para aquellos que lo hablan [*Proberen* (intentar) y *Verhogen* (elevar, elevar más)], utilizaremos los términos `down` y `up` en su defecto. Éstos se introdujeron por primera vez en el lenguaje de programación Algol 68.

www.FreeLibros.me

Cómo resolver el problema del productor-consumidor mediante el uso de semáforos

Los semáforos resuelven el problema de pérdida de señales de despertar, como se muestra en la figura 2-28. Para que funcionen de manera correcta, es esencial que se implementen de una forma indivisible. Lo normal es implementar `up` y `down` como llamadas al sistema, en donde el sistema operativo deshabilita brevemente todas las interrupciones, mientras evalúa el semáforo, lo actualiza y pone el proceso a dormir, si es necesario. Como todas estas acciones requieren sólo unas cuantas instrucciones, no hay peligro al deshabilitar las interrupciones. Si se utilizan varias CPUs, cada semáforo debe estar protegido por una variable de candado, en donde se utilicen las instrucciones `TSL` o `XCHG` para asegurar que sólo una CPU a la vez pueda examinar el semáforo.

Asegúrese de comprender que el uso de `TSL` o `XCHG`, para evitar que varias CPUs tengan acceso al semáforo al mismo tiempo, es algo muy distinto al caso en el que el productor o el consumidor están en espera ocupada, esperando a que el otro vacíe o llene el búfer. La operación de semáforo sólo tarda unos cuantos microsegundos, mientras que el productor o el consumidor podrían tardarse una cantidad de tiempo arbitraria y extensa.

Esta solución utiliza tres semáforos: uno llamado *llenas* para contabilizar el número de ranuras llenas, otro llamado *vacías* para contabilizar el número de ranuras vacías y el último llamado *mutex*, para asegurar que el productor y el consumidor no tengan acceso al búfer al mismo tiempo. Al principio *llenas* es 0, *vacías* es igual al número de ranuras en el búfer y *mutex* es 1. Los semáforos que se inicializan a 1 y son utilizados por dos o más procesos para asegurar que sólo uno de ellos pueda entrar a su región crítica en un momento dado se llaman **semáforos binarios**. Si cada proceso realiza una operación `down` justo antes de entrar a su región crítica y una operación `up` justo después de salir de ella, se garantiza la exclusión mutua.

Ahora que tenemos una buena primitiva de comunicación entre procesos a nuestra disposición, volvamos a analizar la secuencia de interrupción de la figura 2-5. En un sistema que utiliza semáforos, la forma natural de ocultar las interrupciones es asociar un semáforo (que al principio es 0) con cada dispositivo de E/S. Justo después de iniciar un dispositivo de E/S, el proceso administrativo realiza una operación `down` en el semáforo asociado, con lo cual se bloquea de inmediato. Cuando entra la interrupción, el manejador de interrupciones realiza una operación `up` en el semáforo asociado, lo cual hace que el proceso relevante esté listo para ejecutarse de nuevo. En este modelo, el paso 5 de la figura 2-5 consiste en realizar una operación `up` en el semáforo del dispositivo, de manera que en el paso 6 el planificador pueda ejecutar el administrador de dispositivos. Desde luego que, si hay varios procesos listos en ese momento, el planificador puede optar por ejecutar un proceso aún más importante a continuación. Más adelante en este capítulo analizaremos algunos de los algoritmos que se utilizan para la planificación.

En el ejemplo de la figura 2-28, en realidad hemos utilizado semáforos de dos maneras distintas. Esta diferencia es lo bastante importante como para recalcarla explícitamente. El semáforo *mutex* se utiliza para la exclusión mutua. Está diseñado para garantizar que sólo un proceso pueda leer o escribir en el búfer y sus variables asociadas en un momento dado. Esta exclusión mutua es requerida para evitar un caos. En la siguiente sección estudiaremos la exclusión mutua y cómo podemos lograrla.

www.FreeLibros.me

```
#define N 100                                   /* número de ranuras en el búfer */
typedef int semaforo;                           /* los semáforos son un tipo especial de int */
semaforo mutex = 1;                             /* controla el acceso a la región crítica */
semaforo vacias = N;                            /* cuenta las ranuras vacías del búfer */
semaforo llenas = 0;                            /* cuenta las ranuras llenas del búfer */

void productor(void)
{
     int elemento;

     while(TRUE){                               /* TRUE es la constante 1 */
          elemento = producir_elemento();       /* genera algo para colocar en el búfer */
          down(&vacias);                        /* disminuye la cuenta de ranuras vacías */
          down(&mutex);                         /* entra a la región crítica */
          insertar_elemento(elemento);          /* coloca el nuevo elemento en el búfer */
          up(&mutex);                           /* sale de la región crítica */
          up(&llenas);                          /* incrementa la cuenta de ranuras llenas */
     }
}


void consumidor(void)
{
     int elemento;

     while(TRUE){                               /* ciclo infinito */
          down(&llenas);                        /* disminuye la cuenta de ranuras llenas */
          down(&mutex);                         /* entra a la región crítica */
          elemento = quitar_elemento();         /* saca el elemento del búfer */
          up(&mutex);                           /* sale de la región crítica */
          up(&vacias);                          /* incrementa la cuenta de ranuras vacías */
          consumir_elemento(elemento);          /* hace algo con el elemento */
     }
}
```

Figura 2-28. El problema del productor-consumidor mediante el uso de semáforos.

El otro uso de los semáforos es para la **sincronización**. Los semáforos *vacías* y *llenas* se necesitan para garantizar que ciertas secuencias de eventos ocurran o no. En este caso, aseguran que el productor deje de ejecutarse cuando el búfer esté lleno y que el consumidor deje de ejecutarse cuando el búfer esté vacío. Este uso es distinto de la exclusión mutua.

2.3.6 Mutexes

Cuando no se necesita la habilidad del semáforo de contar, algunas veces se utiliza una versión simplificada, llamada mutex. Los *mutexes* son buenos sólo para administrar la exclusión mutua para cierto recurso compartido o pieza de código. Se implementan con facilidad y eficiencia, lo cual

www.FreeLibros.me

hace que sean especialmente útiles en paquetes de hilos que se implementan en su totalidad en espacio de usuario.

Un **mutex** es una variable que puede estar en uno de dos estados: abierto (desbloqueado) o cerrado (bloqueado). En consecuencia, se requiere sólo 1 bit para representarla, pero en la práctica se utiliza con frecuencia un entero, en donde 0 indica que está abierto y todos los demás valores indican que está cerrado. Se utilizan dos procedimientos con los mutexes. Cuando un hilo (o proceso) necesita acceso a una región crítica, llama a *mutex_lock*. Si el mutex está actualmente abierto (lo que significa que la región crítica está disponible), la llamada tiene éxito y entonces el hilo llamador puede entrar a la región crítica.

Por otro lado, si el mutex ya se encuentra cerrado, el hilo que hizo la llamada se bloquea hasta que el hilo que está en la región crítica termine y llame a *mutex_unlock*. Si se bloquean varios hilos por el mutex, se selecciona uno de ellos al azar y se permite que adquiera el mutex.

Como los mutexes son tan simples, se pueden implementar con facilidad en espacio de usuario, siempre y cuando haya una instrucción `TSL` o `XCHG` disponible. El código para *mutex_lock* y *mutex_unlock* para utilizar estos procedimientos con un paquete de hilos de nivel usuario se muestra en la figura 2-29. La solución con `XCHG` es en esencia similar.

```
mutex_lock:
        TSL REGISTRO,MUTEX          |copia el mutex al registro y establece mutex a 1
        CMP REGISTRO,#0             |¿el mutex era 0?
        JZE ok                      |si era cero, el mutex estaba abierto, entonces regresa
        CALL thread_yield           |el mutex está ocupado; planifica otro hilo
        JMP mutex_lock              |intenta de nuevo
ok:     RET                         |regresa al procedimiento llamador; entra a la región crítica

mutex_unlock:
        MOVE MUTEX,#0               |almacena un 0 en el mutex
        RET                         |regresa al procedimiento llamador
```

Figura 2-29. Implementaciones de *mutex_lock* y *mutex_unlock*.

El código de *mutex_lock* es similar al código de *entrar_region* de la figura 2-25, pero con una diferencia crucial. Cuando *entrar_region* no puede entrar a la región crítica, continúa evaluando el mutex en forma repetida (espera ocupada). En algún momento dado el reloj se agotará y algún otro proceso se programará para ejecutarlo. Tarde o temprano el proceso que mantiene el mutex pasa a ejecutarse y lo libera.

Con los hilos (de usuario), la situación es distinta debido a que no hay un reloj que detenga los hilos que se han ejecutado por demasiado tiempo. En consecuencia, un hilo que intente adquirir un mutex mediante la espera ocupada iterará en forma indefinida y nunca adquirirá el mutex debido a que nunca permitirá que algún otro hilo se ejecute y libere el mutex.

Aquí es donde entra la diferencia entre *entrar_region* y *mutex_lock*. Cuando este último procedimiento no puede adquirir un mutex, llama a *thread_yield* para entregar la CPU a otro hilo. En consecuencia, no hay espera ocupada. Cuando el hilo se ejecuta la siguiente vez, evalúa el mutex de nuevo.

www.FreeLibros.me

Como *thread_yield* es sólo una llamada al planificador de hilos en espacio de usuario, es muy rápido. Como consecuencia, ni *mutex_lock* ni *mutex_unlock* requieren llamadas al kernel. Al utilizarlas, los hilos en nivel usuario pueden sincronizarse completamente en el espacio de usuario, utilizando procedimientos que sólo requieren unas cuantas instrucciones.

El sistema con mutex que hemos descrito antes es un conjunto mínimo de llamadas. Con todo el software, siempre hay demanda por más características y las primitivas de sincronización no son la excepción. Por ejemplo, algunas veces un paquete de hilos ofrece una llamada *mutex_trylock* que adquiere el mutex o devuelve un código de falla, pero no bloquea. Esta llamada otorga al hilo la flexibilidad de decidir qué hacer a continuación, si hay alternativas además de esperar.

Hasta ahora hay una sutil cuestión que hemos visto sólo en forma superficial, pero que es conveniente, por lo menos, hacer explícita. Con un paquete de hilos en espacio de usuario no hay problema con que varios hilos tengan acceso al mismo mutex, ya que todos los hilos operan en un espacio de direcciones común. Sin embargo, con la mayoría de las soluciones anteriores, como el algoritmo de Peterson y los semáforos, hay una suposición que no hemos mencionado acerca de que varios procesos tienen acceso cuando menos a una parte de la memoria compartida; tal vez sólo a una palabra, pero es algo. Si los procesos tienen espacios de direcciones que no están juntos, como hemos dicho de manera consistente, ¿cómo pueden compartir la variable *turno* en el algoritmo de Peterson o los semáforos en un búfer común?

Hay dos respuestas. En primer lugar, algunas de las estructuras de datos compartidas (como los semáforos) se pueden almacenar en el kernel y se accede a ellas sólo a través de llamadas al sistema. Este enfoque elimina el problema. En segundo lugar, la mayoría de los sistemas operativos modernos (incluyendo UNIX y Windows) ofrecen la manera de que los procesos compartan cierta porción de su espacio de direcciones con otros procesos. De esta forma se pueden compartir los búferes y otras estructuras de datos. En el peor de los casos, que nada sea posible, se puede utilizar un archivo compartido.

Si dos o más procesos comparten la mayoría de, o todos, sus espacios de direcciones, la distinción entre procesos e hilos se elimina casi por completo, pero sin duda está presente. Dos procesos que comparten un espacio de direcciones común siguen teniendo distintos archivos abiertos, temporizadores de alarma y otras propiedades correspondientes a cada proceso, mientras que los hilos dentro de un solo proceso las comparten. Y siempre es verdad que varios procesos que comparten un espacio de direcciones común nunca tendrán la eficiencia de los hilos de nivel usuario, ya que el kernel está muy involucrado en su administración.

Mutexes en Pthreads

Pthreads proporciona varias funciones que se pueden utilizar para sincronizar los hilos. El mecanismo básico utiliza una variable mutex, cerrada o abierta, para resguardar cada región crítica. Un hilo que desea entrar a una región crítica primero trata de cerrar el mutex asociado. Si el mutex está abierto, el hilo puede entrar de inmediato y el bloqueo se establece en forma automática, evitando que otros hilos entren; si el mutex ya se encuentra cerrado, el hilo que hizo la llamada se bloquea hasta que el mutex esté abierto. Si hay varios hilos esperando el mismo mutex, cuando está abierto sólo uno de ellos podrá continuar y volver a cerrarlo. Estos bloqueos no son obligatorios. Es responsabilidad del programador asegurar que los hilos los utilicen de manera correcta.

www.FreeLibros.me

Las principales llamadas relacionadas con los mutexes se muestran en la figura 2-30. Como es de esperarse, se pueden crear y destruir. Las llamadas para realizar estas operaciones son *pthread_mutex_init* y *pthread_mutex_destroy*, respectivamente. También se pueden cerrar mediante *pthread_mutex_lock*, que trata de adquirir el mutex y se bloquea si ya está cerrado. Además hay una opción para tratar de cerrar un mutex y falla con un código de error en vez de bloquearlo, si ya se encuentra bloqueado. Esta llamada es *pthread_mutex_trylock* y permite que un hilo realice en efecto la espera ocupada, si alguna vez se requiere. Por último, *Pthread_mutex_unlock* abre un mutex y libera exactamente un hilo si uno o más están en espera. Los mutexes también pueden tener atributos, pero éstos se utilizan sólo para propósitos especializados.

Llamada de hilo	Descripción
Pthread_mutex_init	Crea un mutex
Pthread_mutex_destroy	Destruye un mutex existente
Pthread_mutex_lock	Adquiere un mutex o se bloquea
Pthread_mutex_trylock	Adquiere un mutex o falla
Pthread_mutex_unlock	Libera un mutex

Figura 2-30. Algunas de las llamadas de Pthreads relacionadas con mutexes.

Además de los mutexes, pthreads ofrece un segundo mecanismo de sincronización: las **variables de condición**. Los mutexes son buenos para permitir o bloquear el acceso a una región crítica. Las variables de condición permiten que los hilos se bloqueen debido a que cierta condición no se está cumpliendo. Casi siempre se utilizan los dos métodos juntos. Veamos ahora la interacción de hilos, mutexes y variables de condición con un poco más de detalle.

Como un ejemplo simple, considere nuevamente el escenario del productor-consumidor: un hilo coloca cosas en un búfer y otro las saca. Si el productor descubre que no hay más ranuras libres disponibles en el búfer, tiene que bloquear hasta que haya una disponible. Los mutexes hacen que sea posible realizar la comprobación en forma atómica sin interferencia de los otros hilos, pero al descubrir que el búfer está lleno, el productor necesita una forma de bloquearse y ser despertado posteriormente. Esto es lo que permiten las variables de condición.

Algunas de las llamadas relacionadas con las variables de condición se muestran en la figura 2-31. Como podríamos esperar, hay llamadas para crear y destruir variables de condición. Pueden tener atributos y existen varias llamadas para administrarlas (que no se muestran). Las operaciones primarias en las variables de condición son *pthread_cond_wait* y *pthread_cond_signal*. La primera bloquea el hilo que hace la llamada hasta que algún otro hilo le señala (utilizando la segunda llamada). Desde luego que las razones por las que se debe bloquear y esperar no forman parte del protocolo de espera y señalización. A menudo el hilo que bloquea está esperando a que el hilo de señalización realice cierto trabajo, que libere algún recurso o realice alguna otra actividad. Sólo entonces podrá continuar el hilo bloqueado. Las variables de condición permiten que esta espera y

www.FreeLibros.me

bloqueo se realicen en forma atómica. La llamada a *pthread_cond_broadcast* se utiliza cuando hay varios hilos potencialmente bloqueados y esperan la misma señal.

Llamada de hilo	Descripción
Pthread_cond_init	Crea una variable de condición
Pthread_cond_destroy	Destruye una variable de condición
Pthread_cond_wait	Bloquea en espera de una señal
Pthread_cond_signal	Envia señal a otro hilo y lo despierta
Pthread_cond_broadcast	Envia señal a varios hilos y los despierta

Figura 2-31. Algunas de las llamadas a Pthreads que se relacionan con las variables de condición.

Las variables de condición y los mutexes siempre se utilizan en conjunto. El patrón es que un hilo cierre un mutex y después espere en una variable condicional cuando no pueda obtener lo que desea. En algún momento dado, otro hilo lo señalará y podrá continuar. La llamada a *pthread_cond_wait* cierra y abre en forma atómica el mutex que contiene. Por esta razón, el mutex es uno de los parámetros.

También vale la pena observar que las variables de condición (a diferencia de los semáforos) no tienen memoria. Si se envía una señal a una variable de condición en la que no haya un hilo en espera, se pierde la señal. Los programadores deben tener cuidado de no perder las señales.

Como ejemplo de la forma en que se utilizan los mutexes y las variables de condición, la figura 2-32 muestra un problema productor-consumidor muy simple con un solo búfer. Cuando el productor ha llenado el búfer, debe esperar hasta que el consumidor lo vacíe para poder producir el siguiente elemento. De manera similar, cuando el consumidor ha quitado un elemento, debe esperar hasta que el productor haya producido otro. Aunque es muy simple, este ejemplo ilustra los mecanismos básicos. La instrucción que pone a un hilo en estado inactivo siempre debe verificar la condición, para asegurarse de que se cumpla antes de continuar, ya que el hilo podría haber despertado debido a una señal de UNIX o por alguna otra razón.

2.3.7 Monitores

Con los semáforos y los mutexes la comunicación entre procesos se ve sencilla, ¿verdad? Olvídelo. Analice de cerca el orden de las operaciones `down` antes de insertar o quitar elementos del búfer en la figura 2-28. Suponga que se invirtió el orden de las dos operaciones `down` en el código del productor, de manera que *mutex* se disminuyó antes de *vacías*, en vez de hacerlo después. Si el búfer estuviera completamente lleno el productor se bloquearía, con *mutex* establecida en 0. En consecuencia, la próxima vez que el consumidor tratara de acceder al búfer, realizaría una operación `down` en *mutex* (que ahora es 0) y se bloquearía también. Ambos procesos permanecerían bloqueados de manera indefinida y no se realizaría más trabajo. Esta desafortunada situación se conoce como interbloqueo (*deadlock*). En el capítulo 6 estudiaremos los interbloqueos con detalle.

www.FreeLibros.me

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000                  /* cuántos números debe producir */
pthread_mutex_t el_mutex;
pthread_cond_t condc,condp;
int bufer = 0;                          /* búfer utilizado entre productor y consumidor */

void *productor(void *ptr)              /* produce datos */
{     int i;

      for (i=1; i<=MAX; i++) {
            pthread_mutex_lock(&el_mutex);     /* obtiene acceso exclusivo al búfer */
            while (bufer!=0) pthread_cond_wait(&condp, &el_mutex);
            bufer = i;                         /* coloca elemento en el búfer */
            pthread_cond_signal(&condc);       /* despierta al consumidor */
            pthread_mutex_unlock(&el_mutex);   /* libera el acceso al búfer */
      }
      pthread_exit(0);
}

void *consumidor(void *ptr)             /* consume datos */
{     int i;

      for (i=1; i<=MAX; i++) {
            pthread_mutex_lock(&el_mutex);     /* obtiene acceso exclusivo al búfer */
            while (bufer==0) pthread_cond_wait(&condc, &el_mutex);
            bufer = 0;                         /* saca elemento del búfer */
            pthread_cond_signal(&condp);       /* despierta al productor */
            pthread_mutex_unlock(&el_mutex);   /* libera el acceso al búfer */
      }
      pthread_exit(0);
}

int main(int argc, char **argv)
{
      pthread_t pro, con;
      pthread_mutex_init(&el_mutex, 0);
      pthread_cond_init(&condc, 0);
      pthread_cond_init(&condp, 0);
      pthread_create(&con, 0, consumidor, 0);
      pthread_create(&pro, 0, productor, 0);
      pthread_join(pro, 0);
      pthread_join(con, 0);
      pthread_cond_destroy(&condc);
      pthread_cond_destroy(&condp);
      pthread_mutex_destroy(&el_mutex);
}
```

Figura 2-32. Uso de hilos para resolver el problema del productor-consumidor.

www.FreeLibros.me

Resaltamos este problema para mostrar qué tan cuidadoso debe ser el programador al utilizar semáforos. Un ligero error y todo se detiene en forma abrupta. Es como la programación en lenguaje ensamblador, sólo que peor, debido a que los errores son condiciones de carrera, interbloqueos y otras formas de comportamiento impredecible e irreproducible.

Para facilitar la escritura de programas correctos, Brinch Hansen (1973) y Hoare (1974) propusieron una primitiva de sincronización de mayor nivel, conocida como **monitor**. Sus proposiciones tenían ligeras variaciones, como se describe a continuación. Un monitor es una colección de procedimientos, variables y estructuras de datos que se agrupan en un tipo especial de módulo o paquete. Los procesos pueden llamar a los procedimientos en un monitor cada vez que lo desean, pero no pueden acceder de manera directa a las estructuras de datos internas del monitor desde procedimientos declarados fuera de éste. La figura 2-33 ilustra un monitor escrito en un lenguaje imaginario, Pidgin Pascal. Aquí no podemos utilizar C debido a que los monitores son un concepto del *lenguaje*, y C no los tiene.

```
monitor ejemplo
        integer i;
        condition c;

        procedure productor();
        .
        .
        .
        end;


        procedure consumidor();
        .       .       .
        end;
end monitor;
```

Figura 2-33. Un monitor.

Los monitores tienen una importante propiedad que los hace útiles para lograr la exclusión mutua: sólo puede haber un proceso activo en un monitor en cualquier instante. Los monitores son una construcción del lenguaje de programación, por lo que el compilador sabe que son especiales y puede manejar las llamadas a los procedimientos del monitor en forma distinta a las llamadas a otros procedimientos. Por lo general, cuando un proceso llama a un procedimiento de monitor, las primeras instrucciones del procedimiento comprobarán si hay algún otro proceso activo en un momento dado dentro del monitor. De ser así, el proceso invocador se suspenderá hasta que el otro proceso haya dejado el monitor. Si no hay otro proceso utilizando el monitor, el proceso invocador puede entrar.

Es responsabilidad del compilador implementar la exclusión mutua en las entradas del monitor, pero una forma común es utilizar un mutex o semáforo binario. Como el compilador (y no el programador) está haciendo los arreglos para la exclusión mutua, es mucho menos probable que al-

www.FreeLibros.me

go salga mal. En cualquier caso, la persona que escribe el monitor no tiene que saber acerca de cómo el compilador hace los arreglos para la exclusión mutua. Basta con saber que, al convertir todas las regiones críticas en procedimientos de monitor, nunca habrá dos procesos que ejecuten sus regiones críticas al mismo tiempo.

Aunque los monitores proporcionan una manera fácil de lograr la exclusión mutua, como hemos visto antes, eso no basta; también necesitamos una forma en la que los procesos se bloqueen cuando no puedan continuar. En el problema del productor-consumidor, es bastante sencillo colocar todas las pruebas de búfer-lleno y búfer-vacío en procedimientos de monitor, pero ¿cómo debería bloquearse el productor si encuentra que el búfer está lleno?

La solución está en la introducción de las **variables de condición**, junto con dos operaciones de éstas: `wait` y `signal`. Cuando un procedimiento de monitor descubre que no puede continuar (por ejemplo, el productor encuentra el búfer lleno), realiza una operación `wait` en alguna variable de condición (por decir, *llenas*). Esta acción hace que el proceso que hace la llamada se bloquee. También permite que otro proceso que no haya podido entrar al monitor entre ahora. En el contexto de Pthreads antes descrito vimos las variables de condición y estas operaciones.

Por ejemplo, este otro proceso, el consumidor, puede despertar a su socio dormido mediante la realización de una operación `signal` en la variable de condición que su socio esté esperando. Para evitar tener dos procesos activos en el monitor al mismo tiempo, necesitamos una regla que indique lo que ocurre después de una operación `signal`. Hoare propuso dejar que el proceso recién despertado se ejecutara, suspendiendo el otro. Brinch Hansen propuso requerir que un proceso que realice una operación `signal` *deba* salir del monitor de inmediato. En otras palabras, una instrucción `signal` podría aparecer sólo como la instrucción final en un procedimiento de monitor. Utilizaremos la proposición de Brinch Hansen debido a que, en concepto, es más simple y también más fácil de implementar. Si se realiza una operación signal en una variable de condición que varios procesos estén esperando, sólo uno de ellos, que determine el planificador del sistema, se revivirá.

Adicionalmente hay una tercera solución, que no propusieron Hoare ni Brinch Hansen. Esta solución expone que se debe dejar que el señalizador continúe su ejecución y permita al proceso en espera empezar a ejecutarse sólo después que el señalizador haya salido del monitor.

Las variables de condición no son contadores; no acumulan señales para su uso posterior como hacen los semáforos. Por ende, si una variable de condición se señaliza y no hay ningún proceso esperándola, la señal se pierde para siempre. En otras palabras, la operación `wait` debe ir antes de la operación `signal`. Esta regla facilita la implementación en forma considerable. En la práctica no es un problema debido a que es fácil llevar el registro del estado de cada proceso con variables, si es necesario. Un proceso, que de lo contrario realizaría una operación `signal`, puede ver que esta operación no es necesaria con sólo analizar las variables.

En la figura 2-34 se muestra un esqueleto del problema productor-consumidor con monitores en un lenguaje imaginario, Pidgin Pascal. La ventaja de utilizar Pidgin Pascal aquí es que es un lenguaje puro y simple, que sigue el modelo de Hoare/Brinch Hansen con exactitud.

Tal vez el lector esté pensando que las operaciones `wait` y `signal` se parecen a `sleep` y `wakeup`, operaciones que vimos antes y tenían condiciones de carrera fatales. En realidad *son* muy similares, pero con una diferencia crucial: `sleep` y `wakeup` fallaban debido a que, mientras un proceso estaba tratando de dormir, el otro estaba tratando de despertarlo. Con los monitores, esto no puede

www.FreeLibros.me

```
monitor ProductorConsumidor
        condition llenas, vacias;
        integer cuenta;

        procedure insertar(elemento: integer);
        begin
                if cuenta = N then wait(llenas);
                insertar_elemento(elemento);
                cuenta := cuenta + 1;
                if cuenta = 1 then signal(vacias)
        end;

        function eliminar: integer;
        begin
                if cuenta = 0 then wait(vacias);
                eliminar = eliminar_elemento;
                cuenta := cuenta – 1;
                if cuenta = N – 1 then signal(llenas)
        end;

        cuenta := 0;
end monitor;

procedure productor;
begin
        while true do
        begin
                elemento = producir_elemento;
                ProductorConsumidor.insertar(elemento)
        End
end;

procedure consumidor;
begin
        while true do
        begin
                elemento = ProductorConsumidor.eliminar;
                consumir_elemento(elemento)
        end
end;
```

Figura 2-34. Un esquema del problema productor-consumidor con monitores. Sólo hay un procedimiento de monitor activo a la vez. El búfer tiene *N* ranuras.

ocurrir. La exclusión mutua automática en los procedimientos de monitor garantiza que (por ejemplo) si el productor dentro de un procedimiento de monitor descubre que el búfer está lleno, podrá completar la operación wait sin tener que preocuparse por la posibilidad de que el planificador pueda conmutar al consumidor justo antes de que se complete la operación `wait`. Al consumidor ni si-

www.FreeLibros.me

quiera se le va permitir entrar al monitor hasta que `wait` sea terminado y el productor se haya marcado como no ejecutable.

Aunque Pidgin Pascal es un lenguaje imaginario, algunos lenguajes de programación reales también permiten implementar monitores, aunque no siempre en la forma diseñada por Hoare y Brinch Hansen. Java es uno de esos lenguajes. Java es un lenguaje orientado a objetos que soporta hilos a nivel usuario y también permite agrupar métodos (procedimientos) en clases. Al agregar la palabra clave `synchronized` a la declaración de un método, Java garantiza que, una vez un hilo ha empezado a ejecutar ese método, no se permitirá que ningún otro hilo empiece a ejecutar ningún otro método `synchronized` de ese objeto.

En la figura 2-35 se muestra una solución en Java para el problema del productor-consumidor que utiliza monitores. La solución consiste en cuatro clases: la clase exterior (*ProductorConsumidor*) crea e inicia dos hilos, *p* y *c*; las clases segunda y tercera (*productor* y *consumidor*, respectivamente) contienen el código para el productor y el consumidor; por último, la clase *nuestro_monitor* es el monitor. Contiene dos hilos sincronizados que se utilizan para insertar elementos en el búfer compartido y sacarlos. A diferencia de los ejemplos anteriores, en este caso mostramos el código completo de *insertar* y *eliminar*.

Los hilos productor y consumidor son idénticos en funcionalidad a sus contrapartes en todos nuestros ejemplos anteriores. El productor tiene un ciclo infinito que genera datos y los coloca en el búfer común; el consumidor tiene un ciclo infinito equivalente para sacar datos del búfer común y realizar algo divertido con ellos.

La parte interesante de este programa es la clase *nuestro_monitor*, que contiene el búfer, las variables de administración y dos métodos sincronizados. Cuando el productor está activo dentro de *insertar*, sabe de antemano que el consumidor no puede estar activo dentro de *eliminar*, con lo cual es seguro actualizar las variables y el búfer sin temor de que se produzcan condiciones de carrera. La variable *cuenta* lleva el registro de cuántos elementos hay en el búfer. Sólo puede tomar un valor desde 0 hasta (incluyendo a) $N - 1$. La variable *inf* es el índice de la ranura del búfer de donde se va a obtener el siguiente elemento. De manera similar, *sup* es el índice de la ranura del búfer en donde se va a colocar el siguiente elemento. Se permite que $inf = sup$, lo cual significa que hay 0 o N elementos en el búfer. El valor de *cuenta* indica cuál caso se aplica.

Los métodos sincronizados en Java difieren de los monitores clásicos en un punto esencial: Java no tiene variables de condición integradas. En vez de ello, ofrece dos procedimientos (*wait* y *notify*) que son equivalentes a *sleep* y *wakeup*, con la excepción de que al utilizarlos dentro de métodos sincronizados no están sujetos a condiciones de carrera. En teoría, el método *wait* se puede interrumpir, y esto es de lo que trata el código circundante. Java requiere que el manejo de excepciones sea explícito. Para nuestros fines, sólo imagine que *ir_al_estado_inactivo* es la manera de ir a dormir.

Al automatizar la exclusión mutua de las regiones críticas, los monitores hacen que la programación en paralelo sea mucho menos propensa a errores que con los semáforos. De todas formas tienen ciertas desventajas. No es por nada que nuestros dos ejemplos de monitores se escribieron en Pidgin Pascal en vez de usar C, como los demás ejemplos de este libro. Como dijimos antes, los monitores son un concepto de lenguaje de programación. El compilador debe reconocerlos y hacer los arreglos para la exclusión mutua de alguna manera. C, Pascal y la mayoría de los otros lenguajes no tienen monitores, por lo que es irrazonable esperar que sus compiladores implemen-

www.FreeLibros.me

ten reglas de exclusión mutua. De hecho, ¿cómo podría el compilador saber siquiera cuáles procedimientos están en los monitores y cuáles no?

Estos mismos lenguajes tampoco tienen semáforos, pero es fácil agregarlos: todo lo que necesitamos hacer es agregar dos rutinas cortas en lenguaje ensamblador a la biblioteca para emitir las llamadas al sistema `up` y `down`. Los compiladores ni siquiera tienen que saber que existen. Desde luego que los sistemas operativos tienen que saber acerca de los semáforos, pero si tenemos al menos un sistema operativo basado en semáforos, podemos escribir los programas de usuario para este sistema en C o C++ (o incluso en lenguaje ensamblador, si somos demasiado masoquistas). Con los monitores se requiere un lenguaje que los tenga integrados.

Otro problema con los monitores (y también con los semáforos) es que están diseñados para resolver el problema de exclusión mutua en una o más CPUs que tengan acceso a una memoria común. Al colocar los semáforos en la memoria compartida y protegerlos con instrucciones `TSL` o `XCHG`, podemos evitar las condiciones de carrera. Si utilizamos un sistema distribuido que consista en varias CPUs, cada una con su propia memoria privada, conectadas por una red de área local, estas primitivas no se pueden aplicar. La conclusión es que los semáforos son de un nivel demasiado bajo y los monitores no pueden usarse, excepto en algunos lenguajes de programación. Además, ninguna de las primitivas permite el intercambio de información entre las máquinas. Se necesita algo más.

2.3.8 Pasaje (transmisión) de mensajes

Ese "algo más" es el **pasaje de mensajes** (*message passing*). Este método de comunicación entre procesos utiliza dos primitivas (`send` y `receive`) que, al igual que los semáforos y a diferencia de los monitores, son llamadas al sistema en vez de construcciones del lenguaje. Como tales, se pueden colocar con facilidad en procedimientos de biblioteca, como

```
send(destino, &mensaje);
```

y

```
receive(origen, &mensaje);
```

La primera llamada envía un mensaje a un destino especificado y la segunda recibe un mensaje de un origen especificado (o de *CUALQUIERA*, si al receptor no le importa). Si no hay un mensaje disponible, el receptor se puede bloquear hasta que llegue uno. De manera alternativa, puede regresar de inmediato con un código de error.

Aspectos de diseño para los sistemas con pasaje de mensajes

Los sistemas de paso de mensajes tienen muchos problemas y cuestiones de diseño que presentan un reto y no surgen con los semáforos o monitores, en especial si los procesos que se están comunicando se encuentran en distintas máquinas conectadas por una red. Por ejemplo, se pueden

www.FreeLibros.me

```
public class ProductorConsumidor{
  static final int N = 100;           // constante que proporciona el tamaño del búfer
  static productor p = new productor();  // crea instancia de un nuevo hilo productor
  static consumidor c = new consumidor(); // crea instancia de un nuevo hilo consumidor
  static nuestro_monitor mon = new nuestro_monitor();   // crea instancia de un nuevo monitor

  public static void main(String args[]){
    p.start();   // inicia el hilo productor
    c.start();   // inicia el hilo consumidor
  }

  static class productor extends Thread {
    public void run() { // el método run contiene el código del hilo
      int elemento;
      while(true){   // ciclo del productor
        elemento=producir_elemento();
        mon.insertar(elemento);
      }
    }
    private int producir_elemento(){...}   // realmente produce
  }

  static class consumidor extends Thread{
    public void run() {  // el método run contiene el código del hilo
      int elemento;
      while(true){   // ciclo del consumidor
        elemento=mon.eliminar();
        consumir_elemento(elemento);
      }
    }
    private void consumir_elemento(int elemento){...}  // realmente consume
  }

  static class nuestro_monitor{   // este es un monitor
    private int bufer[]=new int[N];
    private int cuenta=0, inf=0, sup=0;   // contadores e índices

    public synchronized void insertar(int val){
      if (cuenta==N) ir_a_estado_inactivo();   // si el búfer está lleno, pasa al estado inactivo
      bufer[sup]=val;   // inserta un elemento en el búfer
      sup=(sup+1)%N;   // ranura en la que se va a colocar el siguiente elemento
      cuenta=cuenta+1;   // ahora hay un elemento más en el búfer
      if (cuenta==1) notify();   // si el consumidor estaba inactivo, lo despierta
    }

    public synchronized int eliminar(){
      int val;
      if (cuenta==0) ir_a_estado_inactivo();  // si el búfer está vacío, pasa al estado inactivo
      val=bufer[inf];  // obtiene un elemento del búfer
      lo = (lo + 1) %N;    // ranura en la que se va a colocar el siguiente elemento
      cuenta=cuenta-1;  // un elemento menos en el búfer
      if (cuenta==N−1) notify();  // si el productor estaba inactivo, lo despierta
      return val;
    }
    private void ir_a_estado_inactivo() {try{wait();}catch(InterruptedException exc){};}
  }
}
```

Figura 2-35. Una solución al problema del productor-consumidor en Java.

www.FreeLibros.me

perder mensajes en la red. Para protegerse contra los mensajes perdidos, el emisor y el receptor pueden acordar que, tan pronto como haya recibido un mensaje, el receptor enviará de vuelta un mensaje especial de acuse de recibo (**acknowledgement**). Si el emisor no ha recibido el acuse dentro de cierto intervalo de tiempo, vuelve a transmitir el mensaje.

Ahora considere lo que ocurre si el mensaje se recibió en forma correcta, pero se pierde el mensaje de acuse que se envía de vuelta al emisor. Éste volverá a transmitir el mensaje, por lo que el receptor lo recibirá dos veces. Es esencial que el receptor pueda diferenciar un mensaje nuevo de la retransmisión de uno anterior. Por lo general, este problema se resuelve colocando números de secuencia consecutivos en cada mensaje original. Si el receptor recibe un mensaje que contenga el mismo número de secuencia que el mensaje anterior, sabe que el mensaje es un duplicado que se puede ignorar. La comunicación exitosa a la cara del pasaje de mensajes sin confiabilidad constituye una gran parte del estudio de las redes de computadoras. Para obtener más información, consulte (Tanenbaum, 1996).

Los sistemas de mensajes también tienen que lidiar con la forma en que se denominan los procesos, de manera que el proceso especificado en una llamada a `send` o `receive` esté libre de ambigüedad. La **autenticación** también es una cuestión en los sistemas de mensajes: ¿cómo puede saber el cliente que se está comunicando con el verdadero servidor de archivos y no con un impostor?

Al otro extremo del espectro, también hay cuestiones de diseño importantes cuando el emisor y el receptor están en la misma máquina. Una de éstas es el rendimiento. La acción de copiar mensajes de un proceso a otro es siempre más lenta que realizar una operación con un semáforo o entrar a un monitor. Se ha realizado un gran esfuerzo para lograr que el pasaje de mensajes sea eficiente. Por ejemplo, Cheriton (1984) sugirió limitar el tamaño de los mensajes según el tamaño de los registros de la máquina y después realizar el pasaje de mensajes mediante el uso de los registros.

El problema del productor-consumidor con pasaje de mensajes

Ahora veamos cómo se puede resolver el problema del productor-consumidor con el pasaje de mensajes y sin memoria compartida. En la figura 2-36 se muestra una solución. Suponemos que todos los mensajes tienen el mismo tamaño y el sistema operativo coloca los mensajes enviados, pero no recibidos, de manera automática en el búfer. En esta solución se utiliza un total de *N* mensajes, de manera similar a las *N* ranuras en un búfer de memoria compartida. El consumidor empieza por enviar *N* mensajes vacíos al productor. Cada vez que el productor tiene un elemento para dar al consumidor, recibe un mensaje vacío y envía de regreso uno lleno. De esta manera, el número total de mensajes en el sistema permanece constante en el tiempo, para que se puedan almacenar en una cantidad de memoria específica que se conoce de antemano.

Si el productor trabaja con más rapidez que el consumidor, todos los mensajes terminarán llenos, esperando al consumidor; el productor se bloqueará, esperando a que regrese un mensaje vacío. Si el consumidor trabaja con más rapidez, entonces ocurrirá lo inverso: todos los mensajes estarán vacíos, esperando a que el productor los llene; el consumidor se bloqueará en espera de un mensaje lleno.

Hay muchas variantes posibles con el paso de mensajes. Para empezar, veamos la forma en que se direccionan los mensajes. Una manera es asignar a cada proceso una dirección única y direccionar los mensajes a los procesos. Una manera distinta es inventar una estructura de datos, conocida

www.FreeLibros.me

```
#define N 100                                   /* número de ranuras en el búfer */

void productor(void)
{
     int elemento;
     mensaje m;                                 /* búfer de mensajes */

     while (TRUE) {
          elemento=producir_elemento();         /* genera algo para colocar en el búfer */
          receive(consumidor,&m);               /* espera a que llegue un mensaje vacío */
          crear_mensaje(&m,elemento);           /* construye un mensaje para enviarlo */
          send(consumidor,&m);                  /* envía elemento al consumidor */
     }
}

void consumidor(void)
{
     int elemento, i;
     mensaje m;

     for (i=0; i<N; i++) send(productor,&m);     /* envía N mensajes vacíos */
     while (TRUE){
          receive(productor,&m);                /* obtiene el mensaje que contiene el elemento */
          elemento=extraer_elemento(&m);        /* extrae el elemento del mensaje */
          send(productor,&m);                   /* envía de vuelta respuesta con mensaje vacío */
          consumir_elemento(elemento);          /* hace algo con el elemento */
   }
}
```

Figura 2-36. El problema del productor-consumidor con *N* mensajes.

como **buzón**. Un buzón es un lugar para colocar en el búfer cierto número de mensajes, que por lo general se especifica a la hora de crear el buzón. Cuando se utilizan buzones, los parámetros de dirección en las llamadas a `send` y `receive` son buzones, no procesos. Cuando un proceso trata de enviar un buzón que está lleno, se suspende hasta que se remueva un mensaje de ese buzón, haciendo espacio para un nuevo mensaje.

Para el problema del productor-consumidor, tanto el productor como el consumidor crearían buzones con el tamaño suficiente como para contener *N* mensajes. El productor enviaría mensajes que contuvieran los datos actuales al buzón del consumidor y el consumidor enviaría mensajes vacíos al buzón del productor. Cuando se utilizan buzones, el mecanismo de búfer está claro: el buzón de destino contiene los mensajes que se han enviado al proceso de destino, pero que todavía no se han aceptado.

El otro extremo de tener buzones es para eliminar todo el uso de búfer. Cuando se sigue este método, si la operación `send` termina antes de la operación `receive`, el proceso emisor se bloquea hasta que ocurre la operación `receive`, momento en el cual el mensaje se puede copiar de manera directa del emisor al receptor, sin búfer de por medio. De manera similar, si se realiza primero la opera-

www.FreeLibros.me

ción `receive`, el receptor se bloquea hasta que ocurre una operación `send`. A menudo esta estrategia se conoce como **encuentro**. Es más fácil implementar que un esquema de mensajes con búfer, pero menos flexible debido a que el emisor y el receptor se ven obligados a ejecutarse en paso de bloqueo.

El pasaje de mensajes se utiliza con frecuencia en los sistemas de programación en paralelo. Por ejemplo, un sistema de paso de mensajes reconocido es **MPI** (*Message-Passing Interface;* Interfaz de pasaje de mensajes). Se utiliza mucho en la computación científica. Para obtener más información sobre este sistema, consulte (Gropp y colaboradores, 1994; Snir y colaboradores, 1996).

2.3.9 Barreras

Nuestro último mecanismo de sincronización está destinado a los grupos de procesos, en vez de las situaciones de tipo productor-consumidor de dos procesos. Algunas aplicaciones se dividen en fases y tienen la regla de que ningún proceso puede continuar a la siguiente fase sino hasta que todos los procesos estén listos para hacerlo. Para lograr este comportamiento, se coloca una **barrera** al final de cada fase. Cuando un proceso llega a la barrera, se bloquea hasta que todos los procesos han llegado a ella. La operación de una barrera se ilustra en la figura 2-37.

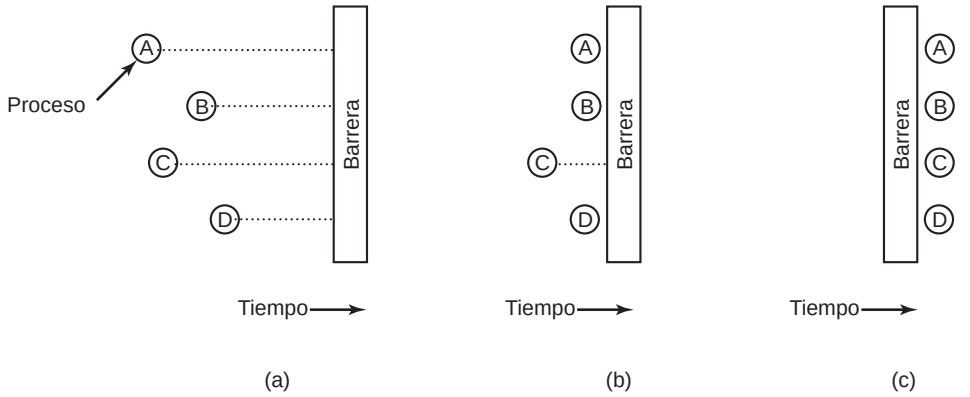


Figura 2-37. Uso de una barrera. (a) Procesos acercándose a una barrera. (b) Todos los procesos, excepto uno, bloqueados en la barrera. (c) Cuando el último proceso llega a la barrera, todos se dejan pasar.

En la figura 2-37(a) podemos ver cuatro procesos que se acercan a una barrera. Lo que esto significa es que sólo están calculando y no han llegado al final de la fase actual todavía. Después de cierto tiempo, el primer proceso termina todos los cálculos requeridos durante la primera fase. Después ejecuta la primitiva `barrier`, por lo general llamando a un procedimiento de biblioteca. Después el proceso se suspende. Un poco después, un segundo y luego un tercer proceso, terminan la primera fase, ejecutando también la primitiva `barrier`. Esta situación se ilustra en la figura 2-37(b). Por último, cuando el último proceso (*C*) llega a la barrera se liberan todos los procesos, como se muestra en la figura 2-37(c).

www.FreeLibros.me

Como ejemplo de un problema que requiere barreras, considere un problema común de relajación en física o ingeniería. Por lo general hay una matriz que contiene ciertos valores iniciales. Los valores podrían representar temperaturas en diversos puntos de una hoja de metal. La idea podría ser calcular cuánto tarda en propagarse el efecto de una flama colocada en una esquina a lo largo de la hoja.

Empezando con los valores actuales, se aplica una transformación a la matriz para obtener la segunda versión de ésta; por ejemplo, mediante la aplicación de las leyes de termodinámica para ver cuál es el valor de todas las temperaturas en un tiempo posterior, representado por ΔT. Después el proceso se repite una y otra vez, proporcionando las temperaturas en los puntos de muestreo como una función del tiempo, a medida que se calienta la hoja. El algoritmo produce una serie de matrices a través del tiempo.

Ahora imagine que la matriz es muy grande (por ejemplo, de 1 millón por 1 millón), por lo cual se requieren procesos en paralelo (posiblemente en un multiprocesador) para agilizar los cálculos. Distintos procesos trabajarán en distintas partes de la matriz, calculando los nuevos elementos de ésta a partir de los anteriores, de acuerdo con las leyes de la física. Sin embargo, ningún proceso puede empezar en la iteración $n + 1$ sino hasta que se complete la iteración n, es decir, hasta que todos los proceso hayan terminado su trabajo actual. La forma de lograr este objetivo es programar cada proceso para que ejecute una operación `barrier` después de que haya terminado su parte de la iteración actual. Cuando todos ellos hayan terminado, la nueva matriz (la entrada para la siguiente iteración) estará completa y todos los procesos se liberarán en forma simultánea para empezar la siguiente iteración.

2.4 PLANIFICACIÓN

Cuando una computadora se multiprograma, con frecuencia tiene varios procesos o hilos que compiten por la CPU al mismo tiempo. Esta situación ocurre cada vez que dos o más de estos procesos se encuentran al mismo tiempo en el estado listo. Si sólo hay una CPU disponible, hay que decidir cuál proceso se va a ejecutar a continuación. La parte del sistema operativo que realiza esa decisión se conoce como **planificador de procesos** y el algoritmo que utiliza se conoce como **algoritmo de planificación**. Estos terminos conforman el tema a tratar en las siguientes secciones.

Muchas de las mismas cuestiones que se aplican a la planificación de procesos también se aplican a la planificación de hilos, aunque algunas son distintas. Cuando el kernel administra hilos, por lo general la planificación se lleva a cabo por hilo, e importa muy poco (o nada) a cuál proceso pertenece ese hilo. Al principio nos enfocaremos en las cuestiones de planificación que se aplican a los procesos e hilos. Más adelante analizaremos de manera explícita la planificación de hilos y algunas de las cuestiones únicas que surgen. En el capítulo 8 trabajaremos con los chips multinúcleo.

2.4.1 Introducción a la planificación

En los días de los sistemas de procesamiento por lotes, con entrada en forma de imágenes de tarjetas en una cinta magnética, el algoritmo de planificación era simple: bastaba con ejecutar el siguiente trabajo en la cinta. Con los sistemas de multiprogramación, el algoritmo de planificación se

www.FreeLibros.me

volvió más complejo debido a que comúnmente había varios usuarios esperando ser atendidos. Algunas mainframe todavía combinan los servicios de procesamiento por lotes y de tiempo compartido, requiriendo que el planificador decida si le toca el turno a un trabajo de procesamiento por lotes o a un usuario interactivo en una terminal. (Como punto y aparte, un trabajo de procesamiento por lotes puede ser una petición para ejecutar varios programas en sucesión, pero para esta sección sólo supondremos que es una petición para ejecutar un solo programa). Debido a que el tiempo de la CPU es un recurso escaso en estas máquinas, un buen planificador puede hacer una gran diferencia en el rendimiento percibido y la satisfacción del usuario. En consecuencia, se ha puesto gran esfuerzo en idear algoritmos de planificación astutos y eficientes.

Con la llegada de las computadoras personales, la situación cambió de dos maneras. En primer lugar, la mayor parte del tiempo sólo hay un proceso activo. Es muy poco probable que un usuario que entra a un documento en un procesador de palabras compile al mismo tiempo un programa en segundo plano. Cuando el usuario escribe un comando para el procesador de palabras, el planificador no tiene que esforzarse mucho en averiguar qué proceso ejecutar; el procesador de palabras es el único candidato.

En segundo lugar, las computadoras se han vuelto tan veloces con el paso de los años que la CPU es raras veces un recurso escaso. La mayoría de los programas para computadoras personales están limitados en cuanto a la velocidad a la que el usuario puede presentar los datos de entrada (al escribir o hacer clic), no por la velocidad con que la CPU puede procesarlos. Incluso las compilaciones, que en el pasado consumían muchos ciclos de la CPU, requieren unos cuantos segundos en la mayoría de los casos hoy en día. Aún y cuando se estén ejecutando dos programas al mismo tiempo, como un procesador de palabras y una hoja de cálculo, raras veces es importante saber quién se ejecuta primero, ya que el usuario probablemente está esperando a que ambos terminen. Como consecuencia, la planificación no es de mucha importancia en las PCs simples. Desde luego que hay aplicaciones que prácticamente se comen viva a la CPU; por ejemplo, para visualizar una hora de video en alta resolución, a la vez que se ajustan los colores en cada uno de los 108,000 cuadros (en NTSC) o 90,000 cuadros (en PAL) se requiere de un poder de cómputo a nivel industrial. Sin embargo, las aplicaciones similares son la excepción, en vez de la regla.

Al tratarse de servidores en red, la situación cambia en forma considerable. Aquí varios procesos compiten a menudo por la CPU, por lo que la planificación retoma importancia. Por ejemplo, cuando la CPU tiene que elegir entre ejecutar un proceso que recopila las estadísticas diarias y uno que atiende las peticiones de los usuarios, habrá usuarios más contentos si el segundo tipo de procesos tiene prioridad sobre la CPU.

Además de elegir el proceso correcto que se va a ejecutar a continuación, el planificador también tiene que preocuparse por hacer un uso eficiente de la CPU, debido a que la conmutación de procesos es cara. Para empezar, debe haber un cambio del modo de usuario al modo kernel. Después se debe guardar el estado del proceso actual, incluyendo el almacenamiento de sus registros en la tabla de procesos para que puedan volver a cargarse más adelante. En muchos sistemas, el mapa de memoria (por ejemplo, los bits de referencia de memoria en la tabla de páginas) se debe guardar también. Luego hay que seleccionar un nuevo proceso mediante la ejecución del algoritmo de planificación. Después de eso, se debe volver a cargar en la MMU el mapa de memoria del nuevo proceso. Por último, se debe iniciar el nuevo proceso. Además de todo esto, generalmente la conmutación de procesos hace inválida toda la memoria caché, por lo que tiene que volver a cargarse en forma diná-

www.FreeLibros.me

mica desde la memoria principal dos veces (al momento de entrar al kernel y al salir de éste). Con todo, si se realizan muchas conmutaciones de procesos por segundo, se puede llegar a consumir una cantidad considerable de tiempo de la CPU, por lo cual se aconseja tener precaución.

Comportamiento de un proceso

Casi todos los procesos alternan ráfagas de cálculos con peticiones de E/S (de disco), como se muestra en la figura 2.38. Por lo general la CPU opera durante cierto tiempo sin detenerse, después se realiza una llamada al sistema para leer datos de un archivo o escribirlos en el mismo. Cuando se completa la llamada al sistema, la CPU realiza cálculos de nuevo hasta que necesita más datos o tiene que escribir más datos y así sucesivamente. Hay que tener en cuenta que algunas actividades de E/S cuentan como cálculos. Por ejemplo, cuando la CPU copia bits a una RAM de video para actualizar la pantalla, está calculando y no realizando operaciones de E/S, ya que la CPU está en uso. En este sentido, la E/S es cuando un proceso entra al estado bloqueado en espera de que un dispositivo externo complete su trabajo.

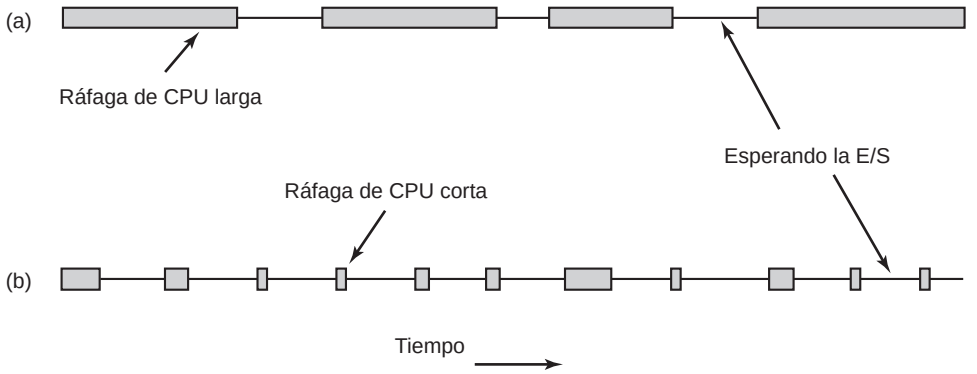


Figura 2-38. Las ráfagas de uso de la CPU se alternan con los periodos de espera por la E/S. (a) Un proceso ligado a la CPU. (b) Un proceso ligado a la E/S.

Lo importante a observar acerca de la figura 2-38 es que algunos procesos, como el que se muestra en la figura 2-38(a), invierten la mayor parte de su tiempo realizando cálculos, mientras que otros, como el que se muestra en la figura 2-38(b), invierten la mayor parte de su tiempo esperando la E/S. A los primeros se les conoce como **limitados a cálculos**; a los segundos como **limitados a E/S (I/O-bound)**. Por lo general, los procesos limitados a cálculos tienen ráfagas de CPU largas y en consecuencia, esperas infrecuentes por la E/S, mientras que los procesos limitados a E/S tienen ráfagas de CPU cortas y por ende, esperas frecuentes por la E/S. Observe que el factor clave es la longitud de la ráfaga de CPU, no de la ráfaga de E/S. Los procesos limitados a E/S están limitados a la E/S debido a que no realizan muchos cálculos entre una petición de E/S y otra, no debido a que tengan peticiones de E/S en especial largas. Se requiere el mismo tiempo para emitir la petición de hardware para leer un bloque de disco, sin importar qué tanto (o qué tan poco) tiempo se requiera para procesar los datos, una vez que lleguen.

www.FreeLibros.me

Vale la pena observar que, a medida que las CPUs se vuelven más rápidas, los procesos tienden a ser más limitados a E/S. Este efecto ocurre debido a que las CPUs están mejorando con mucha mayor rapidez que los discos. Como consecuencia, es probable que la planificación de procesos limitados a E/S se convierta en un tema más importante en un futuro. La idea básica aquí es que, si un proceso limitado a E/S desea ejecutarse, debe obtener rápidamente la oportunidad de hacerlo para que pueda emitir su petición de disco y mantener el disco ocupado. Como vimos en la figura 2-6, cuando los procesos están limitados a E/S, se requieren muchos de ellos para que la CPU pueda estar completamente ocupada.

Cuándo planificar procesos

Una cuestión clave relacionada con la planificación es saber cuándo tomar decisiones de planificación. Resulta ser que hay una variedad de situaciones en las que se necesita la planificación. En primer lugar, cuando se crea un nuevo proceso se debe tomar una decisión en cuanto a si se debe ejecutar el proceso padre o el proceso hijo. Como ambos procesos se encuentran en el estado listo, es una decisión normal de programación y puede ejecutar cualquiera; es decir, el programador de procesos puede elegir ejecutar de manera legítima, ya sea el padre o el hijo.

En segundo lugar, se debe tomar una decisión de planificación cuando un proceso termina. Ese proceso ya no se puede ejecutar (debido a que ya no existe), por lo que se debe elegir algún otro proceso del conjunto de procesos listos. Si no hay un proceso listo, por lo general se ejecuta un proceso inactivo suministrado por el sistema.

En tercer lugar, cuando un proceso se bloquea por esperar una operación de E/S, un semáforo o por alguna otra razón, hay que elegir otro proceso para ejecutarlo. Algunas veces la razón del bloqueo puede jugar un papel en la elección. Por ejemplo, si *A* es un proceso importante y está esperando a que *B* salga de su región crítica, si dejamos que *B* se ejecute a continuación podrá salir de su región crítica y por ende, dejar que *A* continúe. Sin embargo, el problema es que el planificador comúnmente no tiene la información necesaria para tomar en cuenta esta dependencia.

En cuarto lugar, cuando ocurre una interrupción de E/S tal vez haya que tomar una decisión de planificación. Si la interrupción proviene de un dispositivo de E/S que ha terminado su trabajo, tal vez ahora un proceso que haya estado bloqueado en espera de esa operación de E/S esté listo para ejecutarse. Es responsabilidad del planificador decidir si debe ejecutar el proceso que acaba de entrar al estado listo, el proceso que se estaba ejecutando al momento de la interrupción, o algún otro.

Si un reloj de hardware proporciona interrupciones periódicas a 50 o 60 Hz o cualquier otra frecuencia, se puede tomar una decisión de planificación en cada interrupción de reloj o en cada *k*-ésima interrupción de reloj. Los algoritmos de planificación se pueden dividir en dos categorías con respecto a la forma en que manejan las interrupciones del reloj. Un algoritmo de programación **no apropiativo** (*nonpreemptive*) selecciona un proceso para ejecutarlo y después sólo deja que se ejecute hasta que el mismo se bloquea (ya sea en espera de una operación de E/S o de algún otro proceso) o hasta que libera la CPU en forma voluntaria. Incluso aunque se ejecute durante horas, no se suspenderá de manera forzosa. En efecto, no se toman decisiones de planificación durante las interrupciones de reloj. Una vez que se haya completado el procesamiento de la interrupción de reloj, se reanuda la ejecución del proceso que estaba antes de la interrupción, a menos que un proceso de mayor prioridad esté esperando por un tiempo libre que se acabe de cumplir.

www.FreeLibros.me

Por el contrario, un algoritmo de planificación **apropiativa** selecciona un proceso y deja que se ejecute por un máximo de tiempo fijo. Si sigue en ejecución al final del intervalo de tiempo, se suspende y el planificador selecciona otro proceso para ejecutarlo (si hay uno disponible). Para llevar a cabo la planificación apropiativa, es necesario que ocurra una interrupción de reloj al final del intervalo de tiempo para que la CPU regrese el control al planificador. Si no hay un reloj disponible, la planificación no apropiativa es la única opción.

Categorías de los algoritmos de planificación

No es sorprendente que distintos entornos requieran algoritmos de planificación diferentes. Esta situación se presenta debido a que las diferentes áreas de aplicación (y los distintos tipos de sistemas operativos) tienen diferentes objetivos. En otras palabras, lo que el planificador debe optimizar no es lo mismo en todos los sistemas. Tres de los entornos que vale la pena mencionar son:

1. Procesamiento por lotes.
2. Interactivo.
3. De tiempo real.

Los sistemas de procesamiento por lotes siguen utilizándose ampliamente en el mundo de los negocios para realizar nóminas, inventarios, cuentas por cobrar, cuentas por pagar, cálculos de interés (en los bancos), procesamiento de reclamaciones (en las compañías de seguros) y otras tareas periódicas. En los sistemas de procesamiento por lotes no hay usuarios que esperen impacientemente en sus terminales para obtener una respuesta rápida a una petición corta. En consecuencia, son aceptables los algoritmos no apropiativos (o apropiativos con largos periodos para cada proceso). Este método reduce la conmutación de procesos y por ende, mejora el rendimiento. En realidad, los algoritmos de procesamiento por lotes son bastante generales y a menudo se pueden aplicar a otras situaciones también, lo cual hace que valga la pena estudiarlos, incluso para las personas que no están involucradas en la computación con mainframes corporativas.

En un entorno con usuarios interactivos, la apropiación es esencial para evitar que un proceso acapare la CPU y niegue el servicio a los demás. Aun si no hubiera un proceso que se ejecutara indefinidamente de manera intencional, podría haber un proceso que deshabilitara a los demás de manera indefinida, debido a un error en el programa. La apropiación es necesaria para evitar este comportamiento. Los servidores también entran en esta categoría, ya que por lo general dan servicio a varios usuarios (remotos), todos los cuales siempre tienen mucha prisa.

En los sistemas con restricciones de tiempo real, aunque parezca extraño, la apropiación a veces es no necesaria debido a que los procesos saben que no se pueden ejecutar durante periodos extensos, que por lo general realizan su trabajo y se bloquean con rapidez. La diferencia con los sistemas interactivos es que los sistemas de tiempo real sólo ejecutan programas destinados para ampliar la aplicación en cuestión. Los sistemas interactivos son de propósito general y pueden ejecutar programas arbitrarios que no sean cooperativos, o incluso malintencionados.

www.FreeLibros.me

Metas de los algoritmos de planificación

Para poder diseñar un algoritmo de programación, es necesario tener cierta idea de lo que debe hacer un buen algoritmo. Algunos objetivos dependen del entorno (procesamiento por lotes, interactivo o de tiempo real), pero hay también algunos otros que son deseables en todos los casos. En la figura 2-39 se listan algunas metas. A continuación los analizaremos, uno a la vez.

Todos los sistemas
- Equidad - Otorgar a cada proceso una parte justa de la CPU
- Aplicación de políticas - Verificar que se lleven a cabo las políticas establecidas
- Balance - Mantener ocupadas todas las partes del sistema

Sistemas de procesamiento por lotes
- Rendimiento - Maximizar el número de trabajos por hora
- Tiempo de retorno - Minimizar el tiempo entre la entrega y la terminación
- Utilización de la CPU - Mantener ocupada la CPU todo el tiempo

Sistemas interactivos
- Tiempo de respuesta - Responder a las peticiones con rapidez
- Proporcionalidad - Cumplir las expectativas de los usuarios

Sistemas de tiempo real
- Cumplir con los plazos - Evitar perder datos
- Predictibilidad - Evitar la degradación de la calidad en los sistemas multimedia

Figura 2-39. Algunas metas del algoritmo de planificación bajo distintas circunstancias.

Bajo todas las circunstancias, la equidad es importante. Los procesos comparables deben obtener un servicio comparable. No es justo otorgar a un proceso mucho más tiempo de la CPU que a uno equivalente. Desde luego que las distintas categorías de procesos se pueden tratar de manera diferente. Considere el control de la seguridad y la administración de nómina en un centro computacional de un reactor nuclear.

La ejecución de las políticas del sistema está relacionada con la equidad. Si la política local es que los procesos de control de seguridad se ejecuten cada vez que lo deseen, aun si esto significa que la nómina se tardará 30 segundos más, el planificador tiene que asegurar que se ejecute esta política.

Otra meta general es mantener ocupadas todas las partes del sistema siempre que sea posible. Si la CPU y todos los dispositivos de E/S se pueden mantener en ejecución todo el tiempo, se realizará una mayor cantidad de trabajo por segundo que cuando algunos de los componentes están inactivos. Por ejemplo, en un sistema de procesamiento por lotes, el planificador controla cuáles trabajos se llevan a memoria para ejecutarlos. Es mejor tener algunos procesos limitados a CPU y algunos procesos limitados a E/S juntos en la memoria, que primero cargar y ejecutar todos los trabajos limitados a CPU y después, cuando terminen, cargar y ejecutar todos los trabajos limitados a E/S. Si se utiliza esta última estrategia, cuando los procesos ligados a CPU se ejecuten, lucharán por la CPU y el disco estará inactivo. Más adelante, cuando entren los trabajos ligados a E/S, lu-

www.FreeLibros.me

charán por el disco y la CPU estará inactiva. Es mejor mantener todo el sistema en ejecución al mismo tiempo mediante una cuidadosa mezcla de procesos.

Los administradores de los centros grandes de cómputo que ejecutan muchos trabajos de procesamiento por lotes comúnmente se basan en tres métricas para verificar el desempeño de sus sistemas: rendimiento, tiempo de retorno y utilización de la CPU. El **rendimiento** es el número de trabajos por hora que completa el sistema. Considerando todos los detalles, es mejor terminar 50 trabajos por hora que terminar 40. El **tiempo de retorno** es el tiempo estadísticamente promedio desde el momento en que se envía un trabajo por lotes, hasta el momento en que se completa. Este parámetro mide cuánto tiempo tiene que esperar el usuario promedio la salida. He aquí la regla: lo pequeño es bello.

Un algoritmo de planificación que maximiza el rendimiento no necesariamente minimiza el tiempo de retorno. Por ejemplo, dada una mezcla de trabajos cortos y largos, un planificador que siempre ha ejecutado trabajos cortos y nunca ejecutó trabajos largos podría lograr un rendimiento excelente (muchos trabajos cortos por hora), pero a expensas de un terrible tiempo de retorno para los trabajos largos. Si los trabajos cortos llegaran a un tiempo bastante estable, los trabajos largos tal vez nunca se ejecutarían, con lo cual el tiempo de respuesta promedio se volvería infinito, a la vez que se lograría un rendimiento alto.

La utilización de la CPU se emplea a menudo como métrica en los sistemas de procesamiento por lotes. En realidad, no es una buena métrica. Lo que importa es cuántos trabajos por hora salen del sistema (rendimiento) y cuánto tiempo se requiere para obtener un trabajo de vuelta (tiempo de retorno). Con la utilización de la CPU como métrica es como clasificar a los autos con base en cuántas veces por hora gira el motor. Por otro lado, es útil poder detectar cuando el uso de la CPU está llegando a 100%, para saber cuándo es momento de obtener más poder de cómputo.

Para los sistemas interactivos se aplican distintas metas. La más importante es minimizar el **tiempo de respuesta**; es decir, el tiempo que transcurre entre emitir un comando y obtener el resultado. En una computadora personal en la que se ejecuta un proceso en segundo plano (por ejemplo, leer y almacenar correo electrónico de la red), una petición del usuario para iniciar un programa o abrir un archivo debe tener precedencia sobre el trabajo en segundo plano. Hacer que todas las peticiones interactivas se atiendan primero se percibirá como un buen servicio.

Una cuestión algo relacionada es lo que podría denominarse **proporcionalidad**. Los usuarios tienen una idea inherente (pero a menudo incorrecta) acerca de cuánto deben tardar las cosas. Cuando una petición que se percibe como compleja tarda mucho tiempo, los usuarios aceptan eso, pero cuando una petición que se percibe como simple tarda mucho tiempo, los usuarios se irritan. Por ejemplo, si al hacer clic en un icono que empieza a enviar un fax el sistema tarda 60 segundos en completar la oración, el usuario probablemente aceptará eso como un gaje del oficio, debido a que no espera que un fax se envíe en 5 segundos.

Por otro lado, cuando un usuario hace clic en el icono que interrumpe la conexión telefónica una vez que se ha enviado el fax, tiene diferentes expectativas. Si no se ha completado después de 30 segundos, es probable que el usuario esté sumamente enojado y después de 60 segundos estará echando espuma por la boca. Este comportamiento se debe a la percepción común de los usuarios de que se *supone* que hacer una llamada telefónica y enviar un fax requiere mucho más tiempo que sólo colgar el teléfono. En algunos casos (como éste) el planificador no puede hacer nada acerca del

www.FreeLibros.me

tiempo de respuesta, pero en otros casos sí puede, en especial cuando el retraso se debe a una mala elección en el orden de los procesos.

Los sistemas de tiempo real tienen propiedades distintas de los sistemas interactivos y por ende diferentes metas de planificación. Se caracterizan por tener tiempos límite que deben (o deberían) cumplirse. Por ejemplo, si una computadora está controlando un dispositivo que produce datos a un paso regular, al no ejecutar el proceso de recolección de datos a tiempo se podría perder información. Por ende, la principal necesidad en un sistema de tiempo real es cumplir todos los tiempos límite (o la mayoría).

En algunos sistemas de tiempo real, en especial los que involucran el uso de multimedia, la predictibilidad es importante. No es fatal fallar en un tiempo límite, pero si el procesador de audio se ejecuta con muchos errores, la calidad del sonido se deteriorará con rapidez. El video también cuenta, pero el oído es mucho más sensible a la perturbación que el ojo. Para evitar este problema, la planificación de procesos debe ser altamente predecible y regular. En este capítulo estudiaremos los algoritmos de planificación de procesamiento por lotes e interactivos, pero aplazaremos la mayor parte de nuestro estudio de la planificación de tiempo real para cuando analicemos los sistemas operativos multimedia en el capítulo 7.

2.4.2 Planificación en sistemas de procesamiento por lotes

Ahora es tiempo de pasar de las cuestiones de planificación en general a los algoritmos de planificación específicos. En esta sección analizaremos los algoritmos que se utilizan en sistemas de procesamiento por lotes. En las siguientes secciones examinaremos los sistemas interactivos y de tiempo real. Vale la pena aclarar que algunos algoritmos se utilizan tanto en los sistemas de procesamiento por lotes como interactivos. Más adelante estudiaremos estos algoritmos.

Primero en entrar, primero en ser atendido

Probablemente el más simple de todos los algoritmos de planificación es el de tipo **primero en entrar, primero en ser atendido** (FCFS, *First-Come, First-Served*) no apropiativo. Con este algoritmo, la CPU se asigna a los procesos en el orden en el que la solicitan. En esencia hay una sola cola de procesos listos. Cuando el primer trabajo entra al sistema desde el exterior en la mañana, se inicia de inmediato y se le permite ejecutarse todo el tiempo que desee. No se interrumpe debido a que se ha ejecutado demasiado tiempo. A medida que van entrando otros trabajos, se colocan al final de la cola. Si el proceso en ejecución se bloquea, el primer proceso en la cola se ejecuta a continuación. Cuando un proceso bloqueado pasa al estado listo, al igual que un trabajo recién llegado, se coloca al final de la cola.

La gran fuerza de este algoritmo es que es fácil de comprender e igualmente sencillo de programar. También es equitativo, en el mismo sentido en que es equitativo asignar los escasos boletos para eventos deportivos o conciertos a las personas que están dispuestas a permanecer en la línea desde las 2 A.M. Con este algoritmo, una sola lista ligada lleva la cuenta de todos los procesos listos. Para elegir un proceso a ejecutar sólo se requiere eliminar uno de la parte frontal de la cola. Para agregar un nuevo trabajo o desbloquear un proceso sólo hay que adjuntarlo a la parte final de la cola. ¿Qué podría ser más simple de comprender y de implementar?

www.FreeLibros.me

Por desgracia, el algoritmo tipo "primero en entrar, primero en ser atendido" también tiene una importante desventaja. Suponga que hay un proceso ligado a los cálculos que se ejecuta durante 1 segundo en cierto momento, junto con muchos procesos limitados a E/S que utilizan poco tiempo de la CPU, pero cada uno de ellos tiene que realizar 1000 lecturas de disco para completarse. El proceso limitado a los cálculos se ejecuta durante 1 segundo y después lee un bloque de disco. Ahora se ejecutan todos los procesos de E/S e inician lecturas de disco. Cuando el proceso limitado a los cálculos obtiene su bloque de disco, se ejecuta por otro segundo, seguido de todos los procesos limitados a E/S en una rápida sucesión.

El resultado neto es que cada proceso limitado a E/S llega a leer 1 bloque por segundo y requerirá 1000 segundos para completarse. Con un algoritmo de planificación que se apropió del proceso limitado a los cálculos cada 10 mseg, los procesos limitados a E/S terminarían en 10 segundos en vez de 1000 segundos y sin quitar mucha velocidad al proceso limitado a los cálculos.

El trabajo más corto primero

Ahora analicemos otro algoritmo de procesamiento por lotes no apropiativo, que supone que los tiempos de ejecución se conocen de antemano. Por ejemplo, en una compañía de seguros las personas pueden predecir con bastante precisión cuánto tiempo se requerirá para ejecutar un lote de 1000 reclamaciones, ya que se realiza un trabajo similar cada día. Cuando hay varios trabajos de igual importancia esperando a ser iniciados en la cola de entrada, el planificador selecciona **el trabajo más corto primero** (SJF, *Shortest Job First***)**. Analice la figura 2-40. Aquí encontramos cuatro trabajos (*A*, *B*, *C* y *D*) con tiempos de ejecución de 8, 4, 4 y 4 minutos, respectivamente. Al ejecutarlos en ese orden, el tiempo de respuesta para *A* es de 8 minutos, para *B* es de 12 minutos, para *C* es de 16 minutos y para *D* es de 20 minutos, para un promedio de 14 minutos.



Figura 2-40. Un ejemplo de planificación tipo el trabajo más corto primero. (a) Ejecución de cuatro trabajos en el orden original. (b) Ejecución de cuatro trabajos en el orden del tipo "el trabajo más corto primero".

Ahora consideremos ejecutar estos cuatro trabajos usando el trabajo más corto primero, como se muestra en la figura 2-40(b). Los tiempos de respuesta son ahora de 4, 8, 12 y 20 minutos, para un promedio de 11 minutos. Probablemente sea óptimo el algoritmo tipo el trabajo más corto primero. Considere el caso de cuatro trabajos, con tiempos de ejecución de *a*, *b*, *c* y *d*, respectivamente. El primer trabajo termina en el tiempo *a*, el segundo termina en el tiempo $a + b$, y así en lo sucesivo. El tiempo promedio de respuesta es $(4a + 3b + 2c + d)/4$. Está claro que *a* contribuye más al promedio que los otros tiempos, por lo que debe ser el trabajo más corto, con *b* a continuación, después *c* y por último *d* como el más largo, ya que sólo afecta a su tiempo de retorno. El mismo argumento se aplica con igual facilidad a cualquier número de trabajos.

www.FreeLibros.me

Vale la pena observar que el trabajo más corto primero es sólo óptimo cuando todos los trabajos están disponibles al mismo tiempo. Como ejemplo contrario, considere cinco trabajos (del *A* al *E*) con tiempos de ejecución de 2, 4, 1, 1 y 1, respectivamente. Sus tiempos de llegada son 0, 0, 3, 3 y 3. Al principio sólo se pueden elegir *A* o *B*, ya que los otros tres trabajos no han llegado todavía. Utilizando el trabajo más corto primero, ejecutaremos los trabajos en el orden *A*, *B*, *C*, *D*, *E* para un tiempo de espera promedio de 4.6. Sin embargo, al ejecutarlos en el orden *B*, *C*, *D*, *E*, *A* hay una espera promedio de 4.4.

El menor tiempo restante a continuación

Una versión apropiativa del algoritmo tipo el trabajo más corto primero es **el menor tiempo restante a continuación** (SRTN, *Shortest Remaining Time Next*). Con este algoritmo, el planificador siempre selecciona el proceso cuyo tiempo restante de ejecución sea el más corto. De nuevo, se debe conocer el tiempo de ejecución de antemano. Cuando llega un nuevo trabajo, su tiempo total se compara con el tiempo restante del proceso actual. Si el nuevo trabajo necesita menos tiempo para terminar que el proceso actual, éste se suspende y el nuevo trabajo se inicia. Ese esquema permite que los trabajos cortos nuevos obtengan un buen servicio.

2.4.3 Planificación en sistemas interactivos

Ahora analizaremos algunos algoritmos que se pueden utilizar en sistemas interactivos. Estos algoritmos son comunes en computadoras personales, servidores y en otros tipos de sistemas también.

Planificación por turno circular

Uno de los algoritmos más antiguos, simples, equitativos y de mayor uso es el de **turno circular** (*round-robin*). A cada proceso se le asigna un intervalo de tiempo, conocido como **quántum**, durante el cual se le permite ejecutarse. Si el proceso se sigue ejecutando al final del cuanto, la CPU es apropiada para dársela a otro proceso. Si el proceso se bloquea o termina antes de que haya transcurrido el quántum, la conmutación de la CPU se realiza cuando el proceso se bloquea, desde luego. Es fácil implementar el algoritmo de turno circular. Todo lo que el se realizador necesita es mantener una lista de procesos ejecutables, como se muestra en la figura 2-41(a). Cuando el proceso utiliza su quántum, se coloca al final de la lista, como se muestra en la figura 2-41(b).

La única cuestión interesante con el algoritmo de turno circular es la longitud del quántum. Para conmutar de un proceso a otro se requiere cierta cantidad de tiempo para realizar la administración: guardar y cargar tanto registros como mapas de memoria, actualizar varias tablas y listas, vaciar y recargar la memoria caché y así sucesivamente. Suponga que esta **conmutación de proceso** o **conmutación de contexto** (como algunas veces se le llama) requiere 1 mseg, incluyendo el cambio de los mapas de memoria, el vaciado y recarga de la caché, etc. Suponga además que el quántum se establece a 4 mseg. Con estos parámetros, después de realizar 4 mseg de trabajo útil, la CPU tendrá que gastar (es decir, desperdiciar) 1 mseg en la conmutación de procesos. Por ende,

www.FreeLibros.me

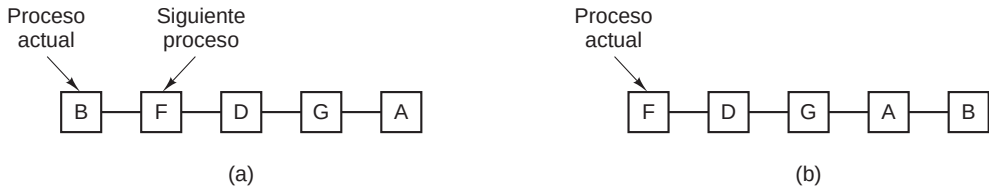


Figura 2-41. Planificación de turno circular. (a) La lista de procesos ejecutables. (b) La lista de procesos ejecutables una vez que *B* utiliza su quántum.

20 por ciento de la CPU se desperdiciará por sobrecarga administrativa. Sin duda, esto es demasiado.

Para mejorar la eficiencia de la CPU, podríamos establecer el quántum a (por decir) 100 mseg. Ahora el tiempo desperdiciado es sólo de 1 por ciento. Pero considere lo que ocurre en un sistema servidor si llegan 50 peticiones dentro de un intervalo de tiempo muy corto y con requerimientos muy variables de la CPU. Se colocarán cincuenta procesos en la lista de procesos ejecutables. Si la CPU está inactiva, el primero empezará de inmediato, el segundo tal vez no inicie sino hasta 100 mseg después y así en lo sucesivo. El último desafortunado tal vez tenga que esperar 5 segundos para poder tener una oportunidad, suponiendo que los demás utilizan sus quántums completos. La mayoría de los usuarios percibirán como lenta una respuesta de 5 segundos a un comando corto. Esta situación es muy mala si algunas de las peticiones cerca del final de la cola requirieron sólo unos cuantos milisegundos de tiempo de la CPU. Con un quántum corto, hubieran obtenido un mejor servicio.

Otro factor es que si al quántum se le asigna un tiempo más largo que la ráfaga promedio de la CPU, la apropiación no ocurrirá con mucha frecuencia. En vez de ello, la mayoría de los procesos realizarán una operación de bloqueo antes de que el quántum se agote, ocasionando una conmutación de proceso. Al eliminar la apropiación mejora el rendimiento, debido a que las conmutaciones de procesos sólo ocurrirán cuando sea lógicamente necesario; es decir, cuando un proceso se bloquee y no pueda continuar.

La conclusión se puede formular de la siguiente manera: si se establece el quántum demasiado corto se producen demasiadas conmutaciones de procesos y se reduce la eficiencia de la CPU, pero si se establece demasiado largo se puede producir una mala respuesta a las peticiones interactivas cortas. A menudo, un quántum con un valor entre 20 y 50 mseg constituye una solución razonable.

Planificación por prioridad

La planificación por turno circular hace la suposición implícita de que todos los procesos tienen igual importancia. Con frecuencia, las personas que poseen y operan computadoras multiusuario tienen diferentes ideas en cuanto a ese aspecto. Por ejemplo, en una universidad el orden jerárquico puede ser: primero los decanos, después los profesores, secretarias, conserjes y por último los estudiantes. La necesidad de tomar en cuenta los factores externos nos lleva a la **planificación por prioridad**. La idea básica es simple: a cada proceso se le asigna una prioridad y el proceso ejecutable con la prioridad más alta es el que se puede ejecutar.

www.FreeLibros.me

Incluso hasta en una PC con un solo propietario puede haber varios procesos, algunos de ellos más importantes que los demás. Por ejemplo, un proceso demonio que envía correo electrónico en segundo plano debería recibir una menor prioridad que un proceso que muestra una película de video en la pantalla en tiempo real.

Para evitar que los procesos con alta prioridad se ejecuten de manera indefinida, el planificador puede reducir la prioridad del proceso actual en ejecución en cada pulso del reloj (es decir, en cada interrupción del reloj). Si esta acción hace que su prioridad se reduzca a un valor menor que la del proceso con la siguiente prioridad más alta, ocurre una conmutación de procesos. De manera alternativa, a cada proceso se le puede asignar un quántum de tiempo máximo que tiene permitido ejecutarse. Cuando este quántum se utiliza, el siguiente proceso con la prioridad más alta recibe la oportunidad de ejecutarse.

A las prioridades se les pueden asignar procesos en forma estática o dinámica. En una computadora militar, los procesos iniciados por los generales podrían empezar con una prioridad de 100, aquéllos iniciados por los coroneles con 90, los mayores con 80, los capitanes con 70, los tenientes con 60 y así sucesivamente. De manera alternativa, en un centro computacional comercial los trabajos con prioridad alta podrían costar $100 por hora, los de prioridad media $75 y los de prioridad baja $50 por hora. El sistema UNIX tiene un comando llamado *nice*, el cual permite a un usuario reducir de manera voluntaria la prioridad de su proceso, para que sea agradable a los otros usuarios. Nadie lo utiliza.

El sistema también puede asignar las prioridades en forma dinámica para lograr ciertos objetivos. Por ejemplo, algunos procesos están muy limitados a E/S y gastan la mayor parte de su tiempo esperando a que la E/S se complete. Cada vez que un proceso así desea la CPU, debe recibirla de inmediato para dejar que inicie su siguiente petición de E/S, que a su vez puede proceder en paralelo con otro proceso que se encuentre realizando cálculos. Hacer que el proceso limitado a E/S espere mucho tiempo por la CPU sólo significa que estará ocupando memoria por un tiempo innecesariamente largo. Un algoritmo simple para dar buen servicio a los procesos limitados a E/S es establecer la prioridad a $1/f$, en donde f es la fracción del último quántum que utilizó un proceso. Un proceso que sólo utilizó 1 mseg de su quántum de 50 mseg obtendría una prioridad de 50, mientras que un proceso que se ejecutara durante 25 mseg antes de bloquearse recibiría la prioridad 2 y un proceso que utilizara el quántum completo recibiría la prioridad 1.

A menudo es conveniente agrupar los procesos en clases de prioridad y utilizar la planificación por prioridad entre las clases, pero la planificación por turno circular dentro de cada clase. La figura 2-42 muestra un sistema con cuatro clases de prioridad. El algoritmo de programación es el siguiente: mientras que haya procesos ejecutables en la clase de prioridad 4, sólo ejecutar cada uno por un cuanto, por turno circular y nunca preocuparse por las clases con menor prioridad. Si la clase de prioridad 4 está vacía, entonces ejecutar los procesos de la clase 3 por turno circular. Si las clases 4 y 3 están vacías, entonces ejecutar la clase 2 por turno circular y así sucesivamente. Si las prioridades no se ajustan de manera ocasional, todas las clases de menor prioridad pueden *morir de hambre* (sin tiempo de CPU).

Múltiples colas

Uno de los primeros planificadores por prioridad estaba en CTSS, el Sistema de tiempo compartido compatible del M.I.T. que se ejecutaba en la IBM 7094 (Corbató y colaboradores, 1962). CTSS

www.FreeLibros.me

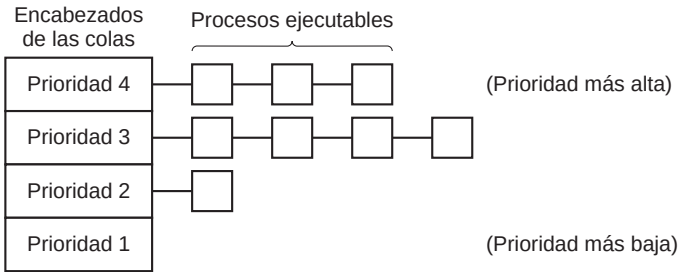


Figura 2-42. Un algoritmo de planificación con cuatro clases de prioridad.

tenía el problema de que la conmutación de procesos era muy lenta, debido a que la 7094 sólo podía contener un proceso en la memoria. Cada conmutación de procesos ocasionaba intercambiar (*swapping*) el proceso actual al disco y leer uno nuevo del disco. Los diseñadores de CTSS se dieron cuenta rápidamente de que era más eficiente dar a los procesos limitados a CPU un quántum largo de vez en cuando, en vez de darles quántums pequeños con frecuencia (para reducir el intercambio). Por otro lado, al proporcionar a todos los procesos un quántum largo se obtendría un tiempo de respuesta pobre, como ya hemos visto. Su solución fue la de establecer clases de prioridades. Los procesos en la clase más alta se ejecutaban durante un quántum. Los procesos en la siguiente clase más alta se ejecutaban por dos quántums. Los procesos en la siguiente clase se ejecutaban por cuatro quántums, y así sucesivamente. Cada vez que un proceso utilizaba todos los quántums que tenía asignados, se movía una clase hacia abajo en la jerarquía.

Como ejemplo, considere un proceso que necesita realizar cálculos en forma continua durante 100 quántums. Al inicio recibe un quántum y después se intercambia con otro proceso en disco. La siguiente vez recibirá dos quántums antes de que se vuelva a intercambiar. En las siguientes ejecuciones obtendrá 4, 8, 12, 16, 32 y 64 quántums, aunque habrá utilizado sólo 37 de los 64 quántums finales para completar su trabajo. Sólo 7 intercambios (incluyendo la carga inicial) se requerirán, en vez de 100, si se utiliza sólo un algoritmo por turno circular. Lo que es más, a medida que el proceso se hunde más y más en las colas de prioridad, se ejecutará con cada vez menos frecuencia, guardando la CPU para procesos cortos e interactivos.

La siguiente política se adoptó para evitar que un proceso, que requería ejecutarse durante un tiempo largo cuando empezó por primera vez pero se volvió interactivo más adelante, fuera castigado para siempre. Cada vez que se escribía un retorno de carro (tecla Intro o Enter) en una terminal, el proceso que pertenecía a esa terminal se movía a la clase de mayor prioridad, con base en la suposición de que estaba a punto de volverse interactivo. Un buen día, algún usuario con un proceso muy limitado a la CPU descubrió que con sólo sentarse en la terminal y teclear retornos de carro al azar durante unos cuantos segundos, su tiempo de respuesta mejoraba en forma considerable. Se lo contó a todos sus amigos. Moraleja de la historia: llegar a lo correcto en la práctica es más difícil que en la teoría.

Se han utilizado muchos otros algoritmos para asignar procesos a las clases de prioridades. Por ejemplo, el influyente sistema XDS 940 (Lampson, 1968) construido en Berkeley tenía cuatro clases de prioridad: terminal, E/S, quántum corto y quántum largo. Cuando un proceso que había estado esperando la entrada de terminal por fin se despertaba, pasaba a la clase de mayor prioridad

www.FreeLibros.me

(terminal). Cuando un proceso en espera de un bloque de disco pasaba al estado listo, se enviaba a la segunda clase. Cuando a un proceso que estaba todavía en ejecución se le agotaba su quántum, al principio se colocaba en la tercera clase. No obstante, si un proceso utilizaba todo su quántum demasiadas veces seguidas sin bloquearse en espera de la terminal o de otro tipo de E/S, se movía hacia abajo hasta la última cola. Muchos otros sistemas utilizan algo similar para favorecer a los usuarios y procesos interactivos en vez de los que se ejecutan en segundo plano.

El proceso más corto a continuación

Como el algoritmo tipo el trabajo más corto primero siempre produce el tiempo de respuesta promedio mínimo para los sistemas de procesamiento por lotes, sería bueno si se pudiera utilizar para los procesos interactivos también. Hasta cierto grado, esto es posible. Por lo general, los procesos interactivos siguen el patrón de esperar un comando, ejecutarlo, esperar un comando, ejecutarlo, etcétera. Si consideramos la ejecución de cada comando como un "trabajo" separado, entonces podríamos minimizar el tiempo de respuesta total mediante la ejecución del más corto primero. El único problema es averiguar cuál de los procesos actuales ejecutables es el más corto.

Un método es realizar estimaciones con base en el comportamiento anterior y ejecutar el proceso con el tiempo de ejecución estimado más corto. Suponga que el tiempo estimado por cada comando para cierta terminal es T_0. Ahora suponga que su siguiente ejecución se mide como T_1. Podríamos actualizar nuestra estimación mediante una suma ponderada de estos dos números, es decir, $aT_0 + (1 - a)T_1$. Por medio de la elección de a podemos decidir hacer que el proceso de estimación olvide las ejecuciones anteriores rápidamente o que las recuerde por mucho tiempo. Con $a = 1/2$, obtenemos estimaciones sucesivas de

$$T_0, \quad T_0/2 + T_1/2, \quad T_0/4 + T_1/4 + T_2/2, \quad T_0/8 + T_1/8 + T_2/4 + T_3/2$$

Después de tres nuevas ejecuciones, el peso de T_0 en la nueva estimación se ha reducido a 1/8.

La técnica de estimar el siguiente valor en una serie mediante la obtención del promedio ponderado del valor actual medido y la estimación anterior se conoce algunas veces como **envejecimiento**. Se aplica en muchas situaciones en donde se debe realizar una predicción con base en valores anteriores. En especial, el envejecimiento es fácil de implementar cuando $a = 1/2$. Todo lo que se necesita es sumar el nuevo valor a la estimación actual y dividir la suma entre 2 (mediante el desplazamiento de un bit a la derecha).

Planificación garantizada

Un método completamente distinto para la planificación es hacer promesas reales a los usuarios acerca del rendimiento y después cumplirlas. Una de ellas, que es realista y fácil de cumplir es: si hay n usuarios conectados mientras usted está trabajando, recibirá aproximadamente $1/n$ del poder de la CPU. De manera similar, en un sistema de un solo usuario con n procesos en ejecución, mientras no haya diferencias, cada usuario debe obtener $1/n$ de los ciclos de la CPU. Eso parece bastante justo.

www.FreeLibros.me

Para cumplir esta promesa, el sistema debe llevar la cuenta de cuánta potencia de CPU ha tenido cada proceso desde su creación. Después calcula cuánto poder de la CPU debe asignarse a cada proceso, a saber el tiempo desde que se creó dividido entre *n*. Como la cantidad de tiempo de CPU que ha tenido cada proceso también se conoce, es simple calcular la proporción de tiempo de CPU que se consumió con el tiempo de CPU al que cada proceso tiene derecho. Una proporción de 0.5 indica que un proceso solo ha tenido la mitad del tiempo que debería tener, y una proporción de 2.0 indica que un proceso ha tenido el doble de tiempo del que debería tener. Entonces, el algoritmo es para ejecutar el proceso con la menor proporción hasta que se haya desplazado por debajo de su competidor más cercano.

Planificación por sorteo

Aunque hacer promesas a los usuarios y cumplirlas es una buena idea, es algo difícil de implementar. Sin embargo, se puede utilizar otro algoritmo para producir resultados similares con una implementación mucho más sencilla. Este algoritmo se conoce como **planificación por sorteo** (Waldspurger y Weihl, 1994).

La idea básica es dar a los procesos boletos de lotería para diversos recursos del sistema, como el tiempo de la CPU. Cada vez que hay que tomar una decisión de planificación, se selecciona un boleto de lotería al azar y el proceso que tiene ese boleto obtiene el recurso. Cuando se aplica a la planificación de la CPU, el sistema podría realizar un sorteo 50 veces por segundo y cada ganador obtendría 20 mseg de tiempo de la CPU como premio.

Parafraseando a George Orwell: “Todos los procesos son iguales, pero algunos son más iguales que otros”. Los procesos más importantes pueden recibir boletos adicionales, para incrementar su probabilidad de ganar. Si hay 100 boletos repartidos y un proceso tiene 20 de ellos, tendrá una probabilidad de 20 por ciento de ganar cada sorteo. A la larga, recibirá un 20 por ciento del tiempo de la CPU. En contraste a un planificador por prioridad, en donde es muy difícil establecer lo que significa tener una prioridad de 40, aquí la regla es clara: un proceso que contenga una fracción *f* de los boletos recibirá aproximadamente una fracción *f* del recurso en cuestión.

La planificación por lotería tiene varias propiedades interesantes. Por ejemplo, si aparece un nuevo proceso y recibe algunos boletos, en el siguiente sorteo tendrá la oportunidad de ganar en proporción al número de boletos que tenga. En otras palabras, la planificación por lotería tiene un alto grado de respuesta.

Los procesos cooperativos pueden intercambiar boletos si lo desean. Por ejemplo, cuando un proceso cliente envía un mensaje a un proceso servidor y después se bloquea, puede dar todos sus boletos al servidor para incrementar la probabilidad de que éste se ejecute a continuación. Cuando el servidor termina, devuelve los boletos para que el cliente se pueda ejecutar de nuevo. De hecho, en ausencia de clientes, los servidores no necesitan boletos.

La planificación por sorteo se puede utilizar para resolver problemas que son difíciles de manejar con otros métodos. Un ejemplo es un servidor de video en el que varios procesos están enviando flujos continuos de video a sus clientes, pero enviando sus cuadros a distintas velocidades. Suponga que el proceso necesita cuadros a 10, 20 y 25 cuadros por segundo. Al asignar a estos procesos 10, 20 y 25 boletos, respectivamente, dividirán de manera automática el tiempo de la CPU en la proporción correcta aproximada; es decir: 10 : 20 : 25.

www.FreeLibros.me

Planificación por partes equitativas

Hasta ahora hemos asumido que cada proceso se planifica por su cuenta, sin importar quién sea su propietario. Como resultado, si el usuario 1 inicia 9 procesos y el usuario 2 inicia 1 proceso, con la planificación por turno circular o por prioridades iguales, el usuario 1 obtendrá 90 por ciento del tiempo de la CPU y el usuario 2 sólo recibirá 10 por ciento.

Para evitar esta situación, algunos sistemas toman en consideración quién es el propietario de un proceso antes de planificarlo. En este modelo, a cada usuario se le asigna cierta fracción de la CPU y el planificador selecciona procesos de tal forma que se cumpla con este modelo. Por ende, si a dos usuarios se les prometió 50 por ciento del tiempo de la CPU para cada uno, eso es lo que obtendrán sin importar cuántos procesos tengan en existencia.

Como ejemplo, considere un sistema con dos usuarios, y a cada uno de los cuales se les prometió 50 por ciento de la CPU. El usuario 1 tiene cuatro procesos (*A*, *B*, *C* y *D*) y el usuario 2 sólo tiene 1 proceso (*E*). Si se utiliza la planificación por turno circular, una posible secuencia de planificación que cumple con todas las restricciones es:

A E B E C E D E A E B E C E D E …

Por otro lado, si el usuario 1 tiene derecho al doble de tiempo de la CPU que el usuario 2, podríamos obtener la siguiente secuencia:

A B E C D E A B E C D E …

Desde luego que existen muchas otras posibilidades, y se pueden explotar dependiendo de cuál sea la noción de equidad.

2.4.4 Planificación en sistemas de tiempo real

En un sistema de **tiempo real**, el tiempo desempeña un papel esencial. Por lo general, uno o más dispositivos físicos externos a la computadora generan estímulo y la computadora debe reaccionar de manera apropiada a ellos dentro de cierta cantidad fija de tiempo. Por ejemplo, la computadora en un reproductor de disco compacto recibe los bits a medida que provienen de la unidad y debe convertirlos en música, en un intervalo de tiempo muy estrecho. Si el cálculo tarda demasiado, la música tendrá un sonido peculiar. Otros sistemas de tiempo real son el monitoreo de pacientes en una unidad de cuidados intensivos de un hospital, el autopiloto en una aeronave y el control de robots en una fábrica automatizada. En todos estos casos, tener la respuesta correcta pero demasiado tarde es a menudo tan malo como no tenerla.

En general, los sistemas de tiempo real se categorizan como de **tiempo real duro**, lo cual significa que hay tiempos límite absolutos que se deben cumplir, y como de **tiempo real suave**, lo cual significa que no es conveniente fallar en un tiempo límite en ocasiones, pero sin embargo es tolerable. En ambos casos, el comportamiento en tiempo real se logra dividiendo el programa en varios procesos, donde el comportamiento de cada uno de éstos es predecible y se conoce de antemano. Por lo general, estos procesos tienen tiempos de vida cortos y pueden ejecutarse hasta completarse en mucho menos de 1 segundo. Cuando se detecta un evento externo, es responsabilidad del planificador planificar los procesos de tal forma que se cumpla con todos los tiempos límite.

www.FreeLibros.me

Los eventos a los que puede llegar a responder un sistema de tiempo real se pueden categorizar como **periódicos** (que ocurren a intervalos regulares) o **aperiódicos** (que ocurren de manera impredecible). Tal vez un sistema tenga que responder a varios flujos de eventos periódicos. Dependiendo de cuánto tiempo requiera cada evento para su procesamiento, tal vez ni siquiera sea posible manejarlos a todos. Por ejemplo, si hay m eventos periódicos y el evento i ocurre con el periodo P_i y requiere C_i segundos de tiempo de la CPU para manejar cada evento, entonces la carga sólo se podrá manejar si

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Se dice que un sistema de tiempo real que cumple con este criterio es **planificable**.

Como ejemplo, considere un sistema de tiempo real con tres eventos periódicos, con periodos de 100, 200 y 500 mseg, respectivamente. Si estos eventos requieren 50, 30 y 100 mseg de tiempo de la CPU por evento, respectivamente, el sistema es planificable debido a que $0.5 + 0.15 + 0.2 < 1$. Si se agrega un cuarto evento con un periodo de 1 segundo, el sistema seguirá siendo planificable mientras que este evento no requiera más de 150 mseg de tiempo de la CPU por evento. En este cálculo está implícita la suposición de que la sobrecarga por la conmutación de contexto es tan pequeña que se puede ignorar.

Los algoritmos de planificación en tiempo real pueden ser estáticos o dinámicos. Los primeros toman sus decisiones de planificación antes de que el sistema empiece a ejecutarse. Los segundos lo hacen durante el tiempo de ejecución. La planificación estática sólo funciona cuando hay información perfecta disponible de antemano acerca del trabajo que se va a realizar y los tiempos límite que se tienen que cumplir. Los algoritmos de planificación dinámicos no tienen estas restricciones. Aplazaremos nuestro estudio de algoritmos específicos hasta el capítulo 7, donde trataremos los sistemas de multimedia en tiempo real.

2.4.5 Política contra mecanismo

Hasta ahora hemos supuesto tácitamente que todos los procesos en el sistema pertenecen a distintos usuarios y por lo tanto compiten por la CPU. Aunque esto es a menudo cierto, algunas veces sucede que un proceso tiene muchos hijos ejecutándose bajo su control. Por ejemplo, un proceso de un sistema de administración de bases de datos puede tener muchos hijos. Cada hijo podría estar trabajando en una petición distinta o cada uno podría tener cierta función específica que realizar (análisis de consultas o acceso al disco, por ejemplo). Es por completo posible que el proceso principal tenga una idea excelente acerca de cuál de sus hijos es el más importante (o que requiere tiempo con más urgencia) y cuál es el menos importante. Por desgracia, ninguno de los planificadores antes descritos acepta entrada de los procesos de usuario acerca de las decisiones de planificación. Como resultado, raras veces el planificador toma la mejor decisión.

La solución a este problema es separar el **mecanismo de planificación** de la **política de planificación**, un principio establecido desde hace tiempo (Levin y colaboradores, 1975). Esto significa que el algoritmo de planificación está parametrizado de cierta forma, pero los procesos de usuario pueden llenar los parámetros. Consideremos el ejemplo de la base de datos una vez más.

www.FreeLibros.me

Suponga que el kernel utiliza un algoritmo de planficación por prioridad, pero proporciona una llamada al sistema mediante la cual un proceso puede establecer (y modificar) las prioridades de sus hijos. De esta forma, el padre puede controlar con detalle la forma en que se planifican sus hijos, aun y cuando éste no se encarga de la planificación. Aquí el mecanismo está en el kernel, pero la política se establece mediante un proceso de usuario.

2.4.6 Planificación de hilos

Cuando varios procesos tienen múltiples hilos cada uno, tenemos dos niveles de paralelismo presentes: procesos e hilos. La planificación en tales sistemas difiere en forma considerable, dependiendo de si hay soporte para hilos a nivel usuario o para hilos a nivel kernel (o ambos).

Consideremos primero los hilos a nivel usuario. Como el kernel no está consciente de la existencia de los hilos, opera en la misma forma de siempre: selecciona un proceso, por decir *A*, y otorga a este proceso el control de su quántum. El planificador de hilos dentro de *A* decide cuál hilo ejecutar, por decir *A1*. Como no hay interrupciones de reloj para multiprogramar hilos, este hilo puede continuar ejecutándose todo el tiempo que quiera. Si utiliza todo el quántum del proceso, el kernel seleccionará otro proceso para ejecutarlo.

Cuando el proceso *A* por fin se ejecute de nuevo, el hilo *A1* continuará su ejecución. Seguirá consumiendo todo el tiempo de *A* hasta que termine. Sin embargo, su comportamiento antisocial no afectará a los demás procesos. Éstos recibirán lo que el planificador de procesos considere que es su parte apropiada, sin importar lo que esté ocurriendo dentro del proceso *A*.

Ahora considere el caso en el que los hilos de *A* tienen relativamente poco trabajo que realizar por cada ráfaga de la CPU; por ejemplo, 5 mseg de trabajo dentro de un quántum de 50 mseg. En consecuencia, cada uno se ejecuta por unos instantes y después entrega la CPU al planificador de hilos. Esto podría producir la secuencia *A1*, *A2*, *A3*, *A1*, *A2*, *A3*, *A1*, *A2*, *A3*, *A1* antes de que el kernel conmute al proceso *B*. Esta situación se ilustra en la figura 2-43(a).

El algoritmo de planificación utilizado por el sistema en tiempo de ejecución puede ser cualquiera de los antes descritos. En la práctica, los algoritmos de planificación por turno circular y de planificación por prioridad son los más comunes. La única restricción es la ausencia de un reloj para interrumpir a un proceso que se ha ejecutado por mucho tiempo.

Ahora considere la situación con hilos a nivel kernel. Aquí el kernel selecciona un hilo específico para ejecutarlo. No tiene que tomar en cuenta a cuál proceso pertenece el hilo, pero puede hacerlo si lo desea. El hilo recibe un quántum y se suspende obligatoriamente si se excede de este quántum. Con un quántum de 50 mseg pero hilos que se bloquean después de 5 mseg, el orden de los hilos para cierto periodo de 30 mseg podría ser *A1*, *B1*, *A2*, *B2*, *A3*, *B3*, algo que no sería posible con estos parámetros e hilos a nivel usuario. Esta situación se ilustra en forma parcial en la figura 2-43(b).

Una diferencia importante entre los hilos a nivel usuario y los hilos a nivel kernel es el rendimiento. Para realizar un conmutación de hilos con hilos a nivel usuario se requiere de muchas instrucciones de máquina. Con hilos a nivel kernel se requiere una conmutación de contexto total, cambiar el mapa de memoria e invalidar la caché, lo cual es varias órdenes de magnitud más lento. Por otro lado, con los hilos a nivel kernel, cuando un hilo se bloquea en espera de E/S no se suspende todo el proceso, como con los hilos a nivel usuario.

www.FreeLibros.me

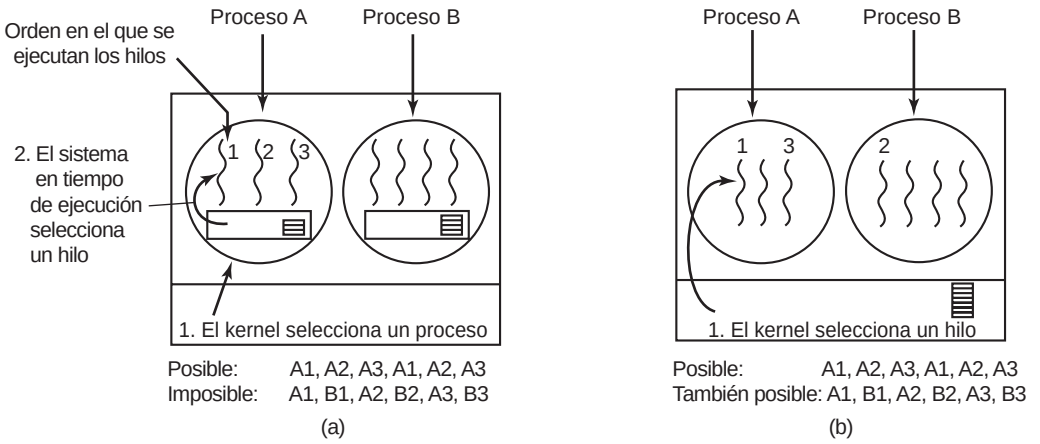


Figura 2-43. (a) Posible planificación de hilos a nivel usuario con un quántum de 50 mseg para cada proceso e hilos que se ejecutan durante 5 mseg por cada ráfaga de la CPU. (b) Posible planificación de hilos a nivel kernel con las mismas características que (a).

Como el kernel sabe que es más costoso conmutar de un hilo en el proceso *A* a un hilo en el proceso *B* que ejecutar un segundo hilo en el proceso *A* (debido a que tiene que cambiar el mapa de memoria y se arruina la memoria caché), puede considerar esta información al tomar una decisión. Por ejemplo, dados dos hilos que son de igual importancia, en donde uno de ellos pertenece al mismo proceso que un hilo que se acaba de bloquear y el otro pertenece a un proceso distinto, se podría dar preferencia al primero.

Otro factor importante es que los hilos a nivel usuario pueden emplear un planificador de hilos específico para la aplicación. Por ejemplo, considere el servidor Web de la figura 2-8. Suponga que un hilo trabajador acaba de bloquearse y que el hilo despachador así como dos hilos trabajadores están listos. ¿Quién debe ejecutarse a continuación? El sistema en tiempo de ejecución, que sabe lo que todos los hilos hacen, puede seleccionar fácilmente el despachador para que se ejecute a continuación, de manera que pueda iniciar la ejecución de otro hilo trabajador. Esta estrategia maximiza la cantidad de paralelismo en un entorno en el que los trabajadores se bloquean con frecuencia en la E/S de disco. Con los hilos a nivel kernel, el kernel nunca sabría lo que hace cada hilo (aunque se les podría asignar distintas prioridades). Sin embargo, en general los planificadores de hilos específicos para la aplicación pueden optimizar mejor una aplicación de lo que el kernel puede.

2.5 PROBLEMAS CLÁSICOS DE COMUNICACIÓN ENTRE PROCESOS (IPC)

La literatura de sistemas operativos está repleta de interesantes problemas que se han descrito y analizado ampliamente, mediante el uso de una variedad de métodos de sincronización. En las siguientes secciones examinaremos tres de los problemas más conocidos.

www.FreeLibros.me

2.5.1 El problema de los filósofos comelones

En 1965, Dijkstra propuso y resolvió un problema de sincronización al que llamó el **problema de los filósofos comelones**. Desde ese momento, todos los que inventaban otra primitiva de sincronización se sentían obligados a demostrar qué tan maravillosa era esa nueva primitiva, al mostrar con qué elegancia resolvía el problema de los filósofos comelones. Este problema se puede enunciar simplemente de la siguiente manera. Cinco filósofos están sentados alrededor de una mesa circular. Cada filósofo tiene un plato de espagueti. El espagueti es tan resbaloso, que un filósofo necesita dos tenedores para comerlo. Entre cada par de platos hay un tenedor. La distribución de la mesa se ilustra en la figura 2-44.

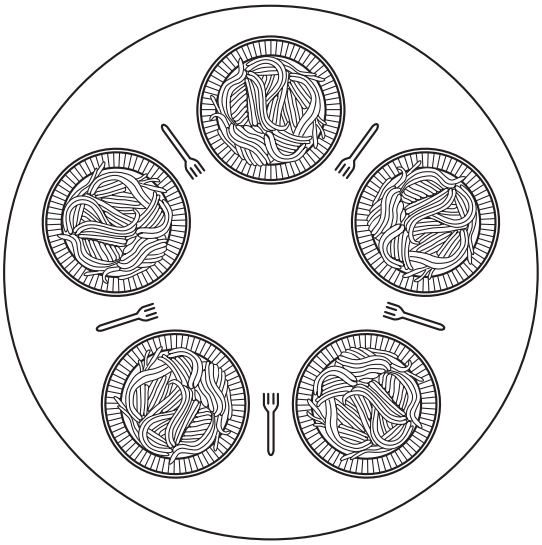


Figura 2-44. Hora de comer en el Departamento de Filosofía.

La vida de un filósofo consiste en periodos alternos de comer y pensar (esto es algo así como una abstracción, incluso para los filósofos, pero las otras actividades son irrelevantes aquí). Cuando un filósofo tiene hambre, trata de adquirir sus tenedores izquierdo y derecho, uno a la vez, en cualquier orden. Si tiene éxito al adquirir dos tenedores, come por un momento, después deja los tenedores y continúa pensando. La pregunta clave es: ¿puede usted escribir un programa para cada filósofo, que haga lo que se supone debe hacer y nunca se trabe? (Hemos recalcado que el requerimiento de los dos tenedores es algo artificial; tal vez deberíamos cambiar de comida italiana a comida china y sustituir el espagueti por arroz y los tenedores por palillos chinos).

La figura 2-45 muestra la solución obvia. El procedimiento *tomar_tenedor* espera hasta que el tenedor específico esté disponible y luego lo toma. Por desgracia, la solución obvia está mal. Suponga que los cinco filósofos toman sus tenedores izquierdos al mismo tiempo. Ninguno podrá tomar sus tenedores derechos y habrá un interbloqueo.

www.FreeLibros.me

```
#define N 5                                 /* número de filósofos */

void filosofo(int i)                        /* i: número de filósofo, de 0 a 4 */
{
  while(TRUE){
        pensar();                           /* el filósofo está pensando */
        tomar_tenedor(i);                   /* toma tenedor izquierdo */
        tomar_tenedor((i+1) % N);           /* toma tenedor derecho; % es el operador módulo */
        comer();                            /* come espagueti */
        poner_tenedor(i);                   /* pone tenedor izquierdo de vuelta en la mesa */
        poner_tenedor((i+1) % N);           /* pone tenedor derecho de vuelta en la mesa */
  }
}
```

Figura 2-45. Una solución incorrecta para el problema de los filósofos comelones.

Podríamos modificar el programa de manera que después de tomar el tenedor izquierdo, el programa compruebe para ver si el tenedor derecho está disponible. Si no lo está, el filósofo regresa el tenedor izquierdo, espera cierto tiempo y después repite todo el proceso. Esta proposición falla también, aunque por una razón distinta. Con un poco de mala suerte, todos los filósofos podrían iniciar el algoritmo en forma simultánea, tomarían sus tenedores izquierdos, verían que sus tenedores derechos no están disponibles, regresarían sus tenedores izquierdos, esperarían, volverían a tomar sus tenedores izquierdos al mismo tiempo y así en lo sucesivo, eternamente. Una situación como ésta, en la que todos los programas continúan ejecutándose en forma indefinida pero no progresan se conoce como **inanición** (*starvation*) (se llama inanición, aun cuando el problema no ocurre en un restaurante italiano o chino).

Ahora podríamos pensar que si los filósofos sólo esperan por un tiempo aleatorio en vez de esperar durante el mismo tiempo al no poder adquirir el tenedor derecho, la probabilidad de que todo continúe bloqueado durante incluso una hora es muy pequeña. Esta observación es verdad y en casi todas las aplicaciones intentar de nuevo en un tiempo posterior no representa un problema. Por ejemplo, en la popular red de área local Ethernet, si dos computadoras envían un paquete al mismo tiempo, cada una espera durante un tiempo aleatorio e intenta de nuevo; en la práctica esta solución funciona bien. Sin embargo, en algunas cuantas aplicaciones sería preferible una solución que funcione siempre y que no pueda fallar debido a una serie improbable de números aleatorios. Piense acerca del control de la seguridad en una planta de energía nuclear.

Una mejora a la figura 2-45 que no tiene interbloqueo ni inanición es proteger las cinco instrucciones que siguen de la llamada a *pensar* mediante un semáforo binario. Antes de empezar a adquirir tenedores, un filósofo realizaría una operación `down` en *mutex*. Después de regresar los tenedores, realizaría una operación `up` en *mutex*. Desde un punto de vista teórico, esta solución es adecuada. Desde un punto de vista práctico, tiene un error de rendimiento: sólo puede haber un filósofo comiendo en cualquier instante. Con cinco tenedores disponibles, deberíamos poder permitir que dos filósofos coman al mismo tiempo.

La solución que se presenta en la figura 2-46 está libre de interbloqueos y permite el máximo paralelismo para un número arbitrario de filósofos. Utiliza un arreglo llamado *estado* para llevar el

www.FreeLibros.me

```
#define N                   5                   /* número de filósofos */
#define IZQUIERDO           (i+N-1)%N           /* número del vecino izquierdo de i */
#define DERECHO             (i+1)%N             /* número del vecino derecho de i */
#define PENSANDO            0                   /* el filósofo está pensando */
#define HAMBRIENTO          1                   /* el filósofo trata de obtener los tenedores */
#define COMIENDO            2                   /* el filósofo está comiendo */
typedef int semaforo;                           /* los semáforos son un tipo especial de int */
int estado[N];                                  /* arreglo que lleva registro del estado de todos*/
semaforo mutex = 1;                             /* exclusión mutua para las regiones críticas */
semaforo s[N];                                  /* un semáforo por filósofo */

void filosofo(int i)                            /* i: número de filósofo, de 0 a N−1 */
{
     while(TRUE){                               /* se repite en forma indefinida */
          pensar();                             /* el filósofo está pensando */
          tomar_tenedores(i);                   /* adquiere dos tenedores o se bloquea */
          comer();                              /* come espagueti */
          poner_tenedores(i);                   /* pone de vuelta ambos tenedores en la mesa */
     }
}

void tomar_tenedores(int i)                     /* i: número de filósofo, de 0 a N−1 */
{
     down(&mutex);                              /* entra a la región crítica */
     estado[i] = HAMBRIENTO;                    /* registra el hecho de que el filósofo i está hambriento */
     probar(i);                                 /* trata de adquirir 2 tenedores */
     up(&mutex);                                /* sale de la región crítica */
     down(&s[i]);                               /* se bloquea si no se adquirieron los tenedores */
}

void poner_tenedores(i)                         /* i: número de filósofo, de 0 a N–1 */
{
     down(&mutex);                              /* entra a la región crítica */
     estado[i] = PENSANDO;                      /* el filósofo terminó de comer */
     probar(IZQUIERDO);                         /* verifica si el vecino izquierdo puede comer ahora */
     probar(DERECHO);                           /* verifica si el vecino derecho puede comer ahora */
     up(&mutex);                                /* sale de la región crítica */
}

void probar(i)              /* i: número de filósofo, de 0 a N−1 */
{
     if (estado[i] == HAMBRIENTO && estado[IZQUIERDO] != COMIENDO && estado[DERECHO] != COMIENDO) {
          estado[i] = COMIENDO;
          up(&s[i]);
     }
}
```

Figura 2-46. Una solución al problema de los filósofos comelones.

www.FreeLibros.me

registro de si un filósofo está comiendo, pensando o hambriento (tratando de adquirir tenedores). Un filósofo sólo se puede mover al estado de comer si ningún vecino está comiendo. Los *i* vecinos del filósofo se definen mediante las macros *IZQUIERDO* y *DERECHO*. En otras palabras, si *i* es 2, *IZQUIERDO* es 1 y *DERECHO* es 3.

El programa utiliza un arreglo de semáforos, uno por cada filósofo, de manera que los filósofos hambrientos puedan bloquearse si los tenedores que necesitan están ocupados. Observe que cada proceso ejecuta el procedimiento *filosofo* como su código principal, pero los demás procedimientos (*tomar_tenedores*, *poner_tenedores* y *probar*) son ordinarios y no procesos separados.

2.5.2 El problema de los lectores y escritores

El problema de los filósofos comelones es útil para modelar procesos que compiten por el acceso exclusivo a un número limitado de recursos, como los dispositivos de E/S. Otro problema famoso es el de los lectores y escritores (Courtois y colaboradores, 1971), que modela el acceso a una base de datos. Por ejemplo, imagine un sistema de reservación de aerolíneas, con muchos procesos en competencia que desean leer y escribir en él. Es aceptable tener varios procesos que lean la base de datos al mismo tiempo, pero si un proceso está actualizando (escribiendo) la base de datos, ningún otro proceso puede tener acceso a la base de datos, ni siquiera los lectores. La pregunta es, ¿cómo se programan los lectores y escritores? Una solución se muestra en la figura 2-47.

En esta solución, el primer lector en obtener acceso a la base de datos realiza una operación `down` en el semáforo *bd*. Los siguientes lectores simplemente incrementan un contador llamado *cl*. A medida que los lectores van saliendo, decrementan el contador y el último realiza una operación `up` en el semáforo, para permitir que un escritor bloqueado (si lo hay) entre.

La solución que se presenta aquí contiene en forma implícita una decisión sutil que vale la pena observar. Suponga que mientras un lector utiliza la base de datos, llega otro lector. Como no es un problema tener dos lectores al mismo tiempo, el segundo lector es admitido. También se pueden admitir más lectores, si es que llegan.

Ahora suponga que aparece un escritor. Tal vez éste no sea admitido a la base de datos, ya que los escritores deben tener acceso exclusivo y por ende, el escritor se suspende. Más adelante aparecen lectores adicionales. Mientras que haya un lector activo, se admitirán los siguientes lectores. Como consecuencia de esta estrategia, mientras que haya un suministro continuo de lectores, todos entrarán tan pronto lleguen. El escritor estará suspendido hasta que no haya un lector presente. Si llega un nuevo lector, por decir cada 2 segundos y cada lector requiere 5 segundos para hacer su trabajo, el escritor nunca entrará.

Para evitar esta situación, el programa se podría escribir de una manera ligeramente distinta: cuando llega un lector y hay un escritor en espera, el lector se suspende detrás del escritor, en vez de ser admitido de inmediato. De esta forma, un escritor tiene que esperar a que terminen los lectores que estaban activos cuando llegó, pero no tiene que esperar a los lectores que llegaron después de él. La desventaja de esta solución es que logra una menor concurrencia y por ende, un menor rendimiento. Courtois y sus colaboradores presentan una solución que da prioridad a los escritores. Para obtener detalles, consulte la bibliografía.

www.FreeLibros.me

```
typedef int semaforo;                       /* use su imaginación */
semaforo mutex=1;                           /* controla el acceso a 'cl' */
semaforo bd=1;                              /* controla el acceso a la base de datos */
int cl=0;                                   /* # de procesos que leen o desean hacerlo */

void lector(void)
{
        while(TRUE){                        /* se repite de manera indefinida */
            down(&mutex);                   /* obtiene acceso exclusivo a 'cl' */
            cl = cl + 1;                    /* ahora hay un lector más */
            if (cl == 1) down(&bd);         /* si este es el primer lector ... */
            up(&mutex);                     /* libera el acceso exclusivo a 'cl' */
            leer_base_de_datos();           /* accede a los datos */
            down(&mutex);                   /* obtiene acceso exclusivo a 'cl' */
            cl = cl – 1;                    /* ahora hay un lector menos */
            if (cl == 0) up(&bd);           /* si este es el último lector ... */
            up(&mutex);                     /* libera el acceso exclusivo a 'cl' */
            usar_lectura_datos();           /* región no crítica */
        }
}

void escritor(void)
{
            while(TRUE){                    /* se repite de manera indefinida */
            pensar_datos();                 /* región no crítica */
            down(&bd);                      /* obtiene acceso exclusivo */
            escribir_base_de_datos();       /* actualiza los datos */
            up(&bd);                        /* libera el acceso exclusivo */
        }
}
```

Figura 2-47. Una solución al problema de los lectores y escritores.

2.6 INVESTIGACIÓN ACERCA DE LOS PROCESOS E HILOS

En el capítulo 1 analizamos parte de la investigación actual acerca de la estructura de los sistemas operativos. En este capítulo y otros, veremos una investigación con enfoque más específico, empezando con los procesos. Como se aclarará con el tiempo, algunos temas están más asentados que otros. La mayor parte de la investigación tiende a estar en los nuevos temas, en vez de los que ya han estado presentes por décadas.

El concepto de un proceso es un ejemplo de algo que está bastante bien asentado. Casi todo sistema tiene cierta noción de un proceso como contenedor para agrupar recursos relacionados como un espacio de direcciones, hilos, archivos abiertos, permisos de protección, etcétera. Los distintos sistemas realizan estos agrupamientos en formas ligeramente diferentes, pero éstas son sólo diferencias de ingeniería. La idea básica ya no es muy controversial y hay poca investigación en cuanto al tema de los procesos.

www.FreeLibros.me

Los hilos son una idea más reciente que los procesos, pero también han estado presentes por un buen tiempo. Aún así, de vez en cuando aparece un artículo ocasional acerca de los hilos, por ejemplo, acerca de la agrupamientos de hilos en multiprocesadores (Tam y colaboradores, 2007) o escalando el número de hilos en un proceso a 100,000 (Von Behren y colaboradores, 2003).

La sincronización de procesos es un tema bastante establecido hoy en día, pero aún así de vez en cuando se presenta algún artículo, como uno acerca del procesamiento concurrente sin candados (Fraser y Harris, 2007) o la sincronización sin bloqueo en los sistemas de tiempo real (Hohmuth y Haertig, 2001).

La planificación de procesos (tanto en uniprocesadores como en multiprocesadores) sigue siendo un tema cercano y querido para algunos investigadores. Algunos temas que se están investigando incluyen la planificación con eficiencia de energía en dispositivos móviles (Yuan y Nahrstedt, 2006), la planificación con capacidad para hiperhilamiento (Bulpin y Pratt, 2005), qué hacer cuando la CPU estaría inactiva en cualquier otro caso (Eggert y Touch, 2005) y la planificación del tiempo virtual (Nieh y colaboradores, 2001). Sin embargo, hay pocos diseñadores de sistemas en la actualidad en busca de un algoritmo decente de planificación de hilos, por lo que parece ser que este tipo de investigación es más por interés del investigador que por verdadera demanda. Con todo, los procesos, hilos y la planificación de los mismos no son temas de tanto interés como antes fueron. La investigación ha avanzado a otras áreas.

2.7 RESUMEN

Para ocultar los efectos de las interrupciones, los sistemas operativos proporcionan un modelo conceptual que consiste en procesos secuenciales ejecutándose en paralelo. Los procesos se pueden crear y terminar en forma dinámica. Cada proceso tiene su propio espacio de direcciones.

Para algunas aplicaciones es conveniente tener varios hilos de control dentro de un solo proceso. Estos hilos se planifican de manera independiente y cada uno tiene su propia pila, pero todos los hilos en un proceso comparten un espacio de direcciones común. Los hilos se pueden implementar en espacio de usuario o en el de kernel.

Los procesos se pueden comunicar entre sí mediante el uso de las primitivas de comunicación entre procesos, como semáforos, monitores o mensajes. Estas primitivas se utilizan para asegurar que no haya dos procesos en sus regiones críticas al mismo tiempo, una situación que produce un caos. Un proceso puede estar en ejecución, listo para ejecutarse o bloqueado, pudiendo cambiar de estado cuando éste u otro proceso ejecute una de las primitivas de comunicación entre procesos. La comunicación entre hilos es similar.

Las primitivas de comunicación entre procesos se pueden utilizar para resolver problemas tales como el del productor-consumidor, los filósofos comelones y el de los lectores y escritores. Aún con estas primitivas, hay que tener cuidado para evitar errores e interbloqueos.

Se han estudiado muchos algoritmos de planificación. Algunos de ellos se utilizan principalmente para sistemas de procesamiento por lotes, como la planificación del trabajo más corto primero. Otros son comunes tanto en los sistemas de procesamiento por lotes como en los sistemas interactivos. Estos algoritmos incluyen el de planificación por turno circular, por prioridad, colas de

www.FreeLibros.me

multiniveles, planificación garantizada, planificación por lotería y planificación por partes equitativas. Algunos sistemas hacen una clara separación entre el mecanismo de planificación y la política de planificación, lo cual permite a los usuarios tener control del algoritmo de planificación.

PROBLEMAS

1. En la figura 2-2 se muestran los estados de tres procesos. En teoría, con tres estados podría haber seis transiciones, dos fuera de cada estado. Sin embargo sólo se muestran cuatro transiciones. ¿Existe alguna circunstancia en la que una o ambas de las transiciones faltantes pudiera ocurrir?

2. Suponga que debe diseñar una arquitectura de computadora avanzada que hiciera conmutación de procesos en el hardware, en vez de tener interrupciones. ¿Qué información necesitaría la CPU? Describa cómo podría trabajar la conmutación de procesos por hardware.

3. En todas las computadoras actuales, al menos una parte de los manejadores de interrupciones se escriben en lenguaje ensamblador. ¿Por qué?

4. Cuando una interrupción o una llamada al sistema transfiere el control al sistema operativo, por lo general se utiliza un área de la pila del kernel separada de la pila del proceso interrumpido. ¿Por qué?

5. Varios trabajos se pueden ejecutar en paralelo y terminar con más rapidez que si se hubieran ejecutado en secuencia. Suponga que dos trabajos, cada uno de los cuales necesita 10 minutos de tiempo de la CPU, inician al mismo tiempo. ¿Cuánto tiempo tardará el último en completarse, si se ejecutan en forma secuencial? ¿Cuánto tiempo si se ejecutan en paralelo? Suponga que hay 50% de espera de E/S.

6. En el texto se estableció que el modelo de la figura 2-11(a) no era adecuado para un servidor de archivos que utiliza una memoria caché. ¿Por qué no? ¿Podría cada proceso tener su propia caché?

7. Si un proceso con multihilamiento utiliza la operación fork, ocurre un problema si el hijo obtiene copias de todos los hilos del padre. Suponga que uno de los hilos originales estaba esperando la entrada del teclado. Ahora hay dos hilos esperando la entrada del teclado, uno en cada proceso. ¿Acaso ocurre este problema en procesos con un solo hilo?

8. En la figura 2-8 se muestra un servidor Web con multihilamiento. Si la única forma de leer un archivo es la llamada al sistema read normal con bloqueo, ¿cree usted que se están usando hilos a nivel usuario o hilos a nivel kernel para el servidor Web? ¿Por qué?

9. En el texto describimos un servidor Web con multihilamiento, mostrando por qué es mejor que un servidor con un solo hilo y que un servidor de máquina de estados finitos. ¿Hay alguna circunstancia en la cual un servidor con un solo hilo podría ser mejor? Dé un ejemplo.

10. En la figura 2-12, el conjunto de registros se lista por hilos, en vez de por procesos. ¿Por qué? Después de todo, la máquina sólo tiene un conjunto de registros.

11. ¿Por qué un hilo otorgaría de manera voluntaria la CPU al llamar a *thread_yield*? Después de todo, como no hay una interrupción periódica de reloj, tal vez nunca obtenga la CPU de vuelta.

12. ¿Puede darse alguna vez el apropiamiento de un hilo mediante una interrupción de reloj? De ser así, ¿bajo qué circunstancias? Si no es así, ¿por qué no?

www.FreeLibros.me

13. En este problema debe comparar la lectura de un archivo, utilizando un servidor de archivos con un solo hilo y un servidor multihilado. Se requieren 15 mseg para obtener una petición, despacharla y realizar el resto del procesamiento necesario, suponiendo que los datos necesarios están en la caché del bloque. Si se necesita una operación de disco, como es el caso una tercera parte del tiempo, se requieren 75 mseg adicionales, durante los cuales el hilo duerme. ¿Cuántas peticiones por segundo puede manejar el servidor, si es de un solo hilo? ¿Si es multihilado?

14. ¿Cuál es la mayor ventaja de implementar hilos en espacio de usuario? ¿Cuál es la mayor desventaja?

15. En la figura 2-15, las creaciones de hilos y los mensajes impresos por los mismos se intercalan al azar. ¿Hay alguna forma de forzar el orden para que sea estrictamente: hilo 1 creado, hilo 1 imprime mensaje, hilo 1 termina, hilo 2 creado, hilo 2 imprime mensaje, hilo 2 termina, y así en lo sucesivo? De ser así, ¿cómo se puede hacer? Si no es así, ¿por qué no?

16. En el análisis de las variables globales en hilos, utilizamos un procedimiento llamado *crear_global* para asignar espacio para un apuntador a la variable, en vez de la misma variable. ¿Es esto esencial o podrían trabajar los procedimientos con los valores en sí mismos de igual forma?

17. Considere un sistema en el cual los hilos se implementan por completo en espacio de usuario, en donde el sistema en tiempo de ejecución obtiene una interrupción de reloj una vez por segundo. Suponga que ocurre una interrupción de reloj mientras un hilo se ejecuta en el sistema en tiempo de ejecución. ¿Qué problema podría ocurrir? ¿Puede usted sugerir una forma de resolverlo?

18. Suponga que un sistema operativo no tiene algo parecido a la llamada al sistema **select** para ver por adelantado si es seguro leer de un archivo, canal o dispositivo, pero que sí permite establecer relojes de alarma que interrumpen las llamadas bloqueadas al sistema. ¿Es posible implementar un paquete de hilos en espacio de usuario bajo estas condiciones? Explique.

19. ¿Puede el problema de inversión de prioridades que vimos en la sección 2.3.4 ocurrir con hilos a nivel usuario? ¿Por qué si o por qué no?

20. En la sección 2.3.4 se describió una situación con un proceso de alta prioridad *H* y un proceso de baja prioridad *L*, que ocasionaba que *H* iterara en forma indefinida. ¿Ocurre el mismo problema si se utiliza la planificación por turno circular en vez de la planificación por prioridad? Explique.

21. En un sistema con hilos, ¿hay una pila por cada hilo o una pila por cada proceso cuando se utilizan hilos a nivel usuario? ¿y cuando se utilizan hilos a nivel de kernel? Explique.

22. Cuando se está desarrollando una computadora, por lo general primero se simula mediante un programa que ejecuta una instrucción a la vez. Incluso hasta los multiprocesadores se simulan estrictamente en forma secuencial como ésta. ¿Es posible que ocurra una condición de carrera cuando no hay eventos simultáneos como éste?

23. ¿Funciona la solución de espera ocupada en la que se utiliza la variable *turno* (figura 2-23) cuando los dos procesos se ejecutan en un multiprocesador con memoria compartida, es decir, dos CPU que comparten una memoria común?

24. La solución de Peterson al problema de exclusión mutua que se muestra en la figura 2-24, ¿funciona cuando la planificación es apropiativa? ¿Y qué pasa cuando es no apropiativa?

25. Dé un bosquejo acerca de cómo un sistema operativo que puede deshabilitar interrupciones podría implementar semáforos.

www.FreeLibros.me

26. Muestre cómo pueden implementarse los semáforos contadores (es decir, semáforos que pueden contener un valor arbitrario) utilizando sólo semáforos binarios e instrucciones de máquina ordinarias.

27. Si un sistema sólo tiene dos procesos, ¿tiene sentido utilizar una barrera para sincronizarlos? ¿Por qué sí o por qué no?

28. ¿Pueden dos hilos en el mismo proceso sincronizarse mediante un semáforo de kernel, si los hilos son implementados por el kernel? ¿Qué pasa si se implementan en espacio de usuario? Suponga que ningún hilo de ningún otro proceso tiene acceso al semáforo. Analice sus respuestas.

29. La sincronización con monitores utiliza variables de condición y dos operaciones especiales: wait y signal. Una forma más general de sincronización tendría que tener una sola primitiva, waituntil, que tuviera un predicado booleano arbitrario como parámetro. Así, podríamos decir por ejemplo:

waituntil $x < 0$ o $y + z < n$

La primitiva signal no sería ya necesaria. Es evidente que este esquema es más general que el de Hoare o Brinch Hansen, pero no se utiliza. ¿Por qué no? *Sugerencia:* Considere la implementación.

30. Un restaurante de comida rápida tiene cuatro tipos de empleados: (1) los que toman pedidos de los clientes; (2) los cocineros, que preparan la comida; (3) los especialistas de empaquetado, que meten la comida en bolsas; y (4) los cajeros, que entregan las bolsas a los clientes y reciben su dinero. Cada empleado puede considerarse como un proceso secuencial comunicativo. ¿Qué forma de comunicación entre procesos utilizan? Relacione este modelo con los procesos en UNIX.

31. Suponga que tenemos un sistema de paso de mensajes que utiliza buzones. Al enviar a un buzón lleno o al tratar de recibir de uno vacío, un proceso no se bloquea. En vez de ello, recibe de vuelta un código de error. Para responder al código de error, el proceso sólo vuelve a intentar, una y otra vez, hasta tener éxito. ¿Produce este esquema condiciones de carrera?

32. Las computadoras CDC 6000 podían manejar hasta 10 procesos de E/S en forma simultánea, utilizando una forma interesante de planificación por turno circular conocida como **compartición del procesador**. Se llevaba a cabo una conmutación de procesos después de cada instrucción, por lo que la instrucción 1 provenía del proceso 1, la instrucción 2 del proceso 2, etcétera. La conmutación de procesos se realizaba mediante hardware especial y la sobrecarga era de cero. Si un proceso necesitaba T segundos para completarse en ausencia de competencia, ¿cuánto tiempo necesitaría si se utilizara la compartición del procesador con n procesos?

33. ¿Puede una medida determinar, analizando el código fuente, si un proceso es más probable que esté limitado a CPU o limitado a E/S? ¿Cómo puede determinarse esto en tiempo de ejecución?

34. En la sección "Cuándo se deben planificar" se mencionó que algunas veces la planificación se podría mejorar si un proceso importante pudiera desempeñar un papel al seleccionar el siguiente proceso a ejecutar al bloquearse. Mencione una situación en la que se podría utilizar esto y explique cómo.

35. Las mediciones de cierto sistema han demostrado que el proceso promedio se ejecuta durante un tiempo T antes de bloquearse debido a una operación de E/S. Una conmutación de procesos requiere de un tiempo S, es efectivamente desperdiciado (sobrecarga). Para la planificación por turno circular con un quántum Q, proporcione una fórmula para la eficiencia de la CPU en cada uno de los siguientes casos:

www.FreeLibros.me

(a) $Q = \infty$
(b) $Q > T$
(c) $S < Q < T$
(d) $Q = S$
(e) Q cercano a 0

36. Hay cinco trabajos en espera de ser ejecutados. Sus tiempos de ejecución esperados son 9, 6, 3, 5 y X. ¿En qué orden se deben ejecutar para minimizar el tiempo de respuesta promedio? (Su respuesta dependerá de X).

37. Cinco trabajos de procesamiento por lotes, A a E, llegan a un centro de cómputo casi al mismo tiempo. Tienen tiempos de ejecución estimados de 10, 6, 2, 4 y 8 minutos. Sus prioridades (determinadas en forma externa) son 3, 5, 2, 1 y 4, respectivamente, en donde 5 es la prioridad más alta. Para cada uno de los siguientes algoritmos de planificación, determine el tiempo de respuesta de proceso promedio. Ignore la sobrecarga por conmutación de procesos.

a) Por turno circular.
b) Por prioridad.
c) Primero en entrar, primero en ser atendido (ejecutados en el orden 10, 6, 2, 4, 8).
d) El trabajo más corto primero.

Para (a), suponga que el sistema es multiprogramado y que cada trabajo recibe su parte equitativa de la CPU. Para los incisos del (b) al (d), suponga que sólo se ejecuta un trabajo a la vez hasta que termina. Todos los trabajos están completamente ligados a la CPU.

38. Un proceso que se ejecuta en un CTSS necesita 30 quántums para completarse. ¿Cuántas veces se debe intercambiar, incluyendo la primera vez (antes de que se haya ejecutado siquiera)?

39. ¿Puede idear una forma de evitar que el sistema de prioridades CTSS sea engañado por los retornos de carro?

40. El algoritmo de envejecimiento con $a = 1/2$ se utiliza para predecir los tiempos de ejecución. Las cuatro ejecuciones anteriores, de la más antigua a la más reciente, son de 40, 20, 40 y 15 mseg. ¿Cuál es la predicción del siguiente tiempo?

41. Un sistema de tiempo real suave tiene cuatro eventos periódicos con periodos de 50, 100, 200 y 250 mseg cada uno. Suponga que los cuatro eventos requieren 35, 20, 10 y x mseg de tiempo de la CPU, respectivamente. ¿Cuál es el mayor valor de x para que el sistema sea planificable?

42. Explique por qué se utiliza comúnmente la planificación de dos niveles.

43. Un sistema de tiempo real necesita manejar dos llamadas de voz, en donde cada una de ellas se ejecuta cada 5 mseg y consume 1 mseg de tiempo de la CPU por ráfaga, más un video a 25 cuadros/segundo, en donde cada cuadro requiere 20 mseg de tiempo de la CPU. ¿Es planificable este sistema?

44. Considere un sistema en el que se desean separar la política y el mecanismo para la planificación de hilos de kernel. Proponga una manera de lograr este objetivo.

45. En la solución al problema de los filósofos comelones (figura 2-46), ¿por qué la variable de estado se establece a *HAMBRIENTO* en el procedimiento *tomar_tenedores*?

www.FreeLibros.me

46. Considere el procedimiento *poner_tenedores* en la figura 2-46. Suponga que la variable *estado*[*i*] se estableció a *PENSANDO después* de las dos llamadas a *probar*, en vez de *antes*. ¿Cómo afectaría esto a la solución?

47. El problema de los lectores y escritores se puede formular de varias maneras respecto a cuál categoría de procesos se puede iniciar y cuándo. Describa con cuidado tres variaciones distintas del problema, cada una favoreciendo (o no favoreciendo) a alguna categoría de procesos. Para cada variación, especifique lo que ocurre cuando un lector o escritor está listo para acceder a la base de datos y qué ocurre cuando un proceso termina de usar la base de datos.

48. Escriba una secuencia de comandos de shell que produzca un archivo de números secuenciales, para lo cual debe leer el último número en el archivo, sumarle 1 y después adjuntar el resultado al archivo. Ejecute una instancia de la secuencia de comandos en segundo plano y una en primer plano, cada una de las cuales debe acceder al mismo archivo. ¿Cuánto tiempo pasa antes de que se manifieste una condición de carrera en sí? ¿Cuál es la región crítica? Modifique la secuencia de comandos para evitar la condición de carrera (*Sugerencia: utilice*

```
ln archivo archivo.lock
```

para bloquear el archivo de datos).

49. Suponga que tiene un sistema operativo que proporciona semáforos. Implemente un sistema de mensajes. Escriba los procedimientos para enviar y recibir mensajes.

50. Resuelva el problema de los filósofos comelones utilizando monitores en vez de semáforos.

51. Suponga que una universidad desea mostrar su rectitud política al aplicar la doctrina "Separado pero igual es inherentemente desigual" de la Suprema Corte de los Estados Unidos al género así como a la raza, finalizando con su vieja práctica de los baños segregados por el género en los campus. Sin embargo, como una concesión a la tradición, decreta que cuando haya una mujer en el baño, pueden entrar otras mujeres pero no hombres, y viceversa. Un signo con una marca deslizable en la puerta de cada baño indica en cuál de los tres estados se encuentra en un momento dado:

- Vacío
- Mujer presente
- Hombre presente

En algún lenguaje de programación de su preferencia, escriba los siguientes procedimientos: *mujer_desea_entrar*, *hombre_desea_entrar*, *mujer_sale*, *hombre_sale*. Puede usar los contadores y técnicas de sincronización que desee.

52. Vuelva a escribir el programa de la figura 2-23 para manejar más de dos procesos.

53. Escriba un problema productor-consumidor que utilice hilos y comparta un búfer común. Sin embargo, no utilice semáforos o cualquier otra primitiva de sincronización para proteger las estructuras de datos compartidas. Sólo deje que cada hilo las utilice cuando quiera. Use `sleep` y `wakeup` para manejar las condiciones de lleno y vacío. Vea cuánto tiempo se requiere para que ocurra una condición de carrera fatal. Por ejemplo, podría hacer que el productor imprima un número de vez en cuando. No imprima más de un número cada minuto, debido a que la E/S podría afectar a las condiciones de carrera.

www.FreeLibros.me

8.1 MULTIPROCESADORES

Un **multiprocesador con memoria compartida** (o sólo multiprocesador, de aquí en adelante) es un sistema de cómputo en el que dos o más CPUs comparten todo el acceso a una RAM común. Un programa que se ejecuta en cualquiera de las CPUs ve un espacio normal de direcciones virtuales (por lo general paginadas). La única propiedad inusual que tiene este sistema es que la CPU puede escribir cierto valor en una palabra de memoria y después puede volver a leer esa palabra y obtener un valor distinto (tal vez porque otra CPU lo cambió). Si se organiza en forma correcta, esta propiedad forma la base de la comunicación entre procesadores: una CPU escribe ciertos datos en la memoria y otra lee esos datos.

En su mayor parte, los sistemas operativos multiprocesadores son sólo sistemas operativos regulares. Manejan las llamadas al sistema, administran la memoria, proveen un sistema de archivos y administran los dispositivos de E/S. Sin embargo, hay ciertas áreas en las que tienen características únicas. Estas áreas son la sincronización de procesos, la administración de recursos y la programación de tareas. A continuación analizaremos con brevedad el hardware de los multiprocesadores y después pasaremos a ver las cuestiones relacionadas con estos sistemas operativos.

8.1.1 Hardware de multiprocesador

Aunque todos los multiprocesadores tienen la propiedad de que cada CPU puede direccionar toda la memoria, algunos tienen la característica adicional de que cada palabra de memoria se puede leer con la misma velocidad que cualquier otra palabra de memoria. Estas máquinas se conocen como multiprocesadores **UMA** (*Uniform Memory Access*, Acceso uniforme a la memoria). Por el contrario, los multiprocesadores **NUMA** (*Non-uniform Memory Access*, Acceso no uniforme a la memoria) no tienen esta propiedad. Más adelante aclararemos por qué existe esta diferencia. Primero examinaremos los multiprocesadores UMA y después los NUMA.

Multiprocesadores UMA con arquitecturas basadas en bus

Los multiprocesadores más simples se basan en un solo bus, como se ilustra en la figura 8-2(a). Dos o más CPUs y uno o más módulos de memoria utilizan el mismo bus para comunicarse. Cuando una CPU desea leer una palabra de memoria, primero comprueba si el bus está ocupado. Si está inactivo, la CPU coloca la dirección de la palabra que desea en el bus, declara unas cuantas señales de control y espera hasta que la memoria coloque la palabra deseada en el bus.

Si el bus está ocupado cuando una CPU desea leer o escribir en la memoria, la CPU espera sólo hasta que el bus esté inactivo. Aquí es donde se encuentra el problema con este diseño. Con dos o tres CPUs, la contención por el bus será manejable; con 32 o 64 será imposible. El sistema estará totalmente limitado por el ancho de banda del bus, y la mayoría de las CPUs estarán inactivas la mayor parte del tiempo.

La solución a este problema es agregar una caché a cada CPU, como se ilustra en la figura 8.2(b). La caché puede estar dentro del chip de la CPU, a un lado de ésta, en el tablero del procesador o puede ser alguna combinación de las tres opciones anteriores. Como esto permite satisfacer

www.FreeLibros.me

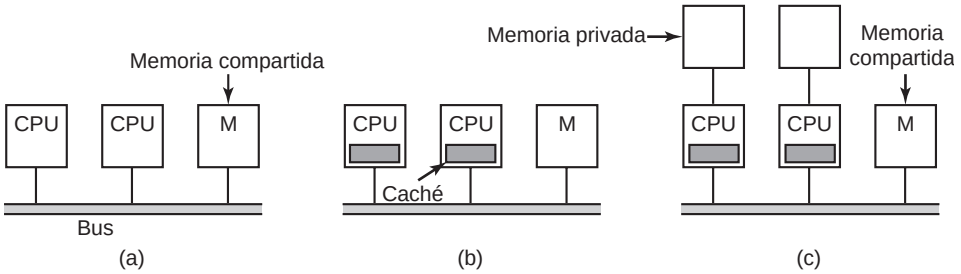


Figura 8-2. Tres multiprocesadores basados en bus. (a) Sin caché. (b) Con caché. (c) Con caché y memorias privadas.

muchas operaciones de lectura desde la caché local, habrá mucho menos tráfico en el bus y el sistema podrá soportar más CPUs. En general, el uso de caché no se realiza por cada palabra individual, sino por bloques de 32 o 64 bytes. Cuando se hace referencia a una palabra, se obtiene su bloque completo (conocido como **línea de caché**) y se coloca en la caché de la CPU que está en contacto con ella.

Cada bloque de la caché se marca como de sólo lectura (en cuyo caso puede estar presente en varias cachés al mismo tiempo) o como de lectura-escritura (no puede estar presente en otras cachés). Si una CPU trata de escribir una palabra que está en una o más cachés remotas, el hardware del bus detecta la escritura y coloca una señal en el bus para informar a todas las demás cachés sobre la operación de escritura. Si otras cachés tienen una copia "limpia" (es decir, una copia exacta de lo que hay en memoria), sólo tienen que descartar sus copias y dejar que la CPU que va a escribir obtenga el bloque de la caché de la memoria antes de modificarla. Si alguna otra caché tiene una copia "sucia" (es decir, modificada), debe escribirla de vuelta a la memoria para que pueda continuar la escritura, o transferirla directamente a la CPU que va a realizar la operación de escritura a través del bus. A este conjunto de reglas se le conoce como **protocolo de coherencia de cachés** y es uno de muchos.

Otra posibilidad es el diseño de la figura 8-2(c), donde cada CPU tiene no sólo una caché, sino también una memoria privada local que utiliza mediante un bus dedicado (privado). Para utilizar esta configuración de manera óptima, el compilador debe colocar todo el texto del programa, las cadenas, constantes y demás datos de sólo lectura, las pilas y las variables locales en las memorias privadas. De esta forma, la memoria compartida se utiliza sólo para escribir variables compartidas. En la mayoría de los casos, esta cuidadosa colocación reducirá de manera considerable el tráfico en el bus, pero requiere de una cooperación activa por parte del compilador.

Multiprocesadores UMA que utilizan interruptores de barras cruzadas

Aun con la mejor caché, el uso de un solo bus limita el tamaño de un multiprocesador UMA a 16 o 32 CPUs, aproximadamente. Para lograr algo más se necesita un tipo distinto de interconexión. El circuito más simple para conectar *n* CPUs a *k* memorias es el **interruptor (conmutador) de barras cruzadas**, que se muestra en la figura 8-3. Los interruptores de barras cruzadas se han utilizado por décadas en los puntos de intercambio de los conmutadores telefónicos para conectar un grupo de líneas entrantes a un conjunto de líneas salientes de manera arbitraria.

www.FreeLibros.me

En cada intersección de una línea horizontal (entrante) y una vertical (saliente) hay un **punto de cruce**, el cual es un pequeño interruptor que puede estar abierto o cerrado en sentido eléctrico, dependiendo de si las líneas horizontal y vertical van a conectarse o no. En la figura 8-3(a) podemos ver tres puntos de cruce cerrados al mismo tiempo, con lo cual se permiten tres conexiones entre los pares (CPU, memoria) (010, 000), (101, 101) y (110, 010) al mismo tiempo. También son posibles muchas otras combinaciones. De hecho, el número de combinaciones es igual al número de las distintas formas en que se pueden colocar ocho torres en un tablero de ajedrez.

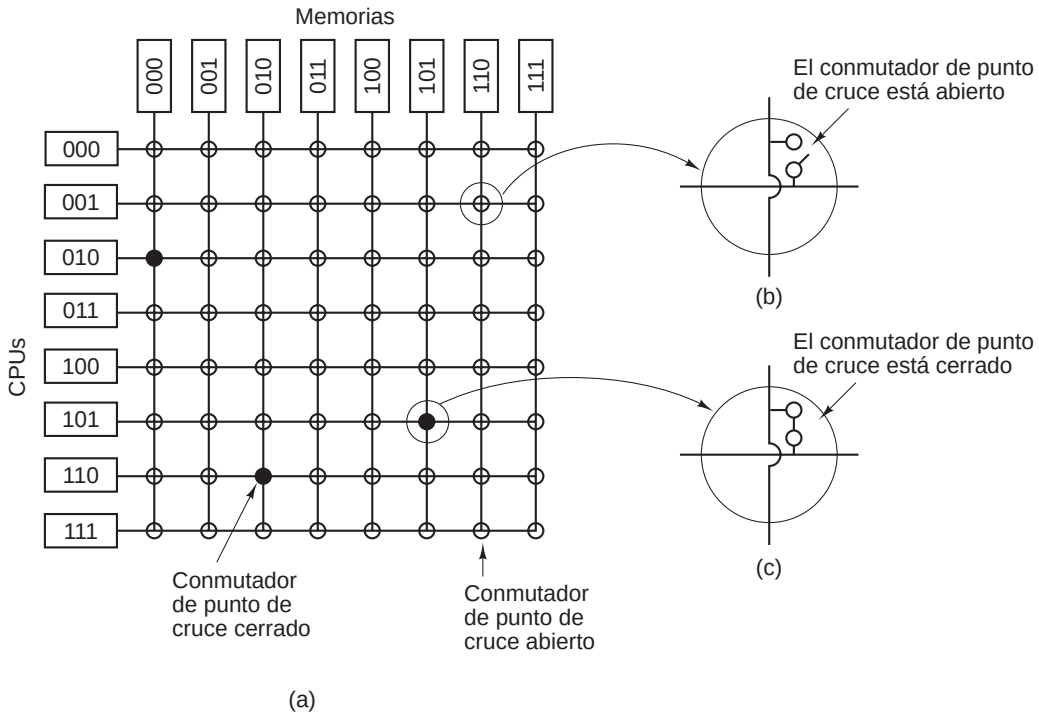


Figura 8-3. (a) Un conmutador de barras cruzadas de 8×8. (b) Un punto de cruce abierto. (c) Un punto de cruce cerrado.

Una de las propiedades más agradables de un interruptor de barras cruzadas es que es una **red sin bloqueos**, lo cual significa que a ninguna CPU se le niega nunca la conexión que necesita, debido a que algún punto de cruce o una línea ya estén ocupados (suponiendo que el módulo de memoria en sí esté disponible). Además, no se necesita una planeación por adelantado. Aun si ya hay siete conexiones arbitrarias establecidas, siempre es posible conectar la CPU restante a la memoria restante.

Desde luego que también es posible la contención de memoria, si dos CPUs desean acceder al mismo módulo, al mismo tiempo. Sin embargo, al particionar la memoria en *n* unidades, la contención se reduce por un factor de *n* en comparación con el modelo de la figura 8-2.

www.FreeLibros.me

Una de las peores propiedades del conmutador de barras cruzadas es el hecho de que el número de puntos de cruce aumenta con base en n^2. Con 1000 CPUs y 1000 módulos de memoria, necesitamos un millón de puntos de cruce. No es factible un interruptor de barras cruzadas de este tamaño. Sin embargo, para los sistemas de tamaño mediano, se puede utilizar el diseño de las barras cruzadas.

Multiprocesadores UMA que utilizan redes de conmutación multietapa

Un diseño completamente distinto de multiprocesador se basa en el humilde conmutador de 2×2 que se muestra en la figura 8-4(a). Este interruptor tiene dos entradas y dos salidas. Los mensajes que llegan en cualquiera de las líneas de entrada se pueden conmutar a cualquiera de las líneas de salida. Para nuestros fines, los mensajes deben contener hasta cuatro partes, como se muestra en la figura 8-4(b). El campo *Módulo* indica qué memoria utilizar. *Dirección* especifica una dirección dentro de un módulo. *CódigoOp* proporciona la operación, como `READ` o `WRITE`. Por último, el campo *Valor* opcional puede contener un operando, como una palabra de 32 bits a escribir mediante una operación `WRITE`. El interruptor inspecciona el campo *Módulo* y lo utiliza para determinar si el mensaje se debe enviar en *X* o en *Y*.

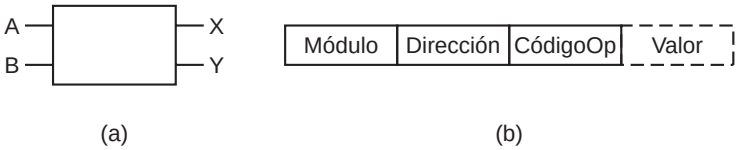


Figura 8-4. (a) Un conmutador de 2×2 con dos líneas de entrada (*A* y *B*) y dos líneas de salida (*X* y *Y*). (b) Un formato de mensaje.

Nuestros conmutadores de 2×2 se pueden organizar de muchas formas para construir **redes de conmutación multietapa** más grandes (Adams y colaboradores, 1987; Bhuyan y colaboradores, 1989; Kumar y Reddy, 1987). Una posibilidad es la **red omega** de clase económica sin adornos, que se ilustra en la figura 8-5. Aquí hemos conectado ocho CPUs a ocho memorias, utilizando 12 conmutadores. De manera más general, para n CPUs y n memorias necesitaríamos $\log_2 n$ etapas, con $n/2$ conmutadores por etapa, para un total de $(n/2)\log_2 n$ conmutadores, que es mucho mejor que n^2 puntos de cruce, en especial para valores grandes de n.

El patrón de cableado de la red omega se conoce comúnmente como **barajado perfecto**, ya que la mezcla de las señales en cada etapa se asemeja al proceso de cortar a la mitad un mazo de cartas y después mezclarlas, carta por carta. Para ver cómo funciona la red omega, suponga que la CPU 011 desea leer una palabra del módulo de memoria 110. La CPU envía un mensaje `READ` al interruptor 1D que contiene el valor 110 en el campo *Módulo*. El interruptor toma el primer bit (de más a la izquierda) de 110 y lo utiliza para el enrutamiento. Un 0 enruta a la salida superior y un 1 a la inferior. Como este bit es un 1, el mensaje se enruta a través de la salida inferior hacia 2D.

Todos los interruptores de la segunda etapa (incluyendo a 2D) utilizan el segundo bit para el enrutamiento. Éste también es un 1, por lo que ahora el mensaje se reenvía a través de la salida inferior hacia 3D. Aquí se evalúa el tercer bit y se encuentra que es 0. En consecuencia, el mensaje

www.FreeLibros.me

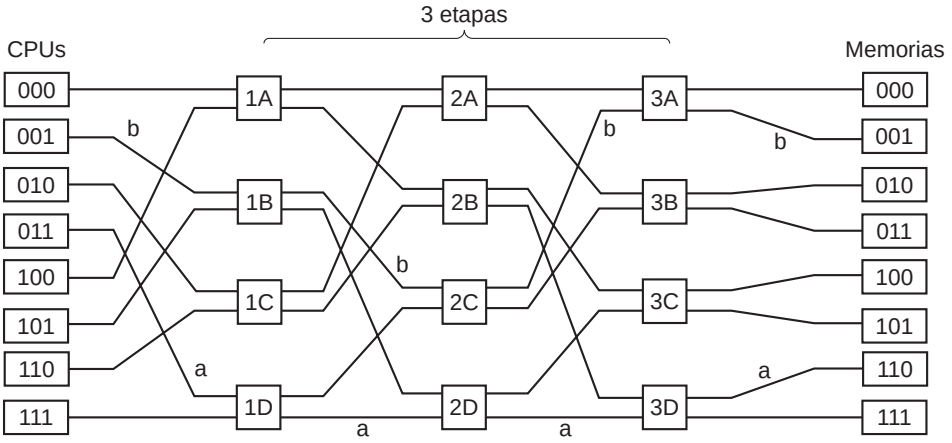


Figura 8-5. Una red de conmutación omega.

pasa por la salida superior y llega a la memoria 110, como se desea. La ruta seguida por este mensaje está marcada en la figura 8-5 por la letra *a*.

A medida que el mensaje pasa a través de la red de conmutación, los bits en el extremo izquierdo del número de módulo ya no se necesitan. Se les puede dar un buen uso al registrar ahí el número de línea entrante, para que la respuesta pueda encontrar su camino de regreso. Para la ruta *a*, las líneas entrantes son 0 (entrada superior a 1D), 1 (entrada inferior a 2D) y 1 (entrada inferior a 3D), respectivamente. La respuesta se enruta de vuelta utilizando 011, sólo que esta vez se lee de derecha a izquierda.

Al mismo tiempo que ocurre todo esto, la CPU 001 desea escribir una palabra en el módulo de memoria 001. Aquí ocurre un proceso análogo, donde el mensaje se enruta a través de las salidas superior, superior e inferior, respectivamente, marcadas por la letra *b*. Cuando llega, en su campo *Módulo* se lee 001, lo cual representa la ruta que tomó. Como estas dos peticiones no utilizan ninguno de los mismos conmutadores, líneas o módulos de memoria, pueden proceder en paralelo.

Ahora considere lo que ocurriría si la CPU 000 quisiera acceder al mismo tiempo al módulo de memoria 000. Su petición entraría en conflicto con la petición de la CPU 001 en el interruptor 3A. Una de ellas tendría que esperar. A diferencia del conmutador de barras cruzadas, la red omega es una **red con bloqueo**. No todos los conjuntos de peticiones pueden procesarse al mismo tiempo. Puede haber conflictos sobre el uso de un cable o de un conmutador, así como entre las peticiones *a* la memoria y las respuestas *de* la memoria.

Sin duda, es conveniente esparcir las referencias a memoria de manera uniforme a través de los módulos. Una técnica común es utilizar los bits de menor orden como el número de módulo. Por ejemplo, considere un espacio de direcciones orientado a bytes para una computadora que por lo general utiliza palabras completas de 32 bits. Los 2 bits de menor orden por lo general serán 00, pero los siguientes 3 bits estarán distribuidos de manera uniforme. Al utilizar estos 3 bits como el número de módulo, las palabras consecutivas estarán en módulos consecutivos. Se dice que un sistema de memoria en el que las palabras consecutivas están en distintos módulos es **entrelazado**. Las memorias entrelazadas maximizan el paralelismo, debido a que la mayoría de las referencias a memo-

www.FreeLibros.me

ria son a direcciones consecutivas. También es posible diseñar redes de conmutación que sean sin bloqueo y que ofrezcan varias rutas desde cada CPU a cada módulo de memoria, para esparcir mejor el tráfico.

Multiprocesadores NUMA

Los multiprocesadores UMA de un solo bus por lo general están limitados a no más de unas cuantas docenas de CPUs, y los multiprocesadores de barras cruzadas o de conmutadores necesitan mucho hardware (costoso) y no son tan grandes. Para poder tener más de 100 CPUs, hay que ceder algo. Por lo general, lo que cede es la idea de que todos los módulos de memoria tienen el mismo tiempo de acceso. Esta concesión nos lleva a la idea de los multiprocesadores NUMA, como dijimos antes. Al igual que sus parientes UMA, proveen un solo espacio de direcciones a través de todas las CPUs, pero a diferencia de las máquinas UMA, el acceso a los módulos de memoria locales es más rápido que el acceso a los remotos. Por ende, todos los programas UMA se ejecutarán sin cambios en las máquinas NUMA, pero el rendimiento será peor que en una máquina UMA a la misma velocidad de reloj.

Las máquinas NUMA poseen tres características clave que, en conjunto, las distinguen de otros multiprocesadores:

1. Hay un solo espacio de direcciones visible para todas las CPUs.

2. El acceso a la memoria remota es mediante instrucciones `LOAD` y `STORE`.

3. El acceso a la memoria remota es más lento que el acceso a la memoria local.

Cuando el tiempo de acceso a la memoria remota no está oculto (debido a que no hay caché), al sistema se le llama **NC-NUMA** (*No Cache NUMA*, NUMA sin caché). Cuando hay cachés coherentes presentes, al sistema se le llama **CC-NUMA** (*Cache-Coherent NUMA*, NUMA con cachés coherentes).

El método más popular para construir grandes multiprocesadores CC-NUMA en la actualidad es el **multiprocesador basado en directorios**. La idea es mantener una base de datos que indique dónde está cada línea de caché y cuál es su estado. Cuando se hace referencia a una línea de caché, se consulta la base de datos para averiguar dónde está, y si está limpia o sucia (modificada). Como esta base de datos se debe consultar en cada instrucción que hace referencia a la memoria, debe mantenerse en un hardware de propósito especial, en extremo veloz, que pueda responder en una fracción de un ciclo del bus.

Para que la idea sobre un multiprocesador basado en directorios sea algo más concreta, vamos a considerar un ejemplo simple (hipotético): un sistema con 256 nodos donde cada nodo consista en una CPU y 16 MB de RAM conectados a la CPU a través de un bus local. La memoria total es de 2^{32} bytes, que se divide en 2^{26} líneas de caché de 64 bytes cada una. La memoria se asigna en forma estática entre los nodos, con 0 a 16M en el nodo 0, 16M a 32M en el nodo 1, y así en lo sucesivo. Los nodos se conectan mediante una red de interconexión, como se muestra en la figura 8-6(a). Cada nodo también contiene las entradas de directorio para las 2^{18} líneas de caché de 64 bytes que

www.FreeLibros.me

conforman su memoria de 2^{24} bytes. Por el momento, vamos a suponer que una línea se puede contener a lo más en una caché.

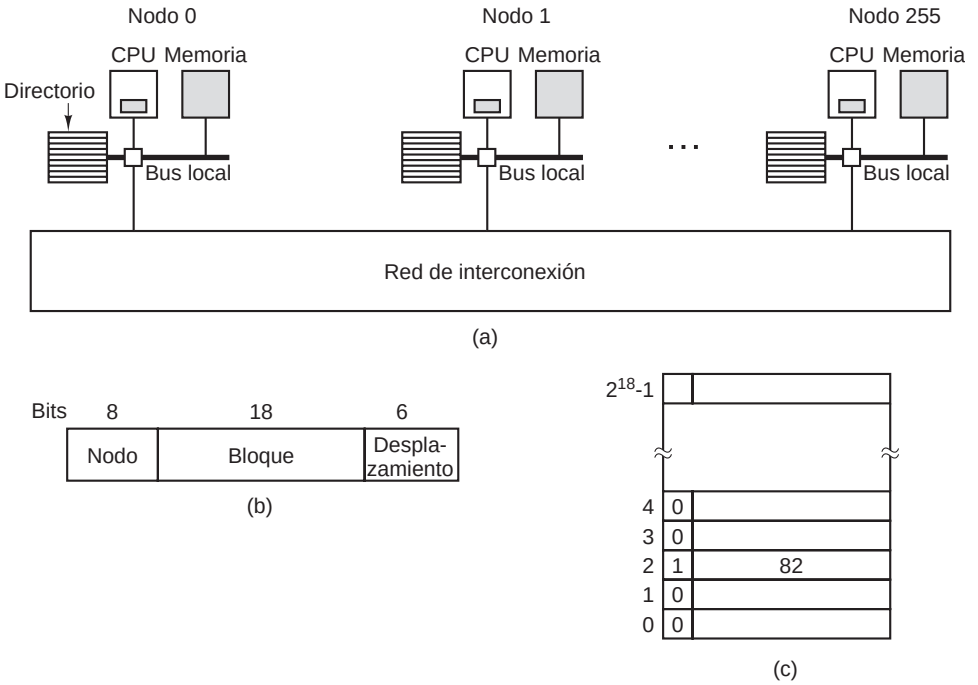


Figura 8.6. (a) Un multiprocesador basado en directorio de 256 nodos. (b) División de una dirección de memoria de 32 bits en campos. (c) El directorio en el nodo 36.

Para ver cómo funciona el directorio, vamos a rastrear una instrucción `LOAD` desde la CPU 20 que hace referencia a una línea en la caché. Primero, la CPU que emite la instrucción la presenta a su MMU, que la traduce en una dirección física; por ejemplo, 0x24000108. La MMU divide esta dirección en las tres partes que se muestran en la figura 8-6(b). En decimal, las tres partes son el nodo 36, la línea 4 y el desplazamiento 8. La MMU ve que la palabra de memoria a la que se hace referencia es del nodo 36, no del nodo 20, por lo que envía un mensaje de petición a través de la red de interconexión al nodo de inicio de la línea (36), en donde le pregunta si su línea 4 está en la caché, y de ser así, en dónde está.

Cuando la petición llega al nodo 36 a través de la red de interconexión, se enruta al hardware de directorio. El hardware la indexa en su tabla de 2^{18} entradas, una para cada una de sus líneas de caché, y extrae la entrada 4. De la figura 8-6(c) podemos ver que la línea no está en la caché, por lo que el hardware obtiene la línea 4 de la RAM local, la envía de vuelta al nodo 20 y actualiza la entrada 4 del directorio para indicar que la línea ahora está en caché en el nodo 20.

Ahora consideremos una segunda petición, esta vez preguntando sobre la línea 2 del nodo 36. En la figura 8-6(c) podemos ver que esta línea está en la caché en el nodo 82. En este punto, el hard-

www.FreeLibros.me

ware podría actualizar la entrada 2 del directorio para decir que la línea se encuentra ahora en el nodo 20, y después enviar un mensaje al nodo 82 para indicarle que debe pasar la línea al nodo 20 e invalidar su caché. Observe que hasta un "multiprocesador con memoria compartida" realiza muchas operaciones de paso de mensajes de manera interna.

Como una observación rápida adicional, vamos a calcular cuánta memoria está siendo ocupada por los directorios. Cada nodo tiene 16 MB de RAM y 2^{18} entradas de 9 bits para llevar la cuenta de esa RAM. Por ende, la sobrecarga del directorio es de aproximadamente 9×2^{18} bits, dividida entre 16 MB o aproximadamente de 1.76%, que por lo general es aceptable (aunque tiene que ser memoria de alta velocidad, lo que desde luego incrementa su costo). Aun con las líneas de caché de 32 bytes, la sobrecarga sería sólo de 4%. Con líneas de caché de 128 bytes, sería menor de 1%.

Una limitación obvia de este diseño es que una línea se puede colocar en caché sólo en un nodo. Para permitir que se coloquen líneas en caché en varios nodos, necesitaríamos alguna forma de localizarlos a todos, por ejemplo para invalidarlos o actualizarlos en una escritura. Hay varias opciones posibles para permitir el uso de caché en varios nodos al mismo tiempo, pero un análisis de estas opciones es algo que está más allá del alcance de este libro.

Chips multinúcleo

A medida que mejora la tecnología de fabricación de chips, los transistores se están haciendo cada vez más pequeños y es posible colocar cada vez más de ellos en un chip. A esta observación empírica se le conoce algunas veces como **Ley de Moore**, en honor del co-fundador de Intel Gordon Moore, quien la descubrió primero. Los chips en la clase del Intel Core 2 Duo contienen cerca de 300 millones de transistores.

Una pregunta obvia es: "¿Qué se debe hacer con todos esos transistores?". Como vimos en la sección 1.3.1, una opción es agregar megabytes de caché al chip. Esta opción es seria, y los chips con 4 MB de caché fuera del chip ya son comunes, con cachés más grandes en camino. Pero en cierto punto, incrementar el tamaño de la caché tal vez sólo eleve la tasa de aciertos de 99% a 99.5%, lo cual no mejora mucho el rendimiento de las aplicaciones.

La otra opción es colocar dos o más CPUs completas, a las que generalmente se les denomina **núcleos**, en el mismo chip (técnicamente, en la misma **pastilla**). Los chips de doble núcleo y de cuádruple núcleo ya son comunes; se han fabricado chips con 80 núcleos, y los chips con cientos de núcleos ya están en el horizonte.

Aunque las CPUs pueden o no compartir cachés (por ejemplo, vea la figura 1-8), siempre comparten la memoria principal, y ésta es consistente en el sentido de que siempre hay un valor único para cada palabra de memoria. Los circuitos de hardware especiales aseguran que si hay una palabra presente en dos o más cachés, y una de las CPUs modifica la palabra, ésta se remueva en forma automática y atómica de todas las cachés para poder mantener la consistencia. A este proceso se le conoce como ***snooping***.

El resultado de este diseño es que los chips multinúcleo son sólo pequeños multiprocesadores. De hecho, a los chips multinúcleo se les conoce algunas veces como **CMP**s (**Multiprocesadores a nivel de chip**). Desde la perspectiva del software, los CMPs no son en realidad tan distintos de los multiprocesadores basados en bus, o de los multiprocesadores que utilizan redes de conmutación.

www.FreeLibros.me

Sin embargo, hay algunas diferencias. Para empezar, en un multiprocesador basado en bus cada CPU tiene su propia caché, como en la figura 8-2(b) y también como en el diseño AMD de la figura 1-8(b). El diseño de caché compartida de la figura 1-8(a), que Intel utiliza, no ocurre en otros multiprocesadores. La caché L2 compartida puede afectar el rendimiento. Si un núcleo necesita mucha memoria caché y los otros no, este diseño permite que ese núcleo tome lo que necesite. Por otro lado, la caché compartida también hace posible que un núcleo avaricioso dañe el rendimiento de los demás núcleos.

Otra área en la que los CMPs difieren de sus primos más grandes es la tolerancia a fallas. Como las CPUs tienen una conexión tan estrecha, las fallas en los componentes compartidos pueden afectar a varias CPUs a la vez, algo que es menos probable en los multiprocesadores tradicionales.

Además de los chips multinúcleo simétricos, en donde todos los núcleos son idénticos, hay otra categoría de chip multinúcleo conocida como **sistema en un chip**. Estos chips tienen una o más CPUs principales, pero también tienen núcleos de propósito especial, como decodificadores de video y audio, criptoprocesadores, interfaces de red y demás, con lo cual se obtiene un sistema de cómputo completo en un chip.

Como ha ocurrido con frecuencia en el pasado, el hardware está muy adelantado al software. Aunque los chips multinúcleo ya están aquí, nuestra habilidad para escribir aplicaciones para ellos todavía no. Los lenguajes de programación actuales no están preparados para escribir programas altamente paralelos, y los buenos compiladores y herramientas de depuración son escasos. Pocos programadores han tenido experiencia con la programación en paralelo y la mayoría saben poco acerca de cómo dividir el trabajo en varios paquetes que se puedan ejecutar en paralelo. La sincronización, la eliminación de las condiciones de carrera y la prevención del interbloqueo van a ser pesadillas, y el rendimiento sufrirá de manera considerable como resultado. Los semáforos no son la respuesta. Y más allá de estos problemas iniciales, está muy lejos de ser obvio qué tipo de aplicación realmente necesita cientos de núcleos. El reconocimiento de voz en lenguaje natural probablemente podría absorber mucho poder de cómputo, pero el problema aquí no es la falta de ciclos, sino de algoritmos que funcionen. En resumen, los diseñadores de hardware pueden estar ofreciendo un producto que los diseñadores de software no saben cómo usar, y que los usuarios no quieren.

8.1.2 Tipos de sistemas operativos multiprocesador

Ahora vamos a pasar del hardware de multiprocesador al software de multiprocesador, y en especial a los sistemas operativos multiprocesador. Hay varias metodologías posibles. A continuación estudiaremos tres de ellas. Observe que todas se pueden aplicar de igual forma a los sistemas multinúcleo, así como a los sistemas con CPUs discretas.

Cada CPU tiene su propio sistema operativo

La manera más simple posible de organizar un sistema operativo multiprocesador es dividir estáticamente la memoria y su propia copia privada del sistema operativo. En efecto, las *n* CPUs operan entonces como *n* computadoras independientes. Una optimización obvia es permitir que todas las

www.FreeLibros.me

CPUs compartan el código del sistema operativo y obtengan copias privadas sólo de las estructuras de datos del sistema operativo, como se muestra en la figura 8-7.

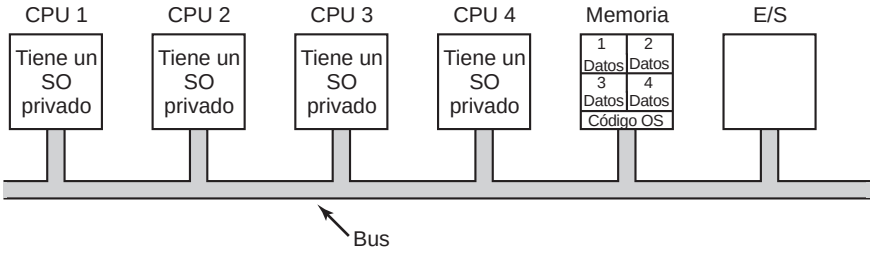


Figura 8-7. Particionamiento de la memoria de un multiprocesador entre cuatro CPUs, pero compartiendo una sola copia del código del sistema operativo. Los cuadros marcados como Datos son los datos privados del sistema operativo para cada CPU.

Este esquema es aún mejor que tener *n* computadoras separadas, ya que permite que todas las máquinas compartan un conjunto de discos y otros dispositivos de E/S, y también permite compartir la memoria con flexibilidad. Por ejemplo, aun con la asignación estática de memoria, una CPU puede recibir una porción extra-grande de la memoria para poder manejar con eficiencia los programas grandes. Además, los procesos pueden comunicarse de manera eficiente unos con otros, al permitir que un productor escriba los datos directamente en la memoria y que un consumidor los obtenga del lugar en el que los escribió el productor. Aún así, desde la perspectiva de un sistema operativo, el que cada CPU tenga su propio sistema operativo es lo más primitivo posible.

Vale la pena mencionar cuatro aspectos de este diseño que tal vez no sean obvios. En primer lugar, cuando un proceso realiza una llamada al sistema, ésta se atrapa y maneja en su propia CPU, usando las estructuras de datos en las tablas de ese sistema operativo.

En segundo lugar, como cada sistema operativo tiene sus propias tablas, también tiene su propio conjunto de procesos que programa por su cuenta. No hay compartición de procesos. Si un usuario se conecta a la CPU 1, todos sus procesos se ejecutan en la CPU 1. Como consecuencia, puede ocurrir que la CPU 1 esté inactiva mientras la CPU 2 está cargada de trabajo.

En tercer lugar, no hay compartición de páginas. Puede ocurrir que la CPU 1 tenga páginas de sobra, mientras que la CPU 2 esté paginando en forma continua. No hay forma de que la CPU 2 pida prestadas algunas páginas a la CPU 1, ya que la asignación de memoria es fija.

En cuarto lugar (y lo que es peor), si el sistema operativo mantiene una caché en búfer de los bloques de disco de uso reciente, cada sistema operativo hace esto de manera independiente a los demás. Por ende, puede ocurrir que haya cierto bloque de disco presente y sucio en varias cachés de búfer al mismo tiempo, con lo cual se producirán resultados inconsistentes. La única forma de evitar este problema es eliminar las cachés del búfer. Eso no es tan difícil, pero daña el rendimiento en forma considerable.

Por estas razones, es muy raro que se utilice este modelo, aunque en los primeros días de los multiprocesadores sí se utilizaba, cuando el objetivo era portar los sistemas operativos existentes hacia algún multiprocesador nuevo, lo más rápido posible.

www.FreeLibros.me

Multiprocesadores maestro-esclavo

En la figura 8-8 se muestra un segundo modelo. Aquí, hay una copia del sistema operativo y sus tablas presente en la CPU 1, y nada más ahí. Todas las llamadas al sistema se redirigen a la CPU 1 para procesarlas ahí. La CPU 1 también puede ejecutar proceso de usuario si tiene tiempo de sobra. A este modelo se le conoce como **maestro-esclavo**, ya que la CPU 1 es el maestro y todas los demás son los esclavos.

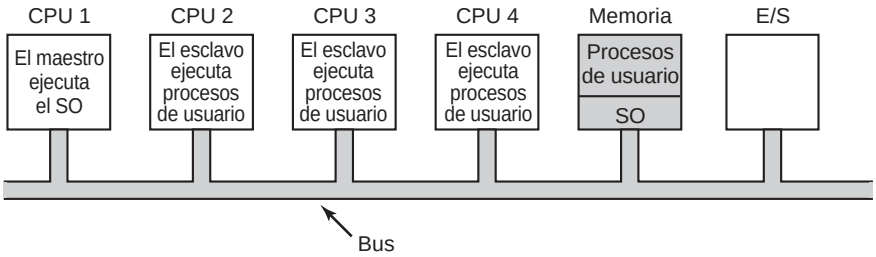


Figura 8.8. Un modelo de multiprocesador maestro-esclavo.

El modelo maestro-esclavo resuelve la mayoría de los problemas del primer modelo. Hay una sola estructura de datos (por ejemplo, una lista o un conjunto de listas con prioridades) que lleva la cuenta de los procesos listos. Cuando una CPU está inactiva, pide al sistema operativo en la CPU 1 un proceso para ejecutarlo, y se le asigna uno. Por ende, nunca puede ocurrir que una CPU esté inactiva mientras que otra esté sobrecargada. De manera similar, se pueden asignar las páginas entre todos los procesos en forma dinámica y sólo hay una caché de búfer, por lo que nunca ocurren inconsistencias.

El problema con este modelo es que con muchas CPUs, el maestro se convertirá en un cuello de botella. Después de todo, debe manejar todas las llamadas al sistema de todas las CPUs. Si, por ejemplo, el 10% del tiempo está manejando llamadas al sistema, entonces con 10 CPUs el maestro casi se saturará y con 20 CPUs estará completamente sobrecargado. Por ende, este modelo es simple y funciona para pequeños multiprocesadores, pero no para los grandes.

Multiprocesadores simétricos

Nuestro tercer modelo, el **SMP** (**Multiprocesador simétrico**), elimina esta asimetría. Hay una copia del sistema operativo en memoria, pero cualquier CPU puede ejecutarlo. Cuando se hace una llamada al sistema, la CPU en la que se hizo la llamada al sistema atrapa para el kernel y procesa la llamada al sistema. El modelo SMP se ilustra en la figura 8-9.

Este modelo equilibra los procesos y la memoria en forma dinámica, ya que sólo hay un conjunto de tablas del sistema operativo. También elimina el cuello de botella de la CPU, ya que no hay maestro, pero introduce sus propios problemas. En especial, si dos o más CPUs están ejecutando código del sistema operativo al mismo tiempo, se puede producir un desastre. Imagine que dos CPUs seleccionan el mismo proceso para ejecutarlo, o que reclaman la misma página de memoria

www.FreeLibros.me

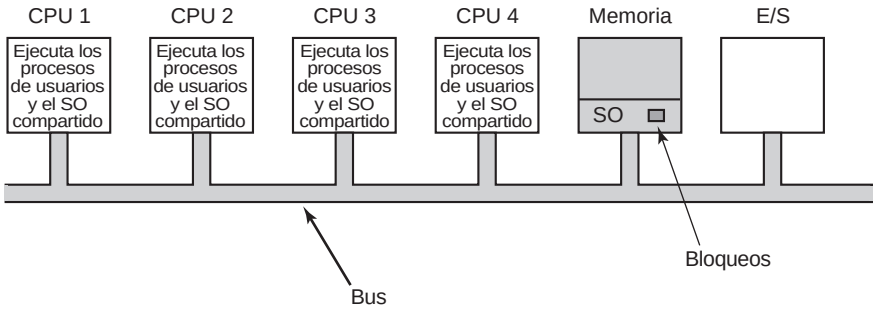


Figura 8-9. El modelo de multiprocesador SMP.

libre. La solución más simple para estos problemas es asociar un mutex (es decir, bloqueo) con el sistema operativo, convirtiendo a todo el sistema en una gran región crítica. Cuando una CPU desea ejecutar código del sistema operativo, debe adquirir primero el mutex. Si el mutex está bloqueado, sólo espera. De esta forma, cualquier CPU puede ejecutar el sistema operativo, pero sólo una a la vez.

Este modelo funciona, pero casi tan mal como el modelo maestro-esclavo. De nuevo, suponga que 10% de todo el tiempo de ejecución se invierte dentro del sistema operativo. Con 20 CPUs, habrá largas colas de CPUs esperando entrar. Por fortuna, es fácil de mejorar. Muchas partes del sistema operativo son independientes unas de otras. Por ejemplo, no habría problema si una CPU ejecutara el planificador mientras que otra CPU manejara una llamada al sistema de archivos, y una tercera CPU estuviera procesando un fallo de página.

Esta observación ocasiona que se divida el sistema operativo en varias regiones críticas independientes que no interactúan entre sí. Cada región crítica está protegida por su propio mutex, por lo que sólo una CPU a la vez puede ejecutarlo. De esta forma se puede lograr mucho más paralelismo. Sin embargo, puede ocurrir que algunas tablas (como las de procesos) sean utilizadas por varias regiones críticas. Por ejemplo, la tabla de procesos es necesaria para la programación, pero también para la llamada al sistema `fork` y para el manejo de señales. Cada tabla que pueda ser utilizada por varias regiones críticas necesita su propio mutex. De esta forma, cada región crítica se puede ejecutar sólo por una CPU a la vez, y cada tabla crítica se puede utilizar sólo por una CPU a la vez.

La mayoría de los multiprocesadores modernos utilizan esta organización. La parte difícil sobre escribir el sistema operativo para una máquina de este tipo no es que el código actual sea tan distinto de un sistema operativo regular. No es así. La parte difícil es dividirlo en regiones críticas que se puedan ejecutar de manera concurrente por distintas CPUs sin interferir unas con otras, ni siquiera en formas sutiles e indirectas. Además, cada tabla utilizada por dos o más regiones críticas debe estar protegida por separado por un mutex, y todo el código que utilice la tabla debe utilizar el mutex en forma correcta.

Además, hay que tener mucho cuidado para evitar los interbloqueos. Si dos regiones críticas necesitan la tabla *A* y la tabla *B*, y una de ellas reclama la tabla *A* primero y la otra reclama la tabla *B* primero, tarde o temprano ocurrirá un interbloqueo y nadie sabrá por qué. En teoría, a todas las

www.FreeLibros.me

tablas se les podrían asignar valores enteros y todas las regiones críticas podrían requerir la adquisición de tablas en orden ascendente. Esta estrategia evita los interbloqueos, pero requiere que el programador piense con mucho cuidado sobre cuáles tablas necesita cada región crítica, y realice las peticiones en el orden correcto.

A medida que el código evoluciona con el tiempo, una región crítica puede llegar a necesitar una nueva tabla que no necesitaba antes. Si el programador es nuevo y no comprende la lógica completa del sistema, entonces la tentación será sólo colocar el mutex en la tabla, en el punto en el que se necesite y liberarlo cuando ya no sea necesario. Sin importar qué tan razonable pueda parecer esto, se pueden producir interbloqueos, que el usuario percibirá como si el sistema se paralizara. No es fácil hacerlo bien, y mantenerlo así durante un periodo de años frente a los cambios de programadores es muy difícil.

8.1.3 Sincronización de multiprocesadores

Con frecuencia, las CPUs en un multiprocesador necesitan sincronizarse. Acabamos de ver el caso en el que las regiones críticas del kernel y las tablas se tienen que proteger mediante mutexes. Ahora vamos a analizar con más detalle la forma en que funciona esta sincronización en un multiprocesador. No carece de importancia, como pronto veremos.

Para empezar, realmente se necesitan primitivas de sincronización apropiadas. Si un proceso en una máquina uniprocesador (sólo una CPU) realiza una llamada al sistema que requiera acceder a cierta tabla crítica del kernel, el código del kernel sólo tiene que deshabilitar las interrupciones antes de tocar la tabla. Después puede hacer su trabajo, sabiendo que podrá terminar sin que ningún otro proceso se entrometa y toque la tabla antes de que haya terminado. En un sistema multiprocesador, deshabilitar las interrupciones sólo afecta a esa CPU, desshilitándola. Las demás CPUs se siguen ejecutando y aún pueden tocar la tabla crítica. Como consecuencia, se debe utilizar un protocolo de mutex apropiado, el cual debe ser respetado por todas las CPUs para garantizar que funcione la exclusión mutua.

El corazón de cualquier protocolo de mutex práctico es una instrucción especial que permite inspeccionar una palabra de memoria y establecerla en una operación indivisible. En la figura 2-22 vimos cómo se utilizaba `TSL` (Probar y establecer bloqueo) para implementar regiones críticas. Como vimos antes, lo que hace esta instrucción es leer una palabra de memoria y almacenarla en un registro. Al mismo tiempo, escribe un 1 (o cualquier otro valor distinto de cero) en la palabra de memoria. Desde luego que se requieren dos ciclos de bus para realizar las operaciones de lectura y escritura de memoria. En un uniprocesador, mientras que la instrucción no se pueda descomponer a mitad de ejecución, `TSL` siempre funcionará como se espera.

Ahora piense en lo que podría ocurrir en un multiprocesador. En la figura 8-10 podemos ver la sincronización en el peor caso, en donde la palabra de memoria 1000, que se utiliza como bloqueo, en un principio es 0. En el paso 1, la CPU 1 lee la palabra y obtiene un 0. En el paso 2, antes de que la CPU 1 tenga la oportunidad de volver a escribir la palabra para que sea 1, la CPU 2 entra y también lee la palabra como un 0. En el paso 3, la CPU 1 escribe un 1 en la palabra. En el paso 4, la CPU 2 también escribe un 1 en la palabra. Ambas CPUs obtuvieron un 0 de la instrucción `TSL`, por lo que ambas tienen ahora acceso a la región crítica y falla la exclusión mutua.

www.FreeLibros.me

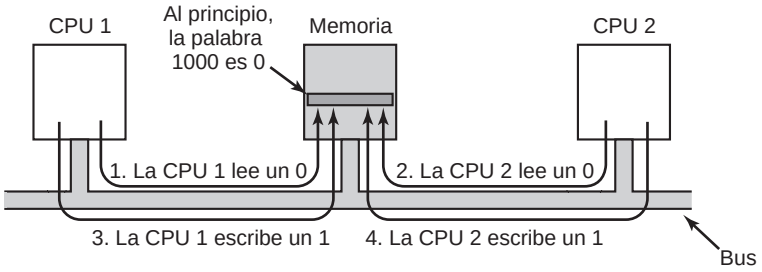


Figura 8-10. La instrucción TSL puede fallar si el bus no se puede bloquear. Estos cuatro pasos muestran una secuencia de eventos en donde se demuestra la falla.

Para evitar este problema, la instrucción TSL debe primero bloquear el bus, para evitar que otras CPUs lo utilicen; después debe realizar ambos accesos a la memoria, y luego desbloquear el bus. Por lo general, para bloquear el bus se hace una petición del bus usando el protocolo de petición de bus común, y después se declara (es decir, se establece a un 1 lógico) cierta línea de bus especial hasta que se hayan completado *ambos* ciclos. Mientras que esté declarada esta línea especial, ninguna otra CPU obtendrá acceso al bus. Esta instrucción puede implementarse sólo en un bus que tenga las líneas necesarias y el protocolo (de hardware) para utilizarlas. Los buses modernos tienen estas facilidades, pero en los primeros que no las tenían, no era posible implementar la instrucción TSL en forma correcta. Ésta es la razón por la que se inventó el protocolo de Peterson: para sincronizar por completo en el software (Peterson, 1981).

Si la instrucción TSL se implementa y utiliza en forma correcta, garantiza que la exclusión mutua pueda llegar a funcionar. Sin embargo, este método de exclusión mutua utiliza un **bloqueo de giro** debido a que la CPU que hace la petición sólo espera en un ciclo estrecho, evaluando el bloqueo lo más rápido que pueda. No sólo se desperdicia por completo el tiempo de la CPU (o CPUs) que hace la petición, sino que también se puede imponer una carga masiva en el bus o la memoria, reduciendo considerablemente a todas las demás CPUs que tratan de realizar su trabajo normal.

En primera instancia, podría parecer que la presencia de la caché debería eliminar el problema de la contención del bus, pero no es así. En teoría, una vez que la CPU que hace la petición ha leído la palabra de bloqueo, debe obtener una copia en su caché. Mientras que ninguna otra CPU intente utilizar el bloqueo, la CPU que hace la petición debe ser capaz de salir de su caché. Cuando la CPU que posee el bloqueo escribe un 1 en la caché para liberarla, el protocolo de la caché invalida de manera automática todas las copias de ésta en las cachés remotas, con lo que se debe volver a obtener el valor correcto.

El problema es que las cachés operan en bloques de 32 o 64 bytes. Por lo general, la CPU que contiene el bloqueo necesita las palabras que lo rodean. Como la instrucción TSL es una escritura (ya que modifica el bloqueo), necesita acceso exclusivo al bloque de la caché que contiene el bloqueo. Por lo tanto, cada instrucción TSL invalida el bloqueo en la caché de la CPU que contiene el bloqueo y obtiene una copia privada y exclusiva para la CPU que hace la petición. Tan pronto como la CPU que contiene el bloqueo toca una palabra adyacente al mismo, el bloque de la caché se

www.FreeLibros.me

mueve a su máquina. En consecuencia, todo el bloque de la caché que contiene el bloqueo se transporta constantemente entre el propietario del bloqueo y el que solicita el bloqueo, generando aún más tráfico en el bus que las lecturas individuales en la palabra de bloqueo.

Si pudiéramos deshacernos de todas las escrituras inducidas por TSL en el lado que hace la petición, podríamos reducir el *thrashing* de caché en forma considerable. Este objetivo se puede lograr al hacer que la CPU que hace la petición realice primero una lectura pura, para ver si el bloqueo está libre. Sólo si el bloqueo parece estar libre es cuando se realiza una TSL para poder adquirirlo. El resultado de este pequeño cambio es que la mayoría de los sondeos son ahora lecturas en vez de escrituras. Si la CPU que contiene el bloqueo sólo lee las variables en el mismo bloque de caché, cada una de ellas puede tener una copia del bloque de la caché en modo de sólo lectura compartido, eliminando todas las transferencias del bloque de caché. Cuando por fin se libera el bloqueo, el propietario realiza una escritura, la cual requiere acceso exclusivo, con lo cual se invalidan todas las otras copias en las cachés remotas. En la siguiente lectura que realice la CPU que hace la petición, el bloque de la caché se volverá a cargar. Observe que si hay dos o más CPUs contendiendo por el mismo bloqueo, puede ocurrir que ambas vean que está libre al mismo tiempo, y ambas puedan llegar a realizar una operación TSL simultánea para adquirirlo. Sólo una de estas operaciones tendrá éxito, por lo que no hay condición de competencia aquí debido a que la verdadera adquisición la realiza la instrucción TSL, y esta instrucción es atómica. Detectar que el bloqueo está libre y tratar de obtenerlo de inmediato con una operación TSL no garantiza la obtención del mismo. Alguien más podría ganar, pero para que el algoritmo sea correcto, no importa quién lo obtenga. El éxito en la lectura pura es tan sólo una sugerencia de que sería un buen momento de adquirir el bloqueo, pero no constituye una garantía de que la adquisición tendrá éxito.

Otra forma de reducir el tráfico en el bus es mediante el uso del popular algoritmo de *backoff* exponencial binario de Ethernet (Anderson, 1990). En vez de sondear en forma continua como en la figura 2-22, se puede insertar un ciclo de retraso entre los sondeos. Al principio, el retraso es una instrucción. Si el bloqueo sigue ocupado, el retraso se duplica a dos instrucciones, después a cuatro instrucciones, y así hasta llegar a cierto valor máximo. Un máximo bajo ofrece una respuesta rápida cuando se libera el bloqueo, pero desperdicia más ciclos de bus en el *thrashing* de la caché. Un máximo alto reduce el *thrashing* de la caché a expensas de no observar que el bloque está libre con tanta rapidez. El algoritmo de *backoff* exponencial binario se puede utilizar con o sin las lecturas puras que van antes de la instrucción TSL.

Una idea mejor incluso es proporcionar a cada CPU que desea adquirir el mutex su propia variable de bloqueo privada para evaluarla, como se ilustra en la figura 8-11 (Mellor-Crummey y Scott, 1991). La variable debe residir en un bloque de caché no utilizado, para evitar conflictos. El algoritmo funciona al hacer que una CPU que no puede adquirir el bloqueo asigne una variable de bloqueo y se adjunte a sí misma al final de una lista de CPUs que esperan el bloqueo. Cuando la CPU que contiene el bloque sale de la región crítica, libera el bloqueo privado que la primera CPU en la lista está evaluando (en su propia caché). Después, esta CPU entra a la región crítica. Cuando termina, libera el bloqueo que está usando su sucesora, y así en lo sucesivo. Aunque el protocolo es algo complicado (para evitar que dos CPUs se adjunten a sí mismas al final de la lista al mismo tiempo), es eficiente y libre de inanición. Para obtener todos los detalles, los lectores deberán consultar el artículo.

www.FreeLibros.me

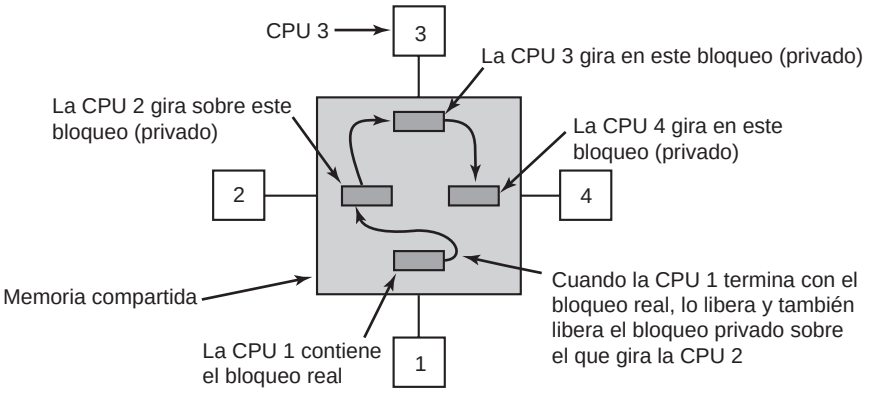


Figura 8-11. Uso de varios bloqueos para evitar el *thrashing* de la caché.

Comparación entre espera activa y conmutación

Hasta ahora hemos asumido que una CPU que necesita un mutex bloqueado sólo lo espera, ya sea mediante un sondeo continuo, un sondeo intermitente o adjuntándose a sí misma a una lista de CPUs en espera. Algunas veces no hay alternativa para que la CPU que hace la petición sólo espere. Por ejemplo, suponga que cierta CPU está inactiva y necesita acceder a la lista compartida de procesos listos para elegir un proceso y ejecutarlo. Si la lista de procesos listos está bloqueada, la CPU no puede sólo decidir suspender lo que está haciendo y ejecutar otro proceso, ya que para ello tendría que leer la lista de procesos listos. *Debe* esperar hasta que pueda adquirir esa lista.

Sin embargo, en otros casos hay una opción. Por ejemplo, si algún hilo en una CPU necesita acceso a la caché del búfer del sistema de archivos y actualmente está bloqueada, la CPU puede decidir cambiar a un hilo distinto en vez de esperar. La cuestión de si debe hacer una espera activa o realizar un cambio de hilo ha sido tema de mucha investigación, parte de la cual analizaremos a continuación. Observe que esta cuestión no ocurre en un uniprocesador, ya que no tiene mucho sentido una espera activa cuando no hay otra CPU para liberar el bloqueo. Si un subproceso trata de adquirir un bloqueo y falla, siempre se bloquea para dar al propietario del bloqueo la oportunidad de ejecutarse y liberar el bloqueo.

Suponiendo que la espera activa y realizar un cambio de hilo son opciones viables, hay que hacer la siguiente concesión. La espera activa desperdicia ciclos de la CPU de manera directa; evaluar un bloqueo en forma repetida no es un trabajo productivo. Sin embargo, cambiar de hilo también desperdicia ciclos de la CPU, ya que se debe guardar el estado del hilo actual, se debe adquirir el bloqueo sobre la lista de procesos listos, hay que seleccionar un hilo, se debe cargar su estado y debe iniciarse. Además, la caché de la CPU contendrá todos los bloques incorrectos, por lo que ocurrirán muchos fallos de caché a medida que el nuevo hilo se empiece a ejecutar. También es probable que ocurran fallos de TLB. En un momento dado se realizará un cambio de vuelta al hilo original, y le seguirán más fallos de caché. Además de los fallos de caché, se desperdician los ciclos invertidos en estos dos cambios de contexto.

www.FreeLibros.me

Por ejemplo, si se sabe que los mutexes se contienen generalmente durante 50 µseg, y se requiere 1 mseg para cambiar del hilo actual, y 1 mseg para volver a cambiar posteriormente, es más eficiente una espera activa sobre el mutex. Por otra parte, si el mutex promedio se contiene durante 10 mseg, vale la pena el esfuerzo de realizar los dos cambios de contexto. El problema es que las regiones críticas pueden variar de manera considerable en cuanto a su duración, así que, ¿cuál método es mejor?

Un diseño es una espera activa todo el tiempo; un segundo diseño es siempre cambiar. Un tercer diseño es tomar una decisión por separado cada vez que nos encontramos con un mutex bloqueado. Al momento en que se tiene que tomar la decisión, no se sabe si es mejor realizar una espera activa o cambiar, pero para cualquier sistema dado, es posible realizar un rastreo de toda la actividad y analizarla después fuera de línea. Así, podemos saber en retrospectiva cuál decisión fue la mejor y cuánto tiempo se desperdició en el mejor caso. Este algoritmo de retrospección se convierte entonces en un punto de referencia contra el que se pueden medir los algoritmos viables.

Los investigadores han estudiado este problema (Karlin y colaboradores, 1989; Karlin y colaboradores, 1991; Ousterhout, 1982). La mayor parte del trabajo utiliza un modelo en el que un hilo que no puede adquirir un mutex gira durante cierto periodo. Si se excede este umbral, hace el cambio. En algunos casos se corrige el umbral, que por lo general es la sobrecarga conocida por cambiar a otro hilo y después cambiar de vuelta. En otros casos es dinámico, dependiendo de la historia observada del mutex sobre el que se espera.

Los mejores resultados se obtienen cuando el sistema lleva el rastro de los últimos tiempos de espera activa observados, y asume que éste será similar a los anteriores. Por ejemplo, suponiendo un tiempo de cambio de contexto de 1 mseg otra vez, un hilo realiza una espera activa por un máximo de 2 mseg, pero observe cuánto tiempo giró en realidad. Si no puede adquirir un bloqueo y detecta que en las tres ejecuciones anteriores esperó un promedio de 200 µseg, debe hacer una espera activa por 2 mseg antes de cambiar de hilo. Pero si detecta que la espera activa dura los 2 mseg completos en cada uno de los intentos anteriores, debe cambiar de inmediato y no realizar una espera activa. Encontrará más detalles en (Karlin y colaboradores, 1991).

8.1.4 Planificación de multiprocesadores

Antes de analizar la forma en que se realiza la planificación en los multiprocesadores, es necesario determinar *qué* es lo que se va a planificar. En los días de antaño, cuando todos los procesos tenían un solo hilo, los procesos eran los que se planificaban; no había nada más que se pudiera planificar. Todos los sistemas operativos modernos soportan procesos multihilo, lo cual complica aún más la planificación.

Es importante saber si los hilos son de kernel o de usuario. Si la ejecución de hilos se lleva a cabo mediante una biblioteca en espacio de usuario y el kernel no sabe nada acerca de los hilos, entonces la planificación se lleva a cabo con base en cada proceso, como siempre. Si el kernel ni siquiera sabe que los hilos existen, es muy difícil que pueda planificarlos.

Con los hilos del kernel sucede algo distinto. Aquí el kernel está al tanto de todos los hilos y puede elegir y seleccionar de entre los hilos que pertenecen a un proceso. En estos sistemas, la tendencia es que el kernel seleccione un hilo para ejecutarlo, y el proceso al que pertenece sólo

www.FreeLibros.me

tiene un pequeño papel (o tal vez ninguno) en el algoritmo de selección de hilos. A continuación hablaremos sobre la planificación de hilos, pero desde luego que, en un sistema con procesos con un solo hilo, o con hilos implementados en espacio de usuario, son los procesos los que se programan.

Proceso contra hilo no es la única cuestión de planificación. En un uniprocesador, la planificación es de una dimensión. La única pregunta que hay que responder (repetidas veces) es: "¿Cuál hilo se debe ejecutar a continuación?". En un multiprocesador, la planificación tiene dos dimensiones: el planificador tiene que decidir qué hilo ejecutar y en cuál CPU lo va a ejecutar. Esta dimensión adicional complica de manera considerable la planificación en los multiprocesadores.

Otro factor que complica las cosas es que, en ciertos sistemas, ninguno de los hilos está relacionado, mientras que en otros se dividen en grupos donde todos pertenecen a la misma aplicación y trabajan en conjunto. Un ejemplo del primer caso es un sistema de tiempo compartido, en el que usuarios independientes inician procesos independientes. Los hilos de los distintos procesos no están relacionados, y cada uno de ellos se puede planificar de manera independiente a los demás.

Un ejemplo del segundo caso ocurre con regularidad en los entornos de desarrollo de programas. A menudo, los sistemas extensos consisten de cierto número de archivos de encabezado que contienen macros, definiciones de tipos y declaraciones de variables que utilizan los archivos de código actuales. Cuando se modifica un archivo de encabezado, se deben volver a compilar todos los archivos de código que lo incluyen. El programa *make* se utiliza comúnmente para administrar el desarrollo. Cuando se invoca *make*, inicia la compilación sólo de aquellos archivos de código que se deben volver a compilar debido a las modificaciones realizadas en el encabezado o los archivos de código. Los archivos de código objeto que siguen siendo válidos no se regeneran.

La versión original de *make* realizaba su trabajo en forma secuencial, pero las versiones más recientes diseñadas para multiprocesadores pueden iniciar todas las compilaciones a la vez. Si se necesitan 10 compilaciones, no tiene caso programar 9 de ellas para que se ejecuten de inmediato y dejar la última hasta mucho después, ya que el usuario no percibirá el trabajo como terminado sino hasta que termine la última compilación. En este caso, tiene sentido considerar los hilos que realizan las compilaciones como si fueran un grupo, y tomar eso en cuenta al programarlos.

Tiempo compartido

Vamos a analizar el caso de la programación de hilos independientes; más adelante consideraremos cómo planificar hilos relacionados. El algoritmo de planificación más simple para lidiar con hilos no relacionados es tener una sola estructura de datos a nivel de sistema para los hilos que estén listos, que tal vez sea sólo una lista, pero es mucho más probable que sea un conjunto de listas para los hilos con distintas prioridades, como se ilustra en la figura 8-12(a). Aquí, las 16 CPUs están ocupadas en un momento dado, y hay un conjunto de 14 hilos por orden de prioridad, esperando ser ejecutados. La primera CPU en terminar su trabajo actual (o en hacer que su hilo se bloquee) es la CPU 4, que después bloquea las colas de programación y selecciona el hilo con mayor prioridad (*A*), como se muestra en la figura 8-12(b). A continuación, la CPU 12 queda inactiva y selecciona el hilo *B*, como se ilustra en la figura 8-12(c). Mientras que los hilos no estén relacionados de nin-

www.FreeLibros.me

guna manera, es una opción razonable llevar a cabo la planificación de esta forma, y es muy simple de implementar con eficiencia.

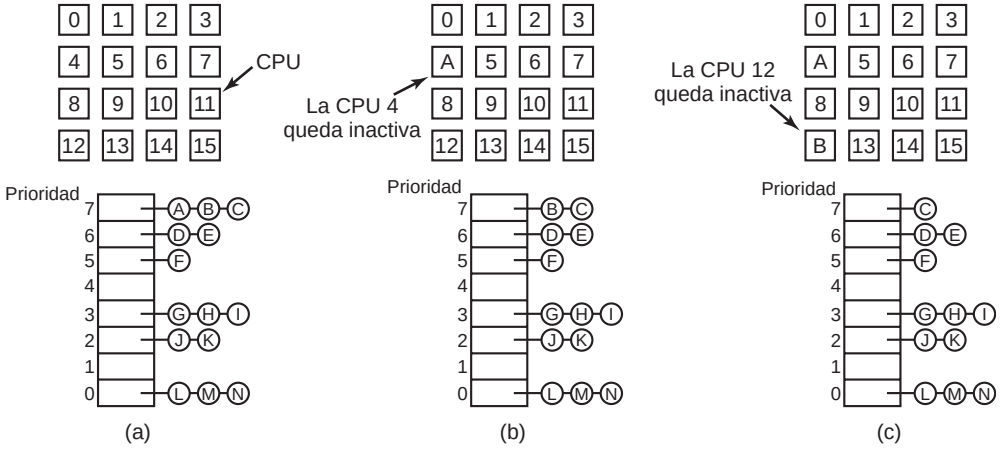


Figura 8-12. Uso de una sola estructura de datos para programar un multiprocesador.

Al tener una sola estructura de datos de planificación utilizada por todas las CPUs, éstas se comparten en el tiempo, de igual forma que como se hace en un sistema uniprocesador. También se proporciona un balance automático de la carga, ya que no puede haber una CPU inactiva mientras otras están sobrecargadas. Dos desventajas de este método son: la potencial contención por la estructura de datos que se utiliza en la programación a medida que aumenta el número de CPUs, y la sobrecarga usual que se produce al realizar un cambio de contexto, cuando un hilo se bloquea en espera de una operación de E/S.

También es posible que ocurra un cambio de contexto al momento en que expire el quantum de un hilo, por ejemplo, en un multiprocesador, que tiene ciertas propiedades que no están presentes en un uniprocesador. Suponga que el hilo contiene un bloqueo de espera activa al momento en que expira su quantum. Las otras CPUs que esperan a que se libere el bloqueo de espera activa sólo desperdician su tiempo en una espera activa hasta que se vuelve a programar ese hilo y se libera el bloqueo. En un uniprocesador, los bloqueos de espera activa se utilizan raras veces, por lo que si un proceso se suspende mientras contiene un mutex, y otro hilo se inicia y trata de adquirir el mutex, se bloqueará de inmediato, por lo que no se desperdicia mucho tiempo.

Para solucionar esta anomalía, algunos sistemas utilizan la **planificación inteligente**, en la cual un hilo que adquiere un bloqueo de espera activa establece una bandera a nivel de proceso, para mostrar que actualmente tiene un bloqueo de espera activa (Zahorjan y colaboradores, 1991). Al liberar el bloqueo, se borra la bandera. Así, el planificador no detiene un hilo que contenga un bloqueo de espera activa, sino que le da un poco más de tiempo para que complete su región crítica y libere el bloqueo.

Otra cuestión que desempeña un papel en la planificación es el hecho de que aunque todas las CPUs son iguales, ciertas CPUs son más iguales que otras. En especial, cuando el subproceso *A* se ha ejecutado durante mucho tiempo en la CPU *k*, la caché de esta CPU estará llena de bloques de *A*. Si *A* se vuelve a ejecutar poco tiempo después, tal vez su rendimiento sea mejor si se ejecuta en

www.FreeLibros.me

la CPU *k* debido a que la caché de esta CPU tal vez todavía contenga algunos de los bloques de *A*. Cuando se cargan previamente los bloques en la caché se incrementa la proporción de aciertos y, en consecuencia, la velocidad del hilo. Además, se puede dar el caso de que la TLB contenga las páginas correctas, con lo cual se reducen sus fallas.

Algunos multiprocesadores toman en cuenta este efecto y utilizan lo que se conoce como **planificación por afinidad** (Vaswani y Zahorjan, 1991). Aquí, la idea básica es esforzarse en serio por hacer que un hilo se ejecute en la misma CPU en que se ejecutó la última vez. Una manera de crear esta afinidad es mediante el uso de un **algoritmo de planificación de dos niveles**. Cuando se crea un hilo, éste se asigna a una CPU, por ejemplo con base en cuál de las CPUs tiene la menor carga en ese momento. Esta asignación de hilos a CPUs es el nivel superior del algoritmo. Como resultado de esta directiva, cada CPU adquiere su propia colección de hilos.

La planificación actual de los hilos es el nivel inferior del algoritmo. Cada CPU la realiza por separado, mediante el uso de prioridades o de algún otro medio. Al tratar de mantener un hilo en la misma CPU durante todo su tiempo de vida, se maximiza la afinidad de la caché. No obstante, si una CPU no tiene hilos qué ejecutar, obtiene uno de otra CPU en vez de quedar inactiva.

La planificación de dos niveles tiene tres beneficios. En primer lugar, distribuye la carga de una manera casi uniforme sobre las CPUs disponibles. En segundo lugar, se aprovecha la afinidad de la caché siempre que sea posible. En tercer lugar, al dar a cada CPU su propia lista de hilos listos se minimiza la contención por estas listas, ya que es muy poco frecuente que una CPU intente utilizar la lista de hilos listos de otra CPU.

Espacio compartido

El otro método general para la planificación de multiprocesadores se puede utilizar cuando los hilos están relacionados entre sí de alguna manera. Anteriormente mencionamos el ejemplo del programa *make* paralelo como uno de los casos. También ocurre con frecuencia que un solo proceso tenga varios hilos que trabajen en conjunto. Por ejemplo, si los hilos de un proceso se comunican con mucha frecuencia, es conveniente hacer que se ejecuten al mismo tiempo. A la planificación de varios hilos al mismo tiempo, esparcidos sobre varias CPUs, se le conoce como **espacio compartido**.

El algoritmo de espacio compartido más simple funciona de la siguiente manera. Suponga que se crea un grupo completo de hilos relacionados al mismo tiempo. Al momento de su creación, el programador comprueba si hay tantas CPUs libres como el número de hilos. Si las hay, a cada hilo se le proporciona su propia CPU dedicada (es decir, sin multiprogramación) y todos empiezan su ejecución. Si no hay suficientes CPUs, ninguno de los hilos se iniciará, sino hasta que haya suficientes CPUs disponibles. Cada hilo se aferra a su CPU hasta que termina, y en ese momento la CPU se regresa a la reserva de CPUs disponibles. Si un hilo se bloquea en espera de una operación de E/S, de todas formas retiene a la CPU, que simplemente se queda inactiva hasta que el hilo se despierta. Cuando aparece el siguiente lote de hilos, se aplica el mismo algoritmo.

En cualquier instante, el conjunto de CPUs se divide de manera estática en cierto número de particiones, cada una de las cuales ejecuta los hilos de un hilo. Por ejemplo, en la figura 8-13 hay particiones con 4, 6, 8 y 12 CPUs, y hay 2 CPUs sin asignar. Con el tiempo cambiará el número y el tamaño de las particiones, a medida que se creen nuevos hilos y cuando los anteriores terminen de ejecutarse.

www.FreeLibros.me

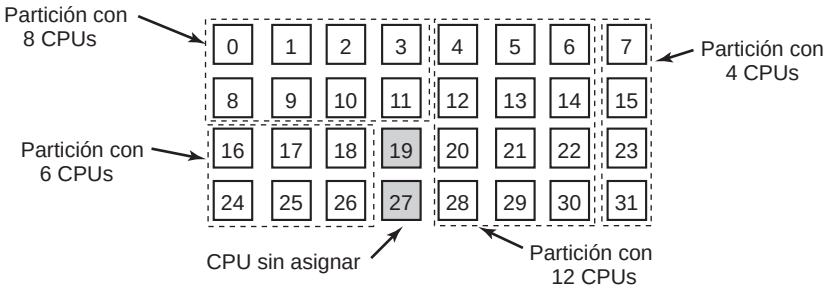


Figura 8-13. Un conjunto de 32 CPUs divididas en cuatro particiones, con dos CPUs disponibles.

De vez en cuando hay que tomar decisiones sobre planificación. En los sistemas con uniprocesador, el algoritmo "el trabajo más corto primero" es un algoritmo popular para la programación por lotes. El algoritmo análogo para un multiprocesador es seleccionar el proceso que necesita el menor número de ciclos de la CPU; es decir, el hilo cuyo valor de la multiplicación "cuenta de la CPU $\times$ tiempo de ejecución" sea el menor de todos. No obstante, en la práctica esta información raras veces está disponible, por lo que es difícil llevar a cabo el algoritmo. De hecho, los estudios han demostrado que, en la práctica, es difícil vencer al algoritmo "primero en llegar, primero en ser atendido" (Krueger y colaboradores, 1994).

En este modelo simple de particionamiento, un hilo sólo pide cierto número de CPUs y los obtiene todos, o tiene que esperar hasta que estén disponibles. Una metodología distinta es que los hilos administren de manera activa el grado de paralelismo. Un método para administrar el paralelismo es tener un servidor central que lleve la cuenta de cuáles hilos se están ejecutando, cuáles desean ejecutarse y cuáles son sus requerimientos de CPU mínimos y máximos (Tucker y Gupta, 1989). De vez en cuando, cada aplicación sondea al servidor central para saber cuántas CPUs puede utilizar. Después aumenta o reduce el número de hilos para ajustarse a lo que haya disponible. Por ejemplo, un servidor Web puede tener 5, 10, 20 o cualquier otro número de hilos ejecutándose en paralelo. Si en un momento dado tiene 10 hilos, aumenta la demanda de CPUs de manera repentina y el servidor Web tiene que reducir su número de hilos a 5, cuando los siguientes 5 hilos terminen su trabajo actual, tendrán que dejar de ejecutarse en vez de recibir más trabajo. Este esquema permite que los tamaños de las particiones varíen en forma dinámica para adaptarse a la carga de trabajo actual de una mejor forma que el sistema fijo de la figura 8-13.

Planificación por pandilla

Una ventaja clara de la compartición de espacio es la eliminación de la multiprogramación, que a su vez elimina la sobrecarga impuesta por el cambio de contexto. Sin embargo, una desventaja también clara es el tiempo que se desperdicia cuando una CPU se bloquea y no tiene nada que hacer hasta que vuelva a estar lista. En consecuencia, las personas han buscado algoritmos que intenten realizar la planificación tanto en tiempo como en espacio, en especial para los hilos que crean varios hilos, que por lo general se necesitan comunicar entre sí.

www.FreeLibros.me

Para ver el tipo de problema que puede ocurrir cuando los hilos de un proceso se planifican de manera independiente, considere un sistema con los hilos A_0 y A_1 que pertenecen al proceso *A*, y los hilos B_0 y B_1 que pertenecen al proceso *B*. Los hilos A_0 y B_0 comparten el tiempo en la CPU 0; los hilos A_1 y B_1 comparten el tiempo en la CPU 1. Los hilos A_0 y A_1 necesitan comunicarse con frecuencia. El patrón de comunicación es que A_0 envía un mensaje a A_1, y después A_1 envía de regreso una respuesta a A_0, seguida de otra secuencia similar. Suponga que A_0 y B_1 se inician primero, como se muestra en la figura 8-14.

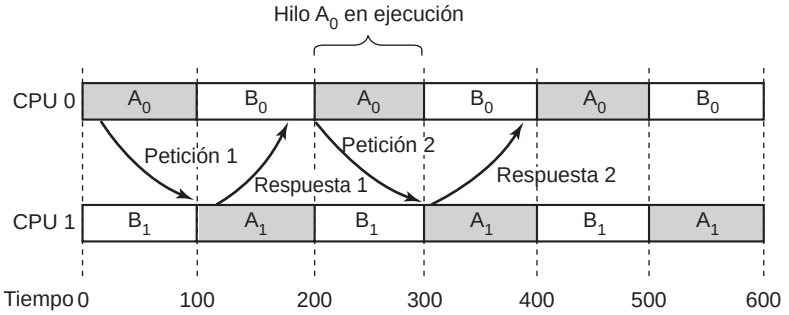


Figura 8-14. Comunicación entre dos hilos que pertenecen al hilo *A*, los cuales se están ejecutando fuera de fase.

En el intervalo 0, A_0 envía una petición a A_1, pero A_1 no la recibe sino hasta que se ejecuta en el intervalo 1, el cual empieza en los 100 mseg. Envía la respuesta de inmediato, pero A_0 no la recibe sino hasta que se vuelve a ejecutar de nuevo, a los 200 mseg. El resultado neto es una secuencia de solicitud-respuesta cada 200 mseg. No es algo muy bueno.

La solución a este problema es la **planificación por pandilla**, que es una consecuencia de la **co-planificación** (Ousterhout, 1982). La planificación por pandilla consta de tres partes:

1. Los grupos de hilos relacionados se programan como una unidad, o pandilla.
2. Todos los miembros de una plantilla se ejecutan en forma simultánea, en distintas CPUs de tiempo compartido.
3. Todos los miembros de la pandilla inician y terminan sus intervalos en conjunto.

El truco que hace funcionar la planificación por pandilla es que todas las CPUs se planifican en forma asíncrona. Esto significa que el tiempo se divide en quantums discretos, como en la figura 8-14. Al inicio de cada nuevo quantum *todas* las CPUs se reprograman, y se inicia un nuevo hilo en cada CPU. Al inicio del siguiente quantum ocurre otro evento de planificación. En el intervalo no se realiza ninguna planificación. Si un hilo se bloquea, su CPU permanece inactiva hasta el final del quantum.

En la figura 8-15 se muestra un ejemplo del funcionamiento de la planificación por pandilla. Aquí tenemos un multiprocesador con seis CPUs que son utilizadas por cinco procesos (*A* a *E*), con un total de 24 hilos listos para ejecutarse. Durante la ranura de tiempo 0, se programan los hilos A_0

www.FreeLibros.me

a A_6 y se ejecutan. Durante la ranura de tiempo 1, se programan los hilos B_0, B_1, B_2, C_0, C_1 y C_2, y se ejecutan. Durante la ranura de tiempo 2, se ejecutan los cinco hilos de D y E_0. Los seis hilos restantes que pertenecen al hilo E se ejecutan en la ranura de tiempo 3. Después se repite el ciclo, donde la ranura 4 es igual a la ranura 0, y así en lo sucesivo.

Ranura de tiempo	CPU 0	CPU 1	CPU 2	CPU 3	CPU 4	CPU 5
0	A_0	A_1	A_2	A_3	A_4	A_5
1	B_0	B_1	B_2	C_0	C_1	C_2
2	D_0	D_1	D_2	D_3	D_4	E_0
3	E_1	E_2	E_3	E_4	E_5	E_6
4	A_0	A_1	A_2	A_3	A_4	A_5
5	B_0	B_1	B_2	C_0	C_1	C_2
6	D_0	D_1	D_2	D_3	D_4	E_0
7	E_1	E_2	E_3	E_4	E_5	E_6

Figura 8-15. Planificación por pandilla.

La idea de la planificación por pandilla es que todos los hilos de un hilo se ejecuten en conjunto, de manera que si uno de ellos envía una petición a otro, recibirá el mensaje casi de inmediato y podrá responder también casi de inmediato. En la figura 8-15, como todos los hilos de *A* se están ejecutando en conjunto, durante un quantum pueden enviar y recibir una gran cantidad de mensajes, con lo cual se elimina el problema de la figura 8-14.

8.2 MULTICOMPUTADORAS

Los multiprocesadores son populares y atractivos debido a que ofrecen un modelo de comunicación simple: todas las CPUs comparten una memoria común. Los procesos pueden escribir mensajes en la memoria, para que después otros procesos los lean. La sincronización se puede realizar mediante el uso de mutexes, semáforos, monitores y otras técnicas bien establecidas. El único contratiempo es la dificultad de construir multiprocesadores grandes, además de que es un proceso costoso.

Para resolver estos problemas se ha investigado mucho sobre las **multicomputadoras**: CPUs con acoplamiento fuerte que no comparten memoria. Cada una tiene su propia memoria, como se muestra en la figura 8-1(b). Estos sistemas también son conocidos por una variedad de nombres, incluyendo **clúster de computadoras** y **COWS** (*Clusters of Workstations*, Clústeres de estaciones de trabajo).

Es fácil construir las multicomputadoras, ya que el componente básico es una PC que tiene sólo los componentes esenciales, además de una tarjeta de interfaz de red de alto rendimiento. Desde luego que el secreto para obtener un alto rendimiento es diseñar de manera inteligente la red de in-

www.FreeLibros.me

terconexión y la tarjeta de interfaz. Este problema es completamente análogo al de construir la memoria compartida en un multiprocesador. Sin embargo, el objetivo es enviar mensajes en una escala de tiempo en microsegundos, en vez de acceder a la memoria en una escala de tiempo de nanosegundos, por lo que es más simple, económico y fácil de lograr.

En las siguientes secciones, primero analizaremos con brevedad el hardware de una multicomputadora, en especial el hardware de interconexión. Luego pasaremos al software: empezaremos con el software de comunicación de bajo nivel y continuaremos con el de comunicación de alto nivel. También analizaremos una manera en que se puede obtener memoria compartida en sistemas que no la tienen. Por último examinaremos la planificación y el equilibrio de carga.

8.2.1 Hardware de una multicomputadora

El nodo básico de una multicomputadora consiste en una CPU, la memoria, una interfaz de red y algunas veces un disco duro. El nodo puede estar empaquetado en un gabinete de PC estándar, pero el adaptador de gráficos, el monitor, el teclado y el ratón casi nunca están presentes. En algunos casos la PC contiene un tablero de multiprocesador de 2 o 4 vías, en donde cada nodo posiblemente tiene un chip de doble o cuádruple núcleo en vez de una sola CPU, pero por cuestión de simplicidad vamos a suponer que cada nodo tiene una CPU. A menudo se conectan cientos, o incluso miles de nodos entre sí, para formar una multicomputadora. A continuación hablaremos un poco acerca de la organización de este hardware.

Tecnología de interconexión

Cada nodo tiene una tarjeta de interfaz de red, de la cual salen uno o dos cables (o fibras). Estos cables se conectan a otros nodos o a switches. En un sistema pequeño, puede haber un switch al que están conectados todos los nodos en la topología de estrella de la figura 8-16(a). Las redes Ethernet modernas con switches utilizan esta topología.

Como alternativa al diseño de un solo switch, los nodos pueden formar un anillo con dos cables que provienen de la tarjeta de interfaz de red: uno que entra al nodo por la izquierda y otro que sale del nodo por la derecha, como se muestra en la figura 8-16(b). En esta topología no se necesitan switches, por lo cual no se muestra ninguno.

La **rejilla** o **malla** de la figura 8-16(c) es un diseño bidimensional que se ha utilizado en muchos sistemas comerciales. Es muy regular y tiene la facilidad de poder escalar a tamaños mayores. Tiene un **diámetro**, que es la ruta más larga entre dos nodos cualesquiera, y que aumenta sólo con base en la raíz cuadrada del número de nodos. Una variante de la malla es el **doble toroide** de la figura 8-16(d), una rejilla con los bordes conectados. No sólo es más tolerante a fallas que la rejilla, sino que el diámetro es también menor debido a que las esquinas opuestas ahora se pueden comunicar en sólo dos saltos.

El **cubo** de la figura 8-16(e) constituye una topología tridimensional regular. Hemos ilustrado un cubo de $2 \times 2 \times 2$, pero en el caso más general podría ser un cubo de $k \times k \times k$. En la figura 8-16(f) tenemos un cubo de cuatro dimensiones, construido a partir de dos cubos tridimensionales con los correspondientes nodos conectados. Podríamos hacer un cubo de cinco dimensiones si

www.FreeLibros.me

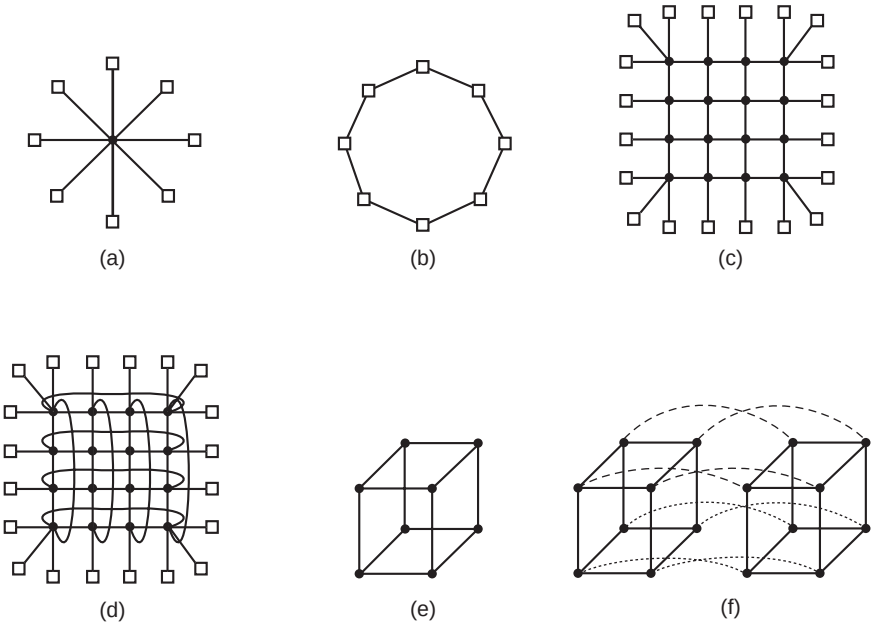


Figura 8-16. Varias topologías de interconexión. (a) Un solo switch. (b) Un anillo. (c) Una rejilla. (d) Un dobel toroide. (e) Un cubo. (f) Un hipercubo 4D.

clonamos la estructura de la figura 8-16(f) y conectamos los nodos correspondientes, para formar un bloque de cuatro cubos. Para obtener seis dimensiones, podríamos duplicar el bloque de cuatro cubos e interconectar los nodos correspondientes, y así en lo sucesivo. A un cubo de *n* dimensiones que se forma así se le conoce como **hipercubo**. Muchas computadoras paralelas utilizan esta topología debido a que el diámetro aumenta en proporción lineal con las dimensiones. Dicho de otra forma, el diámetro es el logaritmo de base 2 del número de nodos; por ejemplo, un hipercubo de 10 dimensiones tiene 1024 nodos pero un diámetro de sólo 10, con lo cual se obtienen excelentes propiedades de retraso. Observe que por el contrario, si se organizan 1024 nodos como una rejilla de 32×32 se tendrá un diámetro de 62, lo cual es más de seis veces peor que el hipercubo. La desventaja de un diámetro más pequeño es que el factor de salida (*fanout*) y por ende el número de vínculos (y el costo), son mucho mayores para el hipercubo.

En las multicomputadoras se utilizan dos tipos de esquemas de conmutación. En el primero, cada mensaje primero se divide (ya sea que lo haga el usuario o la interfaz de red) en un trozo de cierta longitud máxima, conocido como **paquete**. El esquema de conmutación, conocido como **almacenamiento de conmutación de paquetes y retransmisión**, consiste en inyectar el paquete en el primer switch mediante la tarjeta de interfaz de red del nodo de origen, como se muestra en la figura 8-17(a). Los bits llegan uno a la vez, y cuando llega todo el paquete en un búfer de entrada, se copia a la línea que conduce al siguiente switch a lo largo de la ruta, como se muestra en la figura 8-17(b). Cuando llega el paquete al switch conectado al nodo de destino, como se muestra en la

www.FreeLibros.me